

C++

Introduzione a C++

Cos'è il C++, come funziona e perché impararlo.

Perché ti serve il C++?

Ogni volta che giochi a un videogioco, usi un browser, o guardi un film in streaming, stai usando software scritto (almeno in parte) in C++.

Il C++ è il linguaggio che si usa quando le prestazioni contano davvero: quando un programma deve essere velocissimo, consumare poca memoria, o controllare direttamente l'hardware del computer.

Imparare il C++ ti dà una comprensione profonda di come funziona davvero un computer — e questa conoscenza ti tornerà utile qualunque linguaggio userai in futuro.

Cos'è il C++?

Il **C++** è un linguaggio di programmazione creato da **Bjarne Stroustrup** nei primi anni '80. Il nome ("C più più") indica che è un'evoluzione del linguaggio C, con molte funzionalità aggiunte.

Lo trovi ovunque:

- **Sistemi operativi** — parti di Windows e Linux sono scritte in C++
 - **Videogiochi** — motori grafici come Unreal Engine usano C++
 - **Browser** — Chrome e Firefox hanno il motore di rendering scritto in C++
 - **Software scientifico e finanziario** — dove la velocità è fondamentale
 - **Dispositivi embedded** — microcontrollori, robot, sistemi industriali
-

C++ vs Python: le differenze principali

Se hai già studiato Python, ecco le differenze più importanti:

Caratteristica	C++	Python
Come funziona	Compilato — tradotto prima di eseguire	Interpretato — tradotto durante l'esecuzione
Velocità	Molto veloce	Più lento
Sintassi	Più precisa e rigida	Più semplice e flessibile
Tipi delle variabili	Obbligatori da dichiarare	Non obbligatori
Gestione della memoria	Spesso manuale	Automatica

In C++ devi essere più preciso: devi dichiarare il tipo di ogni variabile, chiudere ogni istruzione con `;`, e fare attenzione a come usi la memoria. In cambio, ottieni programmi molto più veloci e un controllo totale su ciò che fa il computer.

Come funziona il C++?

Il C++ è un linguaggio **compilato**. Significa che, prima di eseguire il programma, il codice che scrivi viene trasformato in istruzioni dirette per il processore da un programma chiamato **compilatore**.

Il flusso di lavoro è:

1. Scrivi il codice in un file con estensione `.cpp`
2. Il compilatore legge il file e lo trasforma in un programma eseguibile (`.exe` su Windows, senza estensione su Linux e macOS)
3. Esegui il programma

Questo processo rende i programmi C++ molto veloci: il processore esegue direttamente le istruzioni già tradotte nella sua lingua, senza dover interpretare il codice ogni volta.

Il primo programma

Per tradizione, il primo programma che si scrive in qualsiasi linguaggio è "Hello, World!" — un programma che stampa un saluto sullo schermo:

```
#include <iostream> // include la libreria per stampare
using namespace std; // semplifica la scrittura del codice

int main() { // il punto di partenza del programma
    cout << "Hello, World!" << endl; // stampa il messaggio
    return 0; // segnala che il programma è finito correttamente
}
```

Output:

Hello, World!

Non preoccuparti se non capisci ancora tutte le parti: nelle prossime sezioni spiegheremo ogni elemento in dettaglio, passo dopo passo.

Le versioni del C++

Il C++ viene aggiornato periodicamente con nuove funzionalità. Le versioni principali sono:

- **C++98** — la prima versione standardizzata
- **C++11** — un grande aggiornamento con molte funzionalità moderne
- **C++14, C++17, C++20** — miglioramenti successivi

In questo manuale useremo le funzionalità moderne (C++11 e successive), che sono quelle usate nella pratica oggi.

Da dove continuare

Se parti da zero, questo percorso ti evita salti troppo grandi:

1. [Installazione](#)
2. [Struttura di un programma C++](#)
3. [Sintassi di base](#)
4. [Variabili](#)
5. [Tipi di dati](#)
6. [Condizioni](#)
7. [Ciclo for](#)
8. [Funzioni](#)

Dopo le basi, puoi passare a [array](#), [stringhe](#), [puntatori](#) e [classi e oggetti](#).

Organizzazione del Codice

Come dividere un programma C++ in più file usando header file e file di implementazione.

Organizzare il codice in più file

Quando scrivi un programma piccolo, un unico file `.cpp` va benissimo. Ma immagina di dover scrivere un gestionale scolastico con centinaia di funzioni: tutto in un file solo sarebbe un disastro. Trovare una funzione richiederebbe minuti.

Dividere il codice in più file risolve questo problema. Ogni file ha uno scopo preciso, come i capitoli di un libro.

Vantaggi:

- **Leggibilità:** ogni file è focalizzato su un argomento
- **Riutilizzo:** puoi usare le stesse funzioni in più programmi
- **Manutenzione:** modifichi una parte senza toccare il resto
- **Collaborazione:** più persone lavorano su file diversi senza interferirsi

I due tipi di file

In C++ il codice viene diviso in due tipi di file:

Tipo	Estensione	Cosa contiene
Header	<code>.h</code> o <code>.hpp</code>	Le dichiarazioni (cosa esiste)
Implementazione	<code>.cpp</code>	Le definizioni (come funziona)

Il file header è come il **sommario di un libro**: ti dice cosa c'è, ma non ti mostra tutto il contenuto. Il file `.cpp` è il contenuto vero e proprio.

Esempio pratico

Immaginiamo di creare una piccola libreria con funzioni matematiche personalizzate.

Struttura dei file

```
progetto/  
├─ main.cpp  
├─ matematica.h  
└─ matematica.cpp
```

Il file header: **matematica.h**

Contiene solo le **firme** delle funzioni (senza il corpo):

```
// matematica.h – le dichiarazioni  
  
#ifndef MATEMATICA_H // protezione dall'inclusione multipla (spiegata più  
sotto)  
#define MATEMATICA_H  
  
int somma(int a, int b);  
int prodotto(int a, int b);  
double potenza(double base, int esponente);  
  
#endif
```

Il file di implementazione: **matematica.cpp**

Contiene il codice vero delle funzioni:

```
// matematica.cpp - le implementazioni

#include "matematica.h" // include il proprio header

int somma(int a, int b) {
    return a + b;
}

int prodotto(int a, int b) {
    return a * b;
}

double potenza(double base, int esponente) {
    double risultato = 1.0;
    for (int i = 0; i < esponente; i++) {
        risultato *= base;
    }
    return risultato;
}
```

Il file principale: `main.cpp`

Include l'header per sapere che le funzioni esistono:

```
// main.cpp
#include <iostream>
#include "matematica.h" // include il nostro header (virgolette, non <>)
using namespace std;

int main() {
    cout << "Somma: " << somma(3, 4) << endl;
    cout << "Prodotto: " << prodotto(3, 4) << endl;
    cout << "2^8 = " << potenza(2, 8) << endl;
    return 0;
}
```

Per compilare, passa **tutti** i file `.cpp` al compilatore:

```
g++ main.cpp matematica.cpp -o programma
./programma
```

Le include guard: protezione dall'inclusione multipla

Nell'header hai visto queste righe "strane":

```
#ifndef MATEMATICA_H
#define MATEMATICA_H

// ... contenuto ...

#endif
```

Si chiamano **include guard**. Servono a evitare un errore comune: se lo stesso header venisse incluso due volte nello stesso file, il compilatore si lamenterebbe di funzioni "ridefinite".

Come funzionano:

1. `#ifndef MATEMATICA_H` — "se MATEMATICA_H non è ancora definito..."
2. `#define MATEMATICA_H` — "...definiscilo..."
3. "...ed esegui tutto questo contenuto"
4. `#endif` — fine del blocco

La seconda volta che il file viene incluso, `MATEMATICA_H` è già definito, quindi tutto il contenuto viene saltato automaticamente.

Alternativa moderna: `#pragma once`

```
#pragma once // includi questo file una sola volta

// contenuto dell'header...
```

`#pragma once` fa la stessa cosa delle include guard, ma è più semplice da scrivere. È supportata da tutti i compilatori moderni.

Come includere file

Ci sono due sintassi per `#include` :

```
#include <iostream>    // parentesi angolari → librerie di sistema
#include "matematica.h" // virgolette → file del tuo progetto
```

Esempio con una classe

Le classi si dividono esattamente nello stesso modo: la dichiarazione nell'header, i metodi nel `.cpp` .

studente.h

```
#pragma once

#include <string>
using namespace std;

class Studente {
private:
    string nome;
    int eta;
    double votoMedio;

public:
    Studente(string n, int e);           // solo le firme dei metodi
    void aggiungiVoto(double voto);
    void stampaInfo() const;
    string getNome() const;
    double getVotoMedio() const;
};
```


studente.cpp

```
#include "studente.h"
#include <iostream>
using namespace std;

// L'implementazione dei metodi usa NomeClasse:: davanti al nome del metodo
Studente::Studente(string n, int e) {
    nome = n;
    eta = e;
    votoMedio = 0.0;
}

void Studente::aggiungiVoto(double voto) {
    if (votoMedio == 0.0) {
        votoMedio = voto;
    } else {
        votoMedio = (votoMedio + voto) / 2.0;
    }
}

void Studente::stampaInfo() const {
    cout << "Nome: " << nome << ", Eta: " << eta
        << ", Media: " << votoMedio << endl;
}

string Studente::getNome() const { return nome; }
double Studente::getVotoMedio() const { return votoMedio; }
```

main.cpp

```
#include <iostream>
#include "studente.h"
using namespace std;

int main() {
    Studente s("Mario", 16);
    s.aggiungiVoto(8.5);
    s.aggiungiVoto(7.0);
    s.aggiungiVoto(9.0);
    s.stampaInfo();
    return 0;
}
```

Compilazione:

```
g++ main.cpp studente.cpp -o programma
```

Cosa va dove

Nell'header (`.h`) metti:

- Le firme delle funzioni (prototipi)
- Le dichiarazioni delle classi
- Le definizioni di `struct` ed `enum`
- Le costanti (`const`)

Nel file `.cpp` metti:

- Il corpo delle funzioni
- Il corpo dei metodi delle classi
- La logica del programma

Regola pratica: se qualcuno ha bisogno di *sapere che esiste*, mettilo nell'header. Se qualcuno ha bisogno di *vedere come funziona*, mettilo nel `.cpp`.

Errori comuni

```
// Dimenticare le include guard → errori di ridefinizione
// Soluzione: aggiungi #pragma once o #ifndef

// Includere il .cpp invece dell'header
#include "matematica.cpp" // SBAGLIATO
#include "matematica.h"   // CORRETTO

// Dimenticare di compilare tutti i file .cpp
g++ main.cpp -o programma // SBAGLIATO — manca matematica.cpp
g++ main.cpp matematica.cpp -o programma // CORRETTO
```

Compilazione e Linking

Come funziona il processo di trasformazione del codice C++ in un programma eseguibile.

Come funziona la compilazione?

Quando premi "compila" nel tuo editor, sembra che il programma parta magicamente. In realtà, dietro le quinte succedono tre cose distinte.

Capire questo processo ti aiuta a:

- **interpretare gli errori** — ci sono errori di compilazione ed errori di linking, e si risolvono in modi diversi
- **lavorare con progetti grandi** — con più file `.cpp`, devi sapere come metterli insieme
- **usare il terminale** — spesso si compila da riga di comando, specialmente in ambito professionale

Le tre fasi: dalla ricetta al piatto

Pensa al tuo codice come a una ricetta scritta in italiano. Per "eseguirlo" su un computer, bisogna:

1. **Preprocessing** — il preprocessore prepara gli ingredienti (espande `#include`, sostituisce le macro `#define`, ecc.)
2. **Compilazione** — il compilatore traduce la ricetta in lingua macchina

3. **Linking** — il linker unisce tutti i pezzi in un unico programma eseguibile

Fase 1: Preprocessing

Il preprocessore elabora il tuo codice

prima

della compilazione. Sostituisce ogni `#include` con il contenuto del file incluso, espande le macro e gestisce i blocchi `#ifdef`.

```
#include <iostream>    // viene sostituito con tutto il contenuto di
                        iostream
#define PI 3.14159     // ogni occorrenza di PI viene sostituita con 3.14159

int main() {
    // dopo il preprocessing questa riga diventa:
    // double area = 3.14159 * 5 * 5;
    double area = PI * 5 * 5;
    return 0;
}
```

Puoi vedere il risultato del preprocessing con il flag `-E` :

```
g++ -E main.cpp -o main.i
```

Il file `main.i` contiene il codice dopo il preprocessing — di solito è molto lungo perché include tutto il contenuto degli header.

Fase 2: Compilazione

Il compilatore prende il codice preprocessato e lo trasforma in **codice oggetto** — un file in linguaggio macchina con estensione `.o` (su Linux e macOS) o `.obj` (su Windows).

Il codice oggetto contiene istruzioni che il processore capisce, ma non è ancora un programma completo: mancano i collegamenti alle funzioni scritte in altri file.

```
# Compila senza collegare (-c = solo compilazione)
g++ -c main.cpp -o main.o
g++ -c matematica.cpp -o matematica.o
```

Fase 3: Linking (collegamento)

Il linker prende tutti i file oggetto e li **unisce** in un unico programma eseguibile. Risolve i "ponti" tra i file: se `main.cpp` chiama la funzione `somma()` definita in `matematica.cpp`, il linker collega le due parti.

```
# Collega i file oggetto in un unico eseguibile
g++ main.o matematica.o -o programma
```

Il comando completo con g++

Di solito fai tutto in un unico comando, senza pensare alle fasi:

```
g++ main.cpp matematica.cpp -o programma
```

Questo comando esegue automaticamente tutte e tre le fasi per ogni file `.cpp` e poi collega tutto insieme.

I flag di g++ più utili

I **flag** sono opzioni che puoi aggiungere al comando per modificarne il comportamento. Iniziano sempre con `-`.

Il nome dell'output

```
g++ file.cpp -o mio_programma    # l'eseguibile si chiamerà "mio_programma"
g++ file.cpp                     # senza -o, l'eseguibile si chiama "a.out"
```

Warning — gli avvisi del compilatore

I warning non sono errori: il programma compila lo stesso, ma il compilatore ti avvisa di cose sospette.

```
g++ -Wall file.cpp -o output      # abilita la maggior parte dei warning
g++ -Wextra file.cpp -o output    # warning aggiuntivi
g++ -Wall -Wextra file.cpp -o output # entrambi – consigliato!
```

Esempi di warning utili:

```
int x;           // warning: x usata senza essere inizializzata
if (x = 5)        // warning: probabilmente volevi == invece di =
```

Leggi sempre i warning e correggili — anche se il programma funziona.

Debug — per trovare gli errori

```
g++ -g file.cpp -o output    # include informazioni di debug nel programma
```

Con `-g` puoi usare strumenti come `gdb` per eseguire il programma passo passo e vedere cosa succede.

Ottimizzazione — per rendere il programma più veloce

```
g++ -O0 file.cpp -o output # nessuna ottimizzazione (utile durante lo sviluppo)
g++ -O1 file.cpp -o output # ottimizzazione leggera
g++ -O2 file.cpp -o output # ottimizzazione moderata (consigliata per la versione finale)
g++ -O3 file.cpp -o output # ottimizzazione aggressiva
```

Usa `-O0` durante lo sviluppo e il debug. Usa `-O2` quando vuoi distribuire il programma.

Standard C++ — scegliere la versione del linguaggio

```
g++ -std=c++11 file.cpp -o output # C++11
g++ -std=c++14 file.cpp -o output # C++14
g++ -std=c++17 file.cpp -o output # C++17 (buona scelta moderna)
g++ -std=c++20 file.cpp -o output # C++20
```

Specifica sempre lo standard se usi funzionalità moderne. C++17 è una buona scelta oggi:

```
g++ -std=c++17 -Wall -Wextra main.cpp -o programma
```

Aggiungere cartelle per gli header

```
g++ -I./include file.cpp -o output # cerca gli header anche nella cartella "include"
g++ -I /percorso/assoluto file.cpp -o output
```

Errori di compilazione vs errori di linking

Saper distinguere i due tipi di errore ti fa trovare il problema molto più in fretta.

Errori di compilazione

Riguardano la **sintassi** o la **logica** del tuo codice. Il compilatore non capisce cosa hai scritto:

```
main.cpp:5:10: error: 'somma' was not declared in this scope
```

Hai usato `somma` senza dichiararla. Manca il prototipo della funzione o l' `#include` dell'header giusto.

```
main.cpp:8:5: error: expected ';' before '}' token
```

Hai dimenticato un punto e virgola.

Errori di linking

Riguardano **riferimenti non risolti** tra file diversi. Il compilatore conosce la funzione, ma il linker non trova la sua definizione:

```
undefined reference to `somma(int, int)'
```

Significa: hai dichiarato `somma` (il compilatore è felice), ma non hai dato al linker il file `.cpp` che la definisce.

Soluzione: aggiungi il file mancante al comando:

```
# SBAGLIATO: manca matematica.cpp
g++ main.cpp -o programma

# CORRETTO
g++ main.cpp matematica.cpp -o programma
```


Esempio pratico con più file

Struttura del progetto:

```
progetto/  
├─ main.cpp  
├─ studente.h  
└─ studente.cpp
```

Compilazione in un unico comando:

```
g++ -std=c++17 -Wall main.cpp studente.cpp -o programma
```

Oppure in due fasi (utile nei progetti grandi — ricompili solo i file modificati):

```
# Compila i singoli file  
g++ -std=c++17 -c main.cpp -o main.o  
g++ -std=c++17 -c studente.cpp -o studente.o  
  
# Collega tutto  
g++ main.o studente.o -o programma
```

Makefile — automatizzare la compilazione

Nei progetti con molti file, riscrivere il comando `g++` ogni volta è noioso e rischioso. Il **Makefile** è un file che descrive come compilare il progetto automaticamente.

```
# Makefile
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra

programma: main.o studente.o
    $(CXX) main.o studente.o -o programma

main.o: main.cpp studente.h
    $(CXX) $(CXXFLAGS) -c main.cpp

studente.o: studente.cpp studente.h
    $(CXX) $(CXXFLAGS) -c studente.cpp

clean:
    rm -f *.o programma
```

Per compilare, scrivi solo:

```
make
```

Per cancellare i file compilati:

```
make clean
```

Non devi padroneggiare il Makefile subito — ma è utile sapere che esiste.

Riepilogo dei flag principali

Flag	Cosa fa
<code>-o nome</code>	Specifica il nome del file di output
<code>-Wall</code>	Abilita i warning principali
<code>-Wextra</code>	Warning aggiuntivi
<code>-g</code>	Aggiunge informazioni per il debug
<code>-O2</code>	Ottimizzazione (consigliata per la versione finale)
<code>-std=c++17</code>	Usa lo standard C++17
<code>-c</code>	Solo compilazione, senza collegamento
<code>-I percorso</code>	Aggiunge una cartella dove cercare gli header
<code>-E</code>	Solo preprocessing

Comando consigliato durante lo studio

Usa sempre questo comando quando studi e sviluppi:

```
g++ -std=c++17 -Wall -Wextra file.cpp -o programma
```

I warning sono tuoi alleati: leggili con attenzione e correggili sempre, anche quando il programma compila senza errori.

Namespace

Cosa sono i namespace e come si usa std in C++.

Cos'è un namespace?

Immagina che due studenti nella stessa classe si chiamino entrambi "Marco". Ogni volta che la prof dice "Marco!", nasce confusione. La soluzione? Usare il cognome: "Marco Rossi" e "Marco Bianchi".

I **namespace** in C++ funzionano allo stesso modo. Se due librerie diverse definiscono una funzione chiamata `print`, il compilatore non saprebbe quale usare. Con i namespace, puoi distinguerle: `LibreriaA::print` e `LibreriaB::print`.

Il namespace `std`

Tutto quello che fa parte della libreria standard del C++ si trova nel namespace `std` (abbreviazione di "standard"). Per usare `cout`, devi indicare che appartiene a `std`:

```
std::cout << "Ciao!" << std::endl;
std::string nome = "Alice";
```

Il `::` si legge "appartenente a". Quindi `std::cout` significa "cout appartenente a std".

using namespace std : la scorciatoia

Scrivere `std::` ogni volta è scomodo. Con `using namespace std;` importi tutto il namespace `std` e puoi usarlo senza prefisso:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Ciao!" << endl; // invece di std::cout e std::endl
    string nome = "Alice";   // invece di std::string
    return 0;
}
```

Questa è la forma che hai visto in quasi tutto il codice di questo manuale.

using per nomi specifici

Puoi importare solo i nomi che ti servono, invece di tutto il namespace:

```
#include <iostream>
using std::cout;
using std::endl;
using std::string;

int main() {
    cout << "Ciao!" << endl;    // OK - importato
    // cin >> x;                  // ERRORE - cin non è stato importato
    return 0;
}
```

Questo è più preciso ed è la scelta preferita nei progetti grandi.

Creare i tuoi namespace

Puoi usare i namespace per organizzare il tuo codice ed evitare conflitti tra funzioni con lo stesso nome:

```

namespace Matematica {
    const double PI = 3.14159265358979;

    double area(double raggio) {
        return PI * raggio * raggio;    // area del cerchio
    }

    double perimetro(double raggio) {
        return 2 * PI * raggio;
    }
}

namespace Geometria {
    double area(double base, double altezza) {
        return base * altezza;    // area del rettangolo
    }
}

int main() {
    // Nessun conflitto – il compilatore sa qual è quale
    cout << Matematica::area(5.0) << endl;    // area del cerchio
    cout << Geometria::area(4.0, 6.0) << endl; // area del rettangolo
    cout << Matematica::PI << endl;
    return 0;
}

```

Namespace annidati

Puoi mettere namespace dentro altri namespace, come cartelle dentro cartelle:

```

namespace Scuola {
    namespace Matematica {
        double calcolaMedia(double a, double b) {
            return (a + b) / 2;
        }
    }

    namespace Italiano {
        string maiuscolo(string s) {
            return s; // semplificato
        }
    }
}

int main() {
    cout << Scuola::Matematica::calcolaMedia(8, 9) << endl; // 8.5
    return 0;
}

```

In C++17 puoi scrivere i namespace annidati in modo più compatto:

```

namespace Scuola::Matematica {
    double calcolaMedia(double a, double b) {
        return (a + b) / 2;
    }
}

```

Alias per namespace lunghi

Se un namespace ha un nome lungo, puoi dargli un soprannome:

```

namespace MioSistemaGestioneStudenti {
    void aggiungiStudiante(string nome) { /* ... */ }
}

namespace MSGS = MioSistemaGestioneStudenti;    // soprannome

int main() {
    MSGS::aggiungiStudiante("Alice");    // molto più comodo!
    return 0;
}

```

Regola importante per gli header

Non scrivere mai `using namespace std;` dentro un file header (`.h`). Chi include il tuo header si ritroverebbe importato tutto `std` senza saperlo — potrebbe causare conflitti di nomi inaspettati:

```

// mio_header.h – SBAGLIATO
using namespace std;    // "inquina" il namespace di chi include questo file

// mio_header.h – CORRETTO
void funzione(std::string s);    // usa il prefisso esplicito

```


Esempio pratico: calcolo geometrico

```
#include <iostream>
#include <cmath>
using namespace std;

namespace Cerchio {
    const double PI = 3.14159265358979;
    double area(double r) { return PI * r * r; }
    double perimetro(double r) { return 2 * PI * r; }
}

namespace Rettangolo {
    double area(double b, double a) { return b * a; }
    double perimetro(double b, double a) { return 2 * (b + a); }
    double diagonale(double b, double a) { return sqrt(b*b + a*a); }
}

int main() {
    double r = 5.0;
    cout << "Cerchio con raggio " << r << ":" << endl;
    cout << "  Area: " << Cerchio::area(r) << endl;
    cout << "  Perimetro: " << Cerchio::perimetro(r) << endl;

    double b = 4.0, a = 3.0;
    cout << "\nRettangolo " << b << "x" << a << ":" << endl;
    cout << "  Area: " << Rettangolo::area(b, a) << endl;
    cout << "  Perimetro: " << Rettangolo::perimetro(b, a) << endl;
    cout << "  Diagonale: " << Rettangolo::diagonale(b, a) << endl;

    return 0;
}
```

Preprocessore

Le direttive del preprocessore in C++, come

Cos'è il preprocessore?

Quando scrivi un programma in C++, prima ancora che il compilatore legga il tuo codice, un altro programma — chiamato **preprocessore** — fa una "pulizia e preparazione" del testo.

Immagina di consegnare un compito scritto a matita a un assistente: lui prima cancella i riferimenti, sostituisce alcune parole con quelle giuste, aggiunge i fogli mancanti... e solo dopo lo consegna al professore (il compilatore) per la correzione.

Le istruzioni che dai al preprocessore si chiamano **direttive** e iniziano sempre con il simbolo `#`.

`#include` — Aggiungere altri file

`#include` dice al preprocessore: *"Copia il contenuto di questo file e mettilo qui."*

È come dire: "Aggiungi questo capitolo del libro al mio testo."

Librerie del sistema

Per includere le librerie già pronte che vengono con C++, si usano le parentesi angolari `<>`:

```
#include <iostream>    // per stampare e leggere dati
#include <string>       // per usare le stringhe
#include <cmath>        // per le funzioni matematiche
#include <vector>       // per usare i vettori
```

Il compilatore sa già dove trovare questi file nel tuo computer.

File tuoi

Per includere file che hai scritto tu, si usano le virgolette `""`:

```
#include "studente.h"    // file nella stessa cartella
#include "modelli/persona.h" // file in una sottocartella
```

Il compilatore cerca prima nella cartella del tuo progetto, poi nelle cartelle standard.

#define — Creare una sostituzione automatica

`#define` è come creare una scorciatoia: dici al preprocessore "ogni volta che vedi questa parola, sostituiscila con quest'altra".

Costanti con `#define`

```
#define PI 3.14159265358979
#define MAX_STUDENTI 30
#define SALUTO "Ciao, mondo!"

int main() {
    // Il preprocessore sostituisce PI prima ancora di compilare:
    // double area = 3.14159265358979 * 5.0 * 5.0;
    double area = PI * 5.0 * 5.0;
    return 0;
}
```

Oggi si preferisce usare `const` al posto di `#define` per le costanti, perché è più sicuro. Ma `#define` si vede spesso nel codice esistente.

Macro con parametri (scorciatoie con calcoli)

Puoi creare scorciatoie che accettano valori:

```
#define MASSIMO(a, b) ((a) > (b) ? (a) : (b))
#define QUADRATO(x) ((x) * (x))

int main() {
    int x = 5, y = 3;
    cout << MASSIMO(x, y) << endl;    // stampa 5
    cout << QUADRATO(4) << endl;     // stampa 16
    return 0;
}
```

Attenzione: queste "macro-funzioni" possono dare risultati strani. Preferisci le funzioni normali:

```
// PROBLEMA: n viene incrementato due volte!  
int n = 5;  
int r = QUADRATO(n++);    // diventa: ((n++) * (n++)) – comportamento  
                           imprevedibile  
  
// SOLUZIONE: usa una funzione normale  
int quadrato(int x) { return x * x; }
```

#undef — Cancellare una scorciatoia

```
#define VALORE 42  
cout << VALORE << endl;    // 42  
  
#undef VALORE  
// Da qui in poi VALORE non esiste più  
// cout << VALORE << endl; // ERRORE!
```

Compilazione condizionale — Includere codice solo in certi casi

Puoi dire al preprocessore:

"includi questa parte di codice solo se questa condizione è vera"

.

È come avere due versioni di un documento e scegliere quale stampare:

```

#define DEBUG    // commenta questa riga per disabilitare i messaggi di
debug

int main() {
    int x = 42;

#ifdef DEBUG
    // Questa riga viene inclusa SOLO se DEBUG è definito
    cout << "Debug: x = " << x << endl;
#endif

    cout << "Valore: " << x << endl;
    return 0;
}

```

- `#ifdef NOME` = "se NOME è definito, includi il codice che segue"
- `#ifndef NOME` = "se NOME NON è definito, includi il codice che segue"
- `#endif` = "fine del blocco condizionale"

Include guard — Evitare di includere lo stesso file due volte

Immagina di copiare e incollare lo stesso capitolo due volte in un libro: sarebbe un problema. Stessa cosa in C++.

Quando scrivi un file `.h`, usa sempre questa protezione:

```

// studente.h
#ifndef STUDENTE_H    // se STUDENTE_H non è ancora definito...
#define STUDENTE_H    // ...definiscilo (ed esegui il codice qui sotto)

// Tutto il contenuto del file va qui
void salutaStudiante();
struct Studente { int eta; };

#endif    // fine della protezione

```

La seconda volta che questo file viene incluso (magari attraverso un altro file), `STUDENTE_H` è già definito, quindi tutto il contenuto viene saltato automaticamente.

In alternativa, puoi usare `#pragma once`, che fa la stessa cosa in modo più semplice:

```
// studente.h
#pragma once // includi questo file al massimo una volta

void salutaStudente();
```

`#pragma once` è supportato da quasi tutti i compilatori moderni, anche se non è parte dello standard ufficiale.

#error — Generare un errore personalizzato

Puoi far fallire la compilazione con un messaggio tuo:

```
#ifndef MIA_LIBRERIA_H
#error "Devi includere mia_libreria.h prima di questo file!"
#endif
```

Se la condizione si verifica, il compilatore si ferma e mostra il tuo messaggio.

Macro predefinite — Informazioni automatiche

Il preprocessore definisce alcune parole speciali in automatico:

```
cout << __FILE__ << endl; // nome del file corrente
cout << __LINE__ << endl; // numero della riga corrente
cout << __DATE__ << endl; // data in cui è stato compilato il programma
cout << __TIME__ << endl; // ora in cui è stato compilato
```

Sono molto utili per il debug — per sapere esattamente dove è successo un problema:

```

#ifdef DEBUG
// Crea un messaggio di log con il nome del file e il numero di riga
#define LOG(msg) cout << "[" << __FILE__ << ":" << __LINE__ << "]" " << msg
<< endl
#else
#define LOG(msg) // in modalità normale, LOG non fa nulla
#endif

int main() {
    int x = 42;
    LOG("Il valore di x è: " << x); // messaggio visibile solo in
    modalità debug
    return 0;
}

```

Sintassi di Base

Le regole fondamentali della sintassi del C++.

Le regole di base del C++

Ogni linguaggio ha le sue regole grammaticali. In italiano mettiamo il punto alla fine della frase. In C++ dobbiamo mettere il **punto e virgola** alla fine di ogni istruzione. Se non rispetti queste regole, il programma non funziona.

Imparare la sintassi di base è come imparare l'alfabeto: prima lo conosci, poi puoi scrivere qualsiasi cosa.

La regola più importante: il punto e virgola

Ogni istruzione in C++ deve terminare con un **punto e virgola** (;). Pensa a esso come al punto fermo di una frase.

```

int x = 10;           // istruzione 1 – termina con ;
cout << "Ciao!";      // istruzione 2 – termina con ;
x = x + 1;           // istruzione 3 – termina con ;

```

Dimenticare il punto e virgola è l'errore più comune per chi inizia. Il compilatore te lo segnala subito.

Le parentesi graffe: i blocchi

Le parentesi graffe `{` e `}` delimitano un **blocco di codice**, cioè un gruppo di istruzioni che stanno insieme.

Pensa a un blocco come a un paragrafo in un testo: delimita un insieme di cose che vanno lette insieme.

```
int main() {  
    // tutto qui dentro è il "blocco" del main  
    int x = 5;  
    cout << x;  
}
```

Ogni `{` apre un blocco, ogni `}` lo chiude. Devono sempre essere in coppia.

Maiuscole e minuscole: fanno differenza

Il C++ distingue tra lettere maiuscole e minuscole. `cout`, `Cout` e `COUT` sono tre cose completamente diverse (solo `cout` è quello giusto).

```
int eta = 16;  
int Eta = 20;    // questa è un'altra variabile, diversa dalla prima!  
  
cout << eta;     // stampa 16  
cout << Eta;     // stampa 20
```

Regola pratica: scrivi tutto in minuscolo, tranne dove le convenzioni prevedono il contrario.

L'indentazione: non obbligatoria, ma fondamentale

In C++, gli spazi a inizio riga (l'indentazione) non cambiano il comportamento del programma. Questo funziona:

```
int main(){int x=5;cout<<x;return 0;}
```

Ma è illeggibile. Scrivi sempre il codice così:


```
int main() {
    int x = 5;
    if (x > 0) {
        cout << "positivo";
    }
    return 0;
}
```

Usa 4 spazi (o un tab) per ogni livello di indentazione. Il tuo futuro io te ne sarà grato.

I nomi delle variabili (identificatori)

Un **identificatore** è il nome che dai a una variabile, a una funzione, ecc. Pensa a esso come a un'etichetta su una scatola.

Regole per i nomi:

- Deve iniziare con una lettera (`a-z` , `A-Z`) o un underscore `_`
- Può contenere lettere, numeri e underscore
- Non può contenere spazi o simboli speciali (`@` , `!` , `-` , ecc.)
- Non può essere una parola riservata del linguaggio (come `int` , `if` , `return`)

```
// Nomi validi
int eta;
double altezza_media;
string nomeUtente;
int x1;

// Nomi non validi
int 2variabile;    // inizia con un numero - ERRORE
int nome utente;  // contiene uno spazio - ERRORE
int int;           // è una parola riservata - ERRORE
```

Le parole riservate

Le **parole riservate** sono parole che hanno già un significato speciale in C++. Non puoi usarle come nomi di variabili:

```
int, float, double, char, bool, void  
if, else, while, for, do, switch, case, break, continue  
return, class, struct, public, private, new, delete
```

Le convenzioni di stile

In C++ esistono tre modi principali per scrivere i nomi:

camelCase — la prima parola è minuscola, le successive iniziano con la maiuscola. Usato per le variabili:

```
int numeroStudenti;  
string nomeUtente;
```

snake_case — tutto minuscolo, parole separate da underscore. Anch'esso usato per le variabili:

```
int numero_studenti;  
string nome_utente;
```

PascalCase — ogni parola inizia con la maiuscola. Usato per i nomi delle classi:

```
class NomeClasse;  
struct DatiStudente;
```

Scegli uno stile e usalo in modo coerente in tutto il progetto.

I tipi di istruzione

In C++ ci sono diversi tipi di istruzioni. Ecco le principali:

```
// Dichiarazione di variabile – riserva uno spazio in memoria
int x;

// Assegnazione – mette un valore nella variabile
x = 10;

// Dichiarazione con inizializzazione – le due cose insieme
int y = 20;

// Chiamata a funzione – esegue un'azione
cout << "Ciao";

// Istruzione condizionale – prende una decisione
if (x > 0) {
    cout << "positivo";
}

// Istruzione di ritorno – termina la funzione
return 0;
```

Esempio completo

```
#include <iostream>
using namespace std;

int main() {
    // Dichiariamo e inizializziamo tre variabili
    int numero = 42;
    double pi = 3.14159;
    string messaggio = "Il numero magico è: ";

    // Stampiamo il messaggio e il valore di numero
    cout << messaggio << numero << endl;

    // Stampiamo il valore di pi
    cout << "Pi greco: " << pi << endl;

    // Calcoliamo il doppio di numero e lo stampiamo
    int doppio = numero * 2;
    cout << "Il doppio è: " << doppio << endl;

    return 0;
}
```

Output:

```
Il numero magico è: 42
Pi greco: 3.14159
Il doppio è: 84
```

Commenti in C++

Come usare i commenti in C++ per documentare il codice.

A cosa servono i commenti?

Immagina di tornare a leggere un codice che hai scritto sei mesi fa. Senza commenti, potresti non capire più cosa fa o perché lo hai scritto in quel modo.

I **commenti** sono note che scrivi nel codice per spiegare cosa sta succedendo. Il compilatore le ignora completamente: esistono solo per chi legge il codice (tu, un compagno, un professore).

Scrivere buoni commenti è un'abitudine che distingue un programmatore attento da uno frettoloso.

Commento su riga singola: //

Il commento su riga singola inizia con `//`. Tutto ciò che viene dopo `//`, fino alla fine della riga, viene ignorato dal compilatore.

```
// Questo intero commento viene ignorato
int x = 5; // anche questo, solo la parte dopo //

// Puoi usare i commenti per disabilitare temporaneamente una riga
// int y = 10; ← questa riga non viene eseguita
```

Commento su più righe: /* . . . */

Il commento su più righe inizia con `/*` e finisce con `*/`. Tutto ciò che sta in mezzo viene ignorato, anche se occupa più righe.

```
/* Questo è un commento
   che occupa più righe.
   Utile per spiegazioni lunghe. */

int x = 5;

/*
 * Alcuni programmatori aggiungono un asterisco
 * all'inizio di ogni riga per migliorare la leggibilità.
 * È solo una convenzione stilistica.
 */
```

Cosa commentare: il "perché", non il "cosa"

Un buon commento spiega **perché** fai qualcosa, non **cosa** stai facendo. Il codice stesso dovrebbe essere abbastanza chiaro da spiegare il "cosa".

```
// Commento inutile – il codice si capisce già da solo
x = x + 1; // incrementa x di 1

// Commento utile – spiega il motivo
tentativi = tentativi + 1; // blocchiamo l'accesso dopo 3 tentativi falliti
```

Documentare le funzioni

Usa i commenti per spiegare cosa fa una funzione, i valori che riceve e quello che restituisce:

```
// Calcola l'area di un rettangolo.
// base: la larghezza del rettangolo (in centimetri)
// altezza: l'altezza del rettangolo (in centimetri)
// Restituisce l'area come numero decimale.
double calcolaArea(double base, double altezza) {
    return base * altezza;
}
```

Chi usa questa funzione capisce subito come usarla, senza dover leggere il codice al suo interno.

Disabilitare codice temporaneamente

Durante il debug, puoi commentare del codice per escluderlo momentaneamente dall'esecuzione:

```
int main() {
    int x = 10;
    // cout << "Valore intermedio: " << x << endl; ← disabilitato per ora
    cout << "Valore finale: " << x << endl;
    return 0;
}
```

Questo è più veloce che cancellare il codice e poi riscriverlo.

Commenti da evitare

Evita i commenti che non aggiungono alcuna informazione rispetto al codice:

```
// Da evitare – sono ovvi  
int eta = 16;    // imposta eta a 16  
x = x + 1;      // aggiunge 1 a x  
return 0;       // ritorna 0
```

Evita anche i commenti sbagliati, cioè che non corrispondono più al codice. Sono peggio di nessun commento, perché ingannano chi legge:

```
// Somma due numeri ← SBAGLIATO! In realtà moltiplica  
int risultato = a * b;
```

Esempio completo

```
#include <iostream>
using namespace std;

/*
 * Programma: Calcolatore dell'area di un cerchio
 * Autore: Alice
 *
 * Legge il raggio dall'utente e calcola l'area.
 */

// Valore approssimato di pi greco – non cambia mai
const double PI = 3.14159265358979;

int main() {
    double raggio;

    // Chiediamo il raggio all'utente
    cout << "Inserisci il raggio: ";
    cin >> raggio;

    // Calcoliamo l'area con la formula: A = pi * r²
    double area = PI * raggio * raggio;

    cout << "L'area del cerchio è: " << area << endl;

    return 0;
}
```

Costanti

Come definire valori costanti in C++ con const e

Cos'è una costante?

Immagina di scrivere un programma che usa il numero pi greco (3.14159...) in dieci punti diversi. Se un giorno vuoi usare una versione più precisa, devi cambiarla in dieci posti.

Oppure potresti avere il numero `18` sparso nel codice. Chi legge non sa se è l'età minima, un limite di velocità o qualcos'altro.

Le **costanti** risolvono entrambi i problemi: dai un nome a un valore che non cambia mai, e lo scrivi in un unico posto.

La parola chiave `const`

Scrivi `const` prima del tipo per rendere una variabile costante. Una volta assegnato il valore, non può più essere cambiato.

```
const double PI = 3.14159265358979;  
const int ORE_PER_GIORNO = 24;  
const int ETA_MINIMA = 18;
```

Se provi a modificare una costante, il compilatore ti blocca con un errore:

```
const int MAX = 100;  
MAX = 200; // ERRORE: non puoi modificare una costante
```

Questo è un vantaggio: il compilatore ti protegge da modifiche accidentali.

La convenzione dei nomi

Per le costanti si usa la convenzione di scrivere **tutto in maiuscolo**, con le parole separate da underscore. Così si riconoscono a colpo d'occhio nel codice:

```
const double PI = 3.14159;  
const int ETA_MINIMA = 18;  
const int MAX_TENTATIVI = 3;  
const double VELOCITA_LUCE = 299792458.0;
```

Un altro modo: `#define`

Prima che esistesse `const`, i programmatori usavano la direttiva `#define`:

```
#define PI 3.14159
#define MAX_STUDENTI 30
```

Con `#define`, il compilatore sostituisce ogni occorrenza di `PI` con `3.14159` prima di compilare il codice. Non è una vera variabile: è una semplice sostituzione di testo.

```
#include <iostream>
#define PI 3.14159
#define RAGGIO 5

using namespace std;

int main() {
    // Il compilatore sostituisce PI con 3.14159 e RAGGIO con 5
    double area = PI * RAGGIO * RAGGIO;
    cout << "Area: " << area << endl;
    return 0;
}
```

const o #define ? Usa const

Oggi si preferisce sempre `const`. Ecco perché:

Caratteristica	const	#define
Ha un tipo definito	Sì	No
Il compilatore la controlla	Sì	No
Visibile nel debugger	Sì	No
Rispetta i blocchi di codice	Sì	No

```
// Meglio: const ha tipo e viene controllata
const double PI = 3.14159;

// Meno sicuro: #define non ha tipo
#define PI 3.14159
```

constexpr : costanti calcolate durante la compilazione

In C++ moderno esiste anche **constexpr**. Si usa per costanti il cui valore può essere calcolato già durante la compilazione (prima ancora che il programma venga eseguito):

```
constexpr int SECONDI_PER_MINUTO = 60;
constexpr int MINUTI_PER_ORA = 60;
// Il compilatore calcola 60 * 60 = 3600 prima ancora di eseguire il
programma
constexpr int SECONDI_PER_ORA = SECONDI_PER_MINUTO * MINUTI_PER_ORA;
```

Per ora puoi usare tranquillamente **const**. Imparerai **constexpr** più avanti.

Esempio pratico

```
#include <iostream>
using namespace std;

// Costanti definite una volta sola, usate ovunque nel programma
const double PI = 3.14159265358979;
const double GRAVITA = 9.81; // accelerazione di gravità in m/s²
const int MAX_VOTO = 10;

int main() {
    // Calcolo dell'area di un cerchio con raggio 4
    double raggio = 4.0;
    double area = PI * raggio * raggio;
    cout << "Area cerchio (r=" << raggio << "): " << area << endl;

    // Calcolo dell'energia potenziale di un oggetto
    double massa = 70.0; // kg
    double altezza = 5.0; // m
    double energia = massa * GRAVITA * altezza;
    cout << "Energia potenziale: " << energia << " J" << endl;

    // Uso della costante del voto massimo
    int voto = 8;
    cout << "Voto: " << voto << "/" << MAX_VOTO << endl;

    return 0;
}
```

Output:

```
Area cerchio (r=4): 50.2655
Energia potenziale: 3434.5 J
Voto: 8/10
```

Tipi di Dati

I tipi di dati fondamentali del C++: int, float, double, char e bool.

Perché esistono i tipi di dati?

Immagina di avere dei cassetti in un armadio. Un cassetto è per i calzini, uno per le magliette, uno per i pantaloni. Non metteresti dei pantaloni nel cassetto dei calzini.

In C++ funziona allo stesso modo: ogni variabile ha un **tipo** che stabilisce che genere di valore può contenere. A differenza di Python, non puoi mettere qualsiasi cosa in qualsiasi variabile: devi dichiarare il tipo in anticipo.

Questo rende il codice più sicuro e veloce.

Numeri interi: `int`

`int` (abbreviazione di *integer*, intero) contiene numeri senza virgola, sia positivi che negativi:

```
int eta = 16;
int temperatura = -5;
int anno = 2024;
int zero = 0;
```

Un `int` occupa 4 byte di memoria e può contenere valori da circa -2 miliardi a +2 miliardi. Per la maggior parte dei programmi, è più che sufficiente.

Se hai bisogno di numeri molto grandi o molto piccoli, esistono varianti:

```
short int piccolo = 32000; // massimo circa 32.000
long long enorme = 9000000000000LL; // fino a 9 miliardi di miliardi
```

Numeri decimali: `float` e `double`

Per numeri con la virgola (come 3.14 o 1.75) hai due opzioni.

`float`

Occupi 4 byte. Preciso fino a circa 7 cifre. Metti la `f` alla fine del numero per indicare che è un float:

```
float temperatura = 36.6f;  
float pi = 3.14159f;
```

double

Occupi 8 byte. Preciso fino a circa 15-16 cifre. È quello più usato:

```
double altezza = 1.75;  
double pi = 3.14159265358979;
```

Quale usare?

Usa sempre **double**, a meno che tu non abbia un motivo specifico per usare **float**. Il **double** è più preciso e il tipo preferito in C++ per i numeri decimali:

```
// Possibile problema con float  
float a = 0.1f + 0.2f;  
cout << a; // potrebbe stampare 0.3000001 invece di 0.3  
  
// Meglio con double  
double b = 0.1 + 0.2;  
cout << b; // molto più preciso
```

Singolo carattere: char

char (abbreviazione di *character*, carattere) contiene un singolo carattere. I caratteri si scrivono tra **apici singoli** '...':

```
char lettera = 'A';  
char cifra = '7';  
char spazio = ' ';
```

Internamente, ogni carattere è memorizzato come un numero (il suo codice ASCII). Per esempio, `'A'` corrisponde al numero 65:

```
char c = 'A';
cout << c;      // stampa: A
cout << (int)c; // stampa: 65 (il codice numerico di 'A')
```

Vero o falso: `bool`

`bool` (abbreviazione di *boolean*, booleano) può contenere solo due valori: `true` (vero) o `false` (falso):

```
bool maggiorenne = true;
bool haPatente = false;
bool risultato = (5 > 3); // true, perché 5 è maggiore di 3
```

Quando stampi un `bool`, di default viene mostrato come `1` (true) o `0` (false). Per stampare le parole `true` e `false`:

```
bool x = true;
cout << x;      // stampa: 1
cout << boolalpha << x; // stampa: true
```

Quanto spazio occupano?

Puoi controllare quanti byte occupa un tipo con `sizeof`:

```
cout << sizeof(int) << endl; // 4
cout << sizeof(double) << endl; // 8
cout << sizeof(char) << endl; // 1
cout << sizeof(bool) << endl; // 1
```

Riepilogo

Tipo	Spazio	Valori
<code>int</code>	4 byte	numeri interi, da -2 miliardi a +2 miliardi
<code>float</code>	4 byte	decimali, ~7 cifre significative
<code>double</code>	8 byte	decimali, ~15 cifre significative
<code>char</code>	1 byte	un singolo carattere
<code>bool</code>	1 byte	solo <code>true</code> oppure <code>false</code>

Convertire tra tipi

A volte hai bisogno di convertire un valore da un tipo all'altro.

La **conversione implicita** avviene automaticamente quando è sicura:

```
int x = 5;
double y = x;    // int viene convertito in double automaticamente
cout << y;      // 5.0
```

La **conversione esplicita** (chiamata *casting*) si usa quando vuoi forzare una conversione:

```
double pi = 3.14159;
int parteIntera = (int)pi; // la parte decimale viene scartata
cout << parteIntera;      // 3 (non arrotondato, solo troncato)
```

Attenzione alla **divisione tra interi**: quando dividi due `int`, il risultato è sempre un intero (la parte decimale viene persa):

```
int x = 5, y = 2;
int risultato = x / y;    // risultato = 2, non 2.5!
double risultato2 = (double)x / y; // risultato2 = 2.5 (corretto)
```


Esempio pratico

```
#include <iostream>
using namespace std;

int main() {
    // Una variabile per ogni tipo principale
    int eta = 16;
    double altezza = 1.75;
    char iniziale = 'A';
    bool maggiorenne = false;

    cout << "Età (int): " << eta << endl;
    cout << "Altezza (double): " << altezza << endl;
    cout << "Iniziale (char): " << iniziale << endl;
    cout << "Maggiorenne (bool): " << boolalpha << maggiorenne << endl;

    return 0;
}
```

Output:

```
Età (int): 16
Altezza (double): 1.75
Iniziale (char): A
Maggiorenne (bool): false
```

Installazione

Come installare il compilatore GCC e configurare Visual Studio Code per programmare in C++.

Cosa ti serve

Per scrivere programmi in C++ hai bisogno di due cose:

1. Un **compilatore**: il programma che trasforma il tuo codice C++ in qualcosa che il computer può eseguire
2. Un **editor di testo**: dove scrivi il codice

In questa guida usiamo **GCC** come compilatore e **Visual Studio Code** (VS Code) come editor. Entrambi sono gratuiti.

Differenza tra C++ e Python: la compilazione

In Python, scrivi il codice e lo esegui subito. In C++ c'è un passaggio intermedio: la **compilazione**.

È come la differenza tra un interprete simultaneo (Python — traduce mentre parli) e un traduttore che prima legge tutto il testo, poi ti consegna il testo tradotto (C++ — prima compila, poi esegui).

Il vantaggio è che i programmi C++ compilati sono molto più veloci di quelli Python.

Installare il compilatore GCC

GCC (GNU Compiler Collection) è il compilatore C++ più usato al mondo. È gratuito e open source.

Su Windows

Il modo più semplice è installare **MSYS2** che include GCC:

1. Vai su **msys2.org** e scarica il programma di installazione
2. Segui le istruzioni di installazione
3. Apri il terminale MSYS2 e digita: `pacman -S mingw-w64-ucrt-x86_64-gcc`
4. Aggiungi la cartella `C:\msys64\ucrt64\bin` alla variabile d'ambiente PATH

Verifica che funzioni aprendo il prompt dei comandi e digitando:

```
g++ --version
```

Se vedi un numero di versione, l'installazione è riuscita.

Su macOS

Su macOS puoi installare gli strumenti di sviluppo di Apple con questo comando nel terminale:

```
xcode-select --install
```

Una finestra ti chiederà conferma — clicca "Installa". Al termine, verifica:

```
g++ --version
```

Su Linux (Ubuntu/Debian)

Su Linux è il più semplice:

```
sudo apt update  
sudo apt install g++
```

Verifica:

```
g++ --version
```

Installare Visual Studio Code

VS Code è un editor di testo gratuito, leggero e molto popolare tra i programmatori.

1. Vai su **code.visualstudio.com** e scaricalo
2. Installalo come qualsiasi altro programma

Estensioni consigliate

Dopo aver aperto VS Code, installa queste estensioni (clicca sull'icona delle estensioni nella barra laterale sinistra):

- **C/C++** (di Microsoft): colora il codice, suggerisce completamenti, aiuta col debugging
- **Code Runner**: esegue il codice con un clic

Il primo programma: Hello, World!

Crea il file

1. Apri VS Code
2. Crea una nuova cartella per i tuoi progetti (es. `progetti-cpp`)
3. Apri quella cartella in VS Code (File > Apri cartella)
4. Crea un nuovo file chiamato `hello.cpp`

Scrivi il codice

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

Compila il programma

Apri il terminale integrato in VS Code (View > Terminal):

```
g++ hello.cpp -o hello
```

Questo comando dice al compilatore GCC di:

- leggere il file `hello.cpp`
- creare un eseguibile chiamato `hello`

Esegui il programma

```
# Su Linux/macOS
./hello

# Su Windows
hello.exe
```

Output:

```
Hello, World!
```

Il comando di compilazione in dettaglio

```
g++ hello.cpp -o hello -Wall -std=c++17
```

Parte	Significato
<code>g++</code>	Il compilatore
<code>hello.cpp</code>	Il file sorgente da compilare
<code>-o hello</code>	Il nome dell'eseguibile da creare
<code>-Wall</code>	Mostra tutti gli avvisi (consigliato!)
<code>-std=c++17</code>	Usa la versione C++17 del linguaggio

Non vuoi installare niente? Usa Replit

Se preferisci iniziare senza installare nulla, puoi usare **Replit** — un ambiente di sviluppo online che funziona direttamente nel browser. Vai su **replit.com**, crea un account gratuito, crea un nuovo progetto C++ e scrivi il codice lì.

Altre alternative:

- **Code::Blocks** — IDE specifico per C/C++, facile da usare
 - **Dev-C++** — semplice IDE per Windows
-

Operatori in C++

Gli operatori aritmetici, di confronto e logici in C++.

Cosa sono gli operatori?

Un programma senza operatori non può fare quasi nulla. Gli **operatori** sono i simboli che usano il programma per fare calcoli, confrontare valori e prendere decisioni.

Sono come i simboli matematici che già conosci (`+` , `-` , `*` , `/`), con qualche aggiunta specifica per la programmazione.

Operatori aritmetici

Questi operatori fanno calcoli matematici:

Operatore	Operazione	Esempio	Risultato
<code>+</code>	Addizione	<code>5 + 3</code>	<code>8</code>
<code>-</code>	Sottrazione	<code>5 - 3</code>	<code>2</code>
<code>*</code>	Moltiplicazione	<code>5 * 3</code>	<code>15</code>
<code>/</code>	Divisione	<code>10 / 3</code>	<code>3</code> (intera!)
<code>%</code>	Modulo (resto)	<code>10 % 3</code>	<code>1</code>

```
int a = 10, b = 3;

cout << a + b << endl; // 13
cout << a - b << endl; // 7
cout << a * b << endl; // 30
cout << a / b << endl; // 3 - attenzione, non 3.33!
cout << a % b << endl; // 1 - il resto della divisione
```

La divisione intera

Quando si dividono due numeri interi, il risultato è sempre un intero. La parte decimale viene scartata, non arrotondata:

```
int x = 7, y = 2;
cout << x / y << endl; // 3 (non 3.5!)

// Per ottenere 3.5, converti uno dei due in double prima di dividere
double risultato = (double)x / y;
cout << risultato << endl; // 3.5
```

Il modulo %

Il modulo restituisce il **resto** della divisione intera. È utile per capire se un numero è pari o dispari:

```
cout << 10 % 3 << endl; // 1 (10 = 3×3 + 1)
cout << 9 % 3 << endl; // 0 (9 è divisibile per 3)
cout << 7 % 2 << endl; // 1 (7 è dispari)
cout << 8 % 2 << endl; // 0 (8 è pari)
```

Operatori di assegnazione

Oltre al semplice `=`, esistono operatori che modificano una variabile in un colpo solo:

Operatore	Equivalente	Esempio con <code>x = 10</code>
<code>=</code>	assegnazione semplice	<code>x = 5</code> → x vale 5
<code>+=</code>	<code>x = x + y</code>	<code>x += 3</code> → x vale 13
<code>-=</code>	<code>x = x - y</code>	<code>x -= 3</code> → x vale 7
<code>*=</code>	<code>x = x * y</code>	<code>x *= 2</code> → x vale 20
<code>/=</code>	<code>x = x / y</code>	<code>x /= 5</code> → x vale 2
<code>%=</code>	<code>x = x % y</code>	<code>x %= 3</code> → x vale 1

```
int x = 10;
x += 5;    // x è ora 15
x -= 3;    // x è ora 12
x *= 2;    // x è ora 24
x /= 4;    // x è ora 6
x %= 4;    // x è ora 2
```

Operatori di incremento e decremento

Aggiungere o sottrarre 1 è così comune in C++ che esistono operatori appositi:

```
int x = 5;
x++;    // x diventa 6 (aggiunge 1)
x--;    // x diventa 5 (sottrae 1)
```

Esistono anche nella forma con `++` prima della variabile:

```
int x = 5;
int b = x++; // prima assegna x a b (b = 5), poi incrementa x (x = 6)
int c = ++x; // prima incrementa x (x = 7), poi assegna a c (c = 7)
```

Per i principianti è sufficiente usare `x++` e `x--` da soli, senza usarli dentro espressioni complesse.

Operatori di confronto

Confrontano due valori e restituiscono `true` (vero) o `false` (falso):

Operatore	Significato	Esempio	Risultato
<code>==</code>	uguale a	<code>5 == 5</code>	<code>true</code>
<code>!=</code>	diverso da	<code>5 != 3</code>	<code>true</code>
<code><</code>	minore di	<code>3 < 5</code>	<code>true</code>
<code>></code>	maggiore di	<code>5 > 3</code>	<code>true</code>
<code><=</code>	minore o uguale	<code>5 <= 5</code>	<code>true</code>
<code>>=</code>	maggiore o uguale	<code>3 >= 5</code>	<code>false</code>

```
int a = 5, b = 3;
```

```
cout << (a == b) << endl; // 0 (false - 5 non è uguale a 3)
cout << (a != b) << endl; // 1 (true - 5 è diverso da 3)
cout << (a > b) << endl; // 1 (true - 5 è maggiore di 3)
cout << (a < b) << endl; // 0 (false - 5 non è minore di 3)
```

Errore comune: confondere `==` (confronto) con `=` (assegnazione). Scrivere `if (x = 5)` invece di `if (x == 5)` cambia il valore di `x` invece di confrontarlo!

Operatori logici

Combinano più condizioni insieme:

Operatore	Significato	Esempio	Risultato
<code>&&</code>	E (AND)	<code>true && false</code>	<code>false</code>
<code> </code>	O (OR)	<code>true false</code>	<code>true</code>
<code>!</code>	NON (NOT)	<code>!true</code>	<code>false</code>

AND (&&): entrambe le condizioni devono essere vere

```
int eta = 20;
bool haPatente = true;

if (eta >= 18 && haPatente) {
    cout << "Puoi guidare." << endl;
}
// Entrambe le condizioni sono vere → stampa il messaggio
```

OR (||): basta che almeno una condizione sia vera

```
int ora = 14;

if (ora < 9 || ora > 18) {
    cout << "Il negozio è chiuso." << endl;
} else {
    cout << "Il negozio è aperto." << endl;
}
// ora = 14, nessuna delle due condizioni è vera → "aperto"
```

NOT (!): inverte il valore booleano

```
bool piove = false;

if (!piove) {
    cout << "Possiamo uscire!" << endl;
}
// !false è true → stampa il messaggio
```

La priorità degli operatori

Come in matematica, alcuni operatori hanno la precedenza su altri. Da quella più alta a quella più bassa:

1. `!`, `++`, `--`
2. `*`, `/`, `%`
3. `+`, `-`
4. `<`, `>`, `<=`, `>=`
5. `==`, `!=`
6. `&&`
7. `||`
8. `=`, `+=`, `-=`, ecc.

Quando hai dubbi, usa le parentesi per rendere esplicito l'ordine:

```
int risultato = 2 + 3 * 4;    // 14 - la moltiplicazione va prima
int risultato2 = (2 + 3) * 4; // 20 - le parentesi cambiano tutto
```

Esempio pratico

```
#include <iostream>
using namespace std;

int main() {
    int a = 15, b = 4;

    // Operatori aritmetici
    cout << "Somma: " << a + b << endl;      // 19
    cout << "Differenza: " << a - b << endl;  // 11
    cout << "Prodotto: " << a * b << endl;    // 60
    cout << "Quoziente: " << a / b << endl;   // 3 (divisione intera)
    cout << "Resto: " << a % b << endl;      // 3

    cout << endl;

    // Operatori di confronto
    cout << "a > b: " << (a > b) << endl;    // 1 (true)
    cout << "a == b: " << (a == b) << endl; // 0 (false)

    // Operatori logici
    bool cond = (a > 10) && (b < 10);
    cout << "a>10 E b<10: " << cond << endl; // 1 (true)

    return 0;
}
```

Struttura di un Programma C++

Le parti fondamentali che compongono ogni programma C++.

Come è fatto un programma C++?

Prima di scrivere il tuo primo programma, devi capire la sua struttura. È come imparare a costruire una casa: devi sapere dove vanno le fondamenta, i muri e il tetto, prima di poter arredare.

Ogni programma C++ segue una struttura fissa. Conoscerla ti permette di capire qualsiasi codice tu incontrerai.

La struttura minima

Ecco il programma C++ più semplice possibile:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Ciao!";
    return 0;
}
```

Questo programma stampa la parola "Ciao!" sullo schermo. Solo cinque righe, ma ognuna ha un ruolo preciso.

Parte 1: `#include <iostream>`

```
#include <iostream>
```

Questa riga dice al compilatore: "Ho bisogno della libreria `iostream`."

Pensa alle librerie come a delle cassette degli attrezzi: `iostream` è la cassetta che contiene gli strumenti per scrivere sullo schermo (`cout`) e leggere l'input dell'utente (`cin`).

Senza questa riga, il compilatore non saprebbe cosa significa `cout` e il programma non funzionerebbe.

Parte 2: `using namespace std;`

```
using namespace std;
```

Pensa a un **namespace** come a un cognome. Se in classe ci sono due ragazzi chiamati Marco, li distingui con il cognome: "Marco Rossi" e "Marco Bianchi".

`cout` e `cin` appartengono alla "famiglia" `std`. Scrivere `using namespace std;` dice al programma: "quando uso `cout`, intendo `std::cout`". Così puoi scrivere `cout` invece del nome completo

```
std::cout .
```

Parte 3: `int main() { ... }`

```
int main() {  
    // il programma inizia qui  
    return 0;  
}
```

La funzione `main()` è il **punto di partenza** di ogni programma C++. Quando avvii un programma, il computer cerca sempre `main()` e inizia a eseguire le istruzioni al suo interno.

- `int` — indica che `main` restituisce un numero intero al termine
- Le parentesi graffe `{` e `}` — delimitano il corpo della funzione
- `return 0;` — segnala che il programma è terminato senza errori (0 = tutto ok)

Parte 4: le istruzioni dentro `main()`

```
cout << "Ciao!";
```

Dentro `main()` scrivi le istruzioni del programma, una per riga. Ogni istruzione termina con il punto e virgola `;`.

`cout` stampa qualcosa sullo schermo. La freccia `<<` indica "manda questo testo a cout".

Un esempio più completo

Quando hai bisogno di più funzionalità, includi più librerie:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    // Dichiariamo due variabili
    string nome = "Alice";
    int eta = 16;

    // Le stampiamo sullo schermo
    cout << "Nome: " << nome << endl;
    cout << "Età: " << eta << endl;

    return 0;
}
```

Output:

```
Nome: Alice
Età: 16
```

Qui abbiamo aggiunto `#include <string>` perché vogliamo usare le stringhe di testo.

L'ordine delle sezioni

Un file C++ tipico segue sempre quest'ordine dall'alto verso il basso:

1. **Le librerie** — cosa includiamo
2. **Il namespace** — quale "famiglia" di nomi usare
3. **Variabili e costanti globali** — valori disponibili ovunque (opzionale)
4. **La funzione `main()`** — da dove parte il programma
5. **Altre funzioni** — pezzi di codice riutilizzabili (opzionale)

```
// 1. Librerie
#include <iostream>
#include <cmath>

// 2. Namespace
using namespace std;

// 3. Costante globale (opzionale)
const double PI = 3.14159;

// 4. Funzione main
int main() {
    double raggio = 5.0;
    // PI * r * r calcola l'area del cerchio
    double area = PI * raggio * raggio;
    cout << "Area: " << area << endl;
    return 0;
}
```

Le parentesi graffe delimitano i blocchi

Le parentesi graffe `{ }` compaiono ovunque in C++: nelle funzioni, nei cicli, nelle condizioni. Ogni coppia apre e chiude un **blocco di codice**.

```
int main() {           // apre il blocco di main
    if (true) {        // apre un blocco if
        cout << "Ciao";
    }                  // chiude il blocco if
    return 0;
}                      // chiude il blocco di main
```

Ogni `{` deve avere il suo `}` corrispondente. Se ne manca uno, il compilatore segnala un errore.

Il punto e virgola alla fine di ogni istruzione

In C++, quasi tutte le istruzioni terminano con `;`. Dimenticarlo è uno degli errori più comuni:


```
// Corretto
cout << "Ciao!" << endl;
int x = 5;

// Errato – manca il ;
cout << "Ciao!" << endl // il compilatore segnala un errore
int x = 5                // anche qui
```

Se non capisci un errore del compilatore, prima di tutto controlla che non manchi un punto e virgola.

Variabili in C++

Come dichiarare e usare le variabili in C++.

Cos'è una variabile?

Un programma deve ricordare delle informazioni mentre lavora: il nome dell'utente, il punteggio di un gioco, il risultato di un calcolo.

Le **variabili** sono i contenitori in cui il programma memorizza queste informazioni. Pensa a una variabile come a una scatola con un'etichetta: l'etichetta è il nome della variabile, il contenuto della scatola è il suo valore.

Dichiarare una variabile

In C++ devi **dichiarare** una variabile prima di usarla. La dichiarazione specifica due cose: il **tipo** (che genere di valore conterrà) e il **nome** (come si chiama):

```
int eta;           // una scatola per numeri interi, chiamata "eta"
double altezza;    // una scatola per numeri decimali, chiamata "altezza"
char iniziale;     // una scatola per un singolo carattere, chiamata
"iniziale"
```

Dopo la dichiarazione, la scatola esiste in memoria, ma è vuota (o meglio: contiene un valore casuale imprevedibile).

Inizializzare una variabile

Inizializzare significa mettere subito un valore nella scatola al momento in cui la crei:

```
int eta = 16;
double altezza = 1.65;
char iniziale = 'A';
bool maggiorenne = false;
```

In C++ moderno puoi anche usare le parentesi graffe per inizializzare:

```
int x{10};           // equivalente a int x = 10;
double pi{3.14};
```

Questa forma è preferita dai programmatori più esperti perché il compilatore la controlla più attentamente.

Cambiare il valore

Puoi cambiare il contenuto di una variabile in qualsiasi momento con l'operatore `=` :

```
int x = 5;
cout << x;  // stampa 5

x = 10;      // cambiamo il valore
cout << x;  // stampa 10

x = x + 3;   // usiamo il valore corrente per calcolarne uno nuovo
cout << x;  // stampa 13
```

Dichiarare più variabili insieme

Puoi dichiarare più variabili dello stesso tipo in una riga sola:

```
int a, b, c;           // tre variabili intere non inizializzate
int x = 1, y = 2, z = 3; // tre variabili con valori diversi
```

Le regole per i nomi

I nomi delle variabili devono seguire queste regole:

- Devono iniziare con una lettera (`a-z` , `A-Z`) o un underscore `_`
- Possono contenere lettere, numeri e underscore
- Non possono contenere spazi o simboli speciali
- Non possono essere parole riservate del linguaggio (`int` , `if` , `return` , ecc.)
- Le maiuscole contano: `eta` e `Eta` sono variabili diverse

```
// Nomi validi
int eta;
double altezza_media;
string nomeUtente;
int x1;

// Nomi non validi
int 2x;           // inizia con un numero - ERRORE
int nome utente; // contiene uno spazio - ERRORE
int int;          // è una parola riservata - ERRORE
```

Le convenzioni di stile

Non ci sono regole fisse sullo stile, ma due convenzioni sono molto diffuse:

- **camelCase:** `nomeUtente` , `etaMinima` , `numeroDiStudenti`
- **snake_case:** `nome_utente` , `eta_minima` , `numero_di_studenti`

Scegli una e usala in modo coerente in tutto il progetto.

Attenzione: variabili non inizializzate

In C++, una variabile dichiarata ma non inizializzata contiene un valore casuale (quello che c'era in quella zona di memoria prima):

```
int x;  
cout << x; // stampa un numero imprevedibile – comportamento non definito!
```

In Python questo causa un errore chiaro. In C++ il compilatore potrebbe non avvertire, e il programma si comporta in modo strano.

Regola pratica: inizializza sempre le variabili quando le dichiari.

Esempio completo

```
#include <iostream>  
#include <string>  
using namespace std;  
  
int main() {  
    // Dichiariamo e inizializziamo quattro variabili di tipo diverso  
    string nome = "Alice";  
    int eta = 16;  
    double altezza = 1.65;  
    bool haPatente = false;  
  
    // Stampiamo i valori  
    cout << "Nome: " << nome << endl;  
    cout << "Età: " << eta << endl;  
    cout << "Altezza: " << altezza << " m" << endl;  
    cout << "Ha la patente: " << boolalpha << haPatente << endl;  
  
    // Modifichiamo il valore di eta  
    eta = 17;  
    cout << "Età aggiornata: " << eta << endl;  
  
    return 0;  
}
```

Output:

Nome: Alice
Età: 16
Altezza: 1.65 m
Ha la patente: false
Età aggiornata: 17

Array

Come usare gli array in C++ per memorizzare sequenze di valori dello stesso tipo.

Cos'è un array?

Immagina di dover memorizzare i voti di 30 studenti. Creeresti 30 variabili diverse? `voto1` , `voto2` , ... `voto30` ? Sarebbe un disastro.

Un **array** è una fila di scatole numerate che contengono valori dello stesso tipo. Un solo nome, tanti valori. Puoi accedere a ciascuna scatola usando il suo numero (chiamato **indice**).

Dichiarare un array

```
tipo nome[dimensione];
```

```
int numeri[5];      // una fila di 5 numeri interi
double voti[10];    // una fila di 10 numeri decimali
char lettere[26];    // una fila di 26 caratteri
```

La dimensione deve essere un numero fisso, noto in anticipo (prima dell'esecuzione del programma).

Inizializzare un array

Puoi assegnare i valori subito alla dichiarazione, tra parentesi graffe:

```
int numeri[5] = {10, 20, 30, 40, 50};
double voti[4] = {8.5, 7.0, 9.0, 6.5};
char vocali[5] = {'a', 'e', 'i', 'o', 'u'};
```

Se non scrivi la dimensione, viene calcolata automaticamente dai valori:

```
int numeri[] = {10, 20, 30, 40, 50}; // dimensione 5, calcolata
automaticamente
```

Per inizializzare tutti gli elementi a zero:

```
int numeri[5] = {0}; // tutti e 5 a zero
int numeri2[5] = {}; // anche questo funziona in C++11
```

Se fornisci meno valori della dimensione, il resto viene azzerato:

```
int numeri[5] = {1, 2}; // diventa {1, 2, 0, 0, 0}
```

Accedere agli elementi

Ogni elemento ha un numero d'ordine chiamato **indice**. Attenzione: gli indici partono da **0**, non da 1.

```
int numeri[] = {10, 20, 30, 40, 50};
//indice:      0   1   2   3   4

cout << numeri[0] << endl; // 10 - primo elemento
cout << numeri[1] << endl; // 20
cout << numeri[4] << endl; // 50 - ultimo elemento
```

Per un array di dimensione **n**, gli indici vanno da **0** a **n-1**.

Modificare un elemento

```
int numeri[] = {10, 20, 30};
numeri[1] = 99;           // cambiamo il secondo elemento
cout << numeri[1] << endl; // 99
```

Pericolo: uscire dai limiti

In C++ non c'è alcun controllo automatico sui limiti. Se accedi a un indice che non esiste, il comportamento è imprevedibile e può corrompere la memoria del programma:

```
int numeri[5] = {1, 2, 3, 4, 5};
cout << numeri[10] << endl; // PERICOLO: indice fuori limiti
numeri[10] = 99;           // PERICOLO: potrebbe causare crash o dati
corrotti
```

Fai sempre attenzione che gli indici siano compresi tra `0` e `dimensione - 1`.

Scorrere un array con `for`

Il modo più comune per lavorare con tutti gli elementi di un array:

```
int numeri[] = {10, 20, 30, 40, 50};
int n = 5;

for (int i = 0; i < n; i++) {
    cout << numeri[i] << " ";
}
// Output: 10 20 30 40 50
```

Calcolare la dimensione automaticamente

Puoi ricavare il numero di elementi con `sizeof` :

```
int numeri[] = {10, 20, 30, 40, 50};  
// sizeof(numeri) = dimensione totale in byte dell'array  
// sizeof(numeri[0]) = dimensione in byte di un singolo elemento  
int dimensione = sizeof(numeri) / sizeof(numeri[0]);  
cout << dimensione << endl; // 5
```

Scorrere senza indice (range-based for)

In C++ moderno puoi scorrere tutti gli elementi in modo più semplice:

```
int numeri[] = {10, 20, 30, 40, 50};  
  
for (int n : numeri) {  
    cout << n << " ";  
}  
// Output: 10 20 30 40 50
```

Si legge: "per ogni `n` nell'array `numeri`".

Operazioni comuni

Trovare il valore massimo

```
int numeri[] = {3, 7, 1, 9, 4};
int n = 5;
int massimo = numeri[0]; // partiamo dal primo elemento

for (int i = 1; i < n; i++) {
    if (numeri[i] > massimo) {
        massimo = numeri[i]; // aggiorniamo il massimo se troviamo un
        valore più grande
    }
}

cout << "Massimo: " << massimo << endl; // 9
```

Calcolare somma e media

```
int numeri[] = {5, 10, 15, 20, 25};
int n = 5;
int somma = 0;

for (int i = 0; i < n; i++) {
    somma += numeri[i];
}

double media = (double)somma / n;
cout << "Somma: " << somma << endl; // 75
cout << "Media: " << media << endl; // 15
```

Esempio pratico

```
#include <iostream>
using namespace std;

int main() {
    const int N = 5;
    int voti[N];

    // Raccogliamo i voti dall'utente
    cout << "Inserisci " << N << " voti:" << endl;
    for (int i = 0; i < N; i++) {
        cout << "Voto " << (i + 1) << ": ";
        cin >> voti[i];
    }

    // Calcoliamo somma, massimo e minimo
    int somma = 0;
    int max = voti[0];
    int min = voti[0];

    for (int i = 0; i < N; i++) {
        somma += voti[i];
        if (voti[i] > max) max = voti[i];
        if (voti[i] < min) min = voti[i];
    }

    double media = (double)somma / N;

    cout << endl;
    cout << "Media: " << media << endl;
    cout << "Voto più alto: " << max << endl;
    cout << "Voto più basso: " << min << endl;

    return 0;
}
```

Stringhe C-style vs string

La differenza tra le stringhe in stile C (char array) e il tipo string del C++.

Le stringhe in stile C

In C++ esistono due modi diversi per lavorare con le parole e le frasi. Il primo è il modo "vecchio", ereditato dal linguaggio C. Il secondo è il modo moderno, molto più semplice.

Devi conoscere entrambi perché li incontrerai nel codice esistente, ma nella pratica ne userai quasi sempre uno solo.

I due metodi

1. **Stringhe C-style** — sono array di caratteri (`char[]`), ereditate dal linguaggio C. Più complicate da usare.
2. **`std::string`** — la stringa moderna del C++, molto più semplice e sicura.

Regola pratica: usa sempre `std::string` . Impara le stringhe C-style solo per capire il codice che le usa.

Le stringhe C-style: array di caratteri

Una stringa C-style è in realtà un array di `char` che termina con un carattere speciale: il carattere nullo `'\0'` . Questo terminatore dice "la stringa finisce qui".

```
char saluto[] = "Ciao";  
// Internamente memorizzata come: {'C', 'i', 'a', 'o', '\0'}  
// Occupa 5 byte (4 lettere + il terminatore)
```

Puoi accedere ai singoli caratteri con l'indice:

```
char nome[] = "Alice";  
cout << nome[0] << endl; // A  
cout << nome[1] << endl; // l
```

Le funzioni per le C-style strings

Per manipolare queste stringhe hai bisogno di funzioni speciali dalla libreria `<cstring>` :

```
#include <cstring>

char a[] = "Ciao";
char b[] = "Mondo";

cout << strlen(a) << endl;           // 4 – lunghezza della stringa (senza
'\0')
strcpy(a, "Hello");                  // copia "Hello" in a
strcat(a, " World");                 // concatena " World" ad a
cout << strcmp("abc", "abc");         // 0 – le due stringhe sono uguali
```

I problemi delle stringhe C-style

- Non puoi concatenarle con `+`
- Devi gestire la memoria manualmente
- È facile scrivere oltre i limiti del buffer (bug difficile da trovare)
- Il codice risultante è lungo e poco leggibile

```
char a[10] = "Ciao";
// ERRORE – non puoi fare a + " mondo" con le C-style strings
// Devi usare: strcat(a, " mondo");
```

Le stringhe moderne: `std::string`

`std::string` risolve tutti i problemi delle stringhe C-style. Per usarla, includi `<string>` :

```
#include <string>
using namespace std;

string nome = "Alice";
string cognome = "Bianchi";

// Concatenazione semplice con +
string nomeCompleto = nome + " " + cognome;

// Lunghezza con .length()
cout << nome.length() << endl;    // 5

// Confronto diretto con ==
if (nome == "Alice") {
    cout << "Ciao Alice!" << endl;
}
```

Tutto quello che con le C-style strings richiedeva funzioni speciali, con `std::string` si fa con operatori normali.

Confronto diretto

Operazione	C-style (char[])	std::string
Dichiarazione	<code>char s[20] = "ciao";</code>	<code>string s = "ciao";</code>
Lunghezza	<code>strlen(s)</code>	<code>s.length()</code>
Copia	<code>strcpy(dest, src)</code>	<code>dest = src</code>
Concatenazione	<code>strcat(a, b)</code>	<code>a + b</code>
Confronto	<code>strcmp(a, b) == 0</code>	<code>a == b</code>
Accesso a un carattere	<code>s[i]</code>	<code>s[i]</code>

Convertire tra i due tipi

A volte devi passare da uno all'altro (per esempio, alcune librerie richiedono il tipo C-style):

```
// Da string a char[] (C-style)
string str = "Ciao";
const char* cstr = str.c_str(); // converte in puntatore al C-string

// Da char[] a string – la conversione è automatica
char cArray[] = "Ciao";
string str2 = cArray;
```

Quando si usano le C-style strings

In pratica moderna, usa sempre `std::string`. Le C-style strings restano necessarie solo in casi particolari:

- Lavorare con librerie scritte in C che richiedono `const char*`
- Programmazione di sistema a basso livello
- Programmi dove ogni byte di memoria è importante

```
// Le librerie C spesso richiedono const char*
// Con std::string usi .c_str() per convertire
string nomefile = "dati.txt";
FILE* f = fopen(nomefile.c_str(), "r");
```

Esempio: confronto pratico

```
#include <iostream>
#include <string>
#include <cstring>
using namespace std;

int main() {
    // Approccio C-style – verboso e fragile
    char c_nome[50] = "Alice";
    char c_cognome[50] = "Bianchi";
    char c_completo[100];
    strcpy(c_completo, c_nome);    // copia il nome
    strcat(c_completo, " ");       // aggiunge uno spazio
    strcat(c_completo, c_cognome); // aggiunge il cognome
    cout << "C-style: " << c_completo << endl;

    // Approccio moderno con string – semplice e leggibile
    string s_nome = "Alice";
    string s_cognome = "Bianchi";
    string s_completo = s_nome + " " + s_cognome;
    cout << "string:  " << s_completo << endl;

    return 0;
}
```

Il codice con `std::string` è molto più semplice e difficile da sbagliare.

Iteratori in C++

Introduzione agli iteratori della STL in C++.

Cos'è un iteratore?

Gli algoritmi della STL (come `sort`, `find`, `count`) non lavorano direttamente con i contenitori. Lavorano con degli oggetti speciali chiamati **iteratori** che indicano "da dove a dove" operare.

Capire gli iteratori ti permette di usare tutti gli algoritmi della STL e di scorrere i contenitori in modi più sofisticati.

Cosa sono gli iteratori

Un iteratore è come un **segnalibro**: punta a un elemento del contenitore e può spostarsi all'elemento successivo (o precedente).

Pensa a un iteratore come a un dito che scorre su una lista: parte dall'inizio, si sposta verso destra elemento per elemento, e si ferma alla fine.

I metodi `begin()` e `end()`

Ogni contenitore STL ha due metodi fondamentali:

- `begin()` — restituisce un iteratore che punta al **primo** elemento
- `end()` — restituisce un iteratore che punta **dopo** l'ultimo elemento

```
vector<int> v = {10, 20, 30, 40, 50};

// begin() punta a 10
// end() punta a una posizione immaginaria dopo il 50
```

`end()` non punta a un elemento reale: è solo un segnaposto che indica "siamo arrivati alla fine".

Usare un iteratore

Dichiara un iteratore con `auto` (il compilatore capisce il tipo da solo):

```
vector<int> v = {10, 20, 30, 40, 50};

auto it = v.begin();           // it punta al primo elemento (10)
cout << *it << endl;          // 10 — il * "legge" il valore puntato

++it;                          // sposta l'iteratore al prossimo elemento
cout << *it << endl;          // 20

++it;
cout << *it << endl;          // 30
```

L'operatore `*` davanti all'iteratore "dereferenzia": legge il valore dell'elemento puntato.

Scorrere con un ciclo

```
vector<int> v = {10, 20, 30, 40, 50};

// Ciclo while con iteratore
auto it = v.begin();
while (it != v.end()) {
    cout << *it << " ";
    ++it; // sposta al prossimo
}
// Output: 10 20 30 40 50
```

```
// Ciclo for con iteratore – forma compatta
for (auto it = v.begin(); it != v.end(); ++it) {
    cout << *it << " ";
}
```

:::note Il range-based for (`for (int n : v)`) usa gli iteratori internamente. Quindi di solito non devi scrivere esplicitamente gli iteratori, ma capire come funzionano ti aiuta a usare la STL. :::

Iteratori che non modificano: `cbegin()` e `cend()`

Se vuoi scorrere gli elementi senza modificarli, usa `cbegin()` e `cend()` (la `c` sta per "constant"):

```
vector<int> v = {1, 2, 3};

for (auto it = v.cbegin(); it != v.cend(); ++it) {
    cout << *it << " ";
    // *it = 99; // ERRORE – con l'iteratore costante non puoi modificare
}
```

Scorrere al contrario: `rbegin()` e `rend()`

```
vector<int> v = {1, 2, 3, 4, 5};

for (auto it = v.rbegin(); it != v.rend(); ++it) {
    cout << *it << " ";
}
// Output: 5 4 3 2 1
```

`rbegin()` punta all'ultimo elemento, `rend()` punta "prima" del primo.

Iteratori e algoritmi STL

Gli iteratori sono il modo in cui colleghi i tuoi dati agli algoritmi:

```
#include <algorithm>
#include <vector>
using namespace std;

vector<int> v = {5, 2, 8, 1, 9, 3};

// Ordina tutti gli elementi da begin a end
sort(v.begin(), v.end());

// Cerca il valore 5 e restituisce un iteratore all'elemento trovato
auto it = find(v.begin(), v.end(), 5);
if (it != v.end()) {
    // it - v.begin() calcola la posizione (distanza dall'inizio)
    cout << "Trovato 5 alla posizione " << (it - v.begin()) << endl;
}

// Rimuove l'elemento alla posizione 2
v.erase(v.begin() + 2);
```

Se `find` non trova il valore, restituisce `v.end()`. Ecco perché il controllo `if (it != v.end())` è importante.

Esempio pratico

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> numeri = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3};

    // Stampa la lista originale usando un iteratore costante
    cout << "Originale: ";
    for (auto it = numeri.cbegin(); it != numeri.cend(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    // Ordiniamo il vector
    sort(numeri.begin(), numeri.end());
    cout << "Ordinato: ";
    for (int n : numeri) cout << n << " ";
    cout << endl;

    // Stampiamo al contrario con l'iteratore inverso
    cout << "Inverso: ";
    for (auto it = numeri.rbegin(); it != numeri.rend(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    // unique rimuove i duplicati consecutivi – richiede il vector ordinato
    auto fine = unique(numeri.begin(), numeri.end());
    numeri.erase(fine, numeri.end()); // rimuoviamo i "resti" lasciati da
unique
    cout << "Senza duplicati: ";
    for (int n : numeri) cout << n << " ";
    cout << endl;

    return 0;
}
```

Output:

```
Originale: 3 1 4 1 5 9 2 6 5 3  
Ordinato: 1 1 2 3 3 4 5 5 6 9  
Inverso: 9 6 5 5 4 3 3 2 1 1  
Senza duplicati: 1 2 3 4 5 6 9
```

Array Multidimensionali

Come usare array a due o più dimensioni in C++.

Array con più dimensioni

Un array normale è come una fila di scatole. Ma a volte i dati sono naturalmente organizzati in una **griglia**: i voti di più studenti in più materie, le celle di una scacchiera, i pixel di un'immagine.

Un **array bidimensionale** è come una tabella con righe e colonne: hai bisogno di due numeri per identificare ogni cella.

Dichiarare un array 2D

```
tipo nome[righe][colonne];
```

```
int tabella[3][4]; // 3 righe, 4 colonne – 12 celle in totale
```

Inizializzare un array 2D

Puoi inizializzarlo riga per riga, usando parentesi graffe interne:

```
int matrice[2][3] = {
    {1, 2, 3},    // riga 0
    {4, 5, 6}     // riga 1
};
```

Accedere agli elementi

Devi usare **due indici**: `[riga][colonna]`. Entrambi partono da 0.

```
int matrice[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};

cout << matrice[0][0] << endl; // 1 - riga 0, colonna 0
cout << matrice[0][2] << endl; // 3 - riga 0, colonna 2
cout << matrice[1][1] << endl; // 5 - riga 1, colonna 1
```

Immagina le coordinate come `[Y][X]`: prima la riga (verticale), poi la colonna (orizzontale).

Scorrere con due cicli annidati

Per visitare tutti gli elementi di una matrice, si usano due cicli `for` annidati: uno per le righe, uno per le colonne:

```
int matrice[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

for (int i = 0; i < 3; i++) {      // ciclo esterno: scorre le righe
    for (int j = 0; j < 3; j++) {  // ciclo interno: scorre le colonne
        cout << matrice[i][j] << "\t";
    }
    cout << endl; // vai a capo dopo ogni riga
}
```

Output:

```
1  2  3
4  5  6
7  8  9
```

Esempio: registro voti degli studenti

Un uso pratico: memorizzare i voti di più studenti in più materie.

```

#include <iostream>
using namespace std;

int main() {
    const int STUDENTI = 3;
    const int MATERIE = 4;

    // voti[studente][materia]
    int voti[STUDENTI][MATERIE] = {
        {8, 7, 9, 6},    // Alice
        {5, 8, 7, 9},    // Bob
        {9, 9, 8, 10}    // Carlo
    };

    string nomi[] = {"Alice", "Bob", "Carlo"};
    string materie[] = {"Matematica", "Italiano", "Inglese", "Storia"};

    // Stampa la tabella con le medie
    for (int s = 0; s < STUDENTI; s++) {
        cout << nomi[s] << ": ";
        int somma = 0;
        for (int m = 0; m < MATERIE; m++) {
            cout << voti[s][m] << " ";
            somma += voti[s][m];
        }
        double media = (double)somma / MATERIE;
        cout << "→ media: " << media << endl;
    }

    return 0;
}

```

Esempio: scacchiera

```
#include <iostream>
using namespace std;

int main() {
    char scacchiera[8][8];

    // Riempiamo la scacchiera alternando # e .
    for (int i = 0; i < 8; i++) {
        for (int j = 0; j < 8; j++) {
            // Se la somma degli indici è pari, casella nera
            if ((i + j) % 2 == 0) {
                scacchiera[i][j] = '#';
            } else {
                scacchiera[i][j] = '.';
            }
        }
    }

    // Stampa la scacchiera
    for (int i = 0; i < 8; i++) {
        for (int j = 0; j < 8; j++) {
            cout << scacchiera[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Array a tre dimensioni

Puoi avere anche array con tre o più dimensioni, anche se diventano difficili da visualizzare:

```
int cubo[2][3][4]; // 2 "strati", 3 righe, 4 colonne

// Accesso con tre indici
cubo[0][1][2] = 42;
```


Un array 3D è come un cubo: ha profondità, altezza e larghezza. Nella pratica, gli array 2D coprono la maggior parte dei casi.

Limitazioni degli array C++

Gli array tradizionali hanno alcuni svantaggi:

- La dimensione deve essere fissa e nota prima dell'esecuzione
- Non si "ricordano" quanti elementi contengono
- Non controllano se stai uscendo dai limiti

Per questi motivi, nelle applicazioni moderne si usa spesso `vector` (vedi la sezione apposita), che è più flessibile e sicuro.

Cenni alla STL

Panoramica della Standard Template Library del C++ con i contenitori più utili.

Cos'è la STL?

Quando programmi, ti ritrovi spesso ad avere gli stessi problemi: "devo memorizzare una lista di elementi", "devo associare nomi a valori", "devo gestire una coda". Risolverli da zero ogni volta sarebbe faticoso.

La **STL** (Standard Template Library) è una raccolta di "strumenti pronti all'uso" inclusa nel C++. Fornisce strutture dati già pronte (liste, dizionari, code...) e algoritmi già scritti (ordinamento, ricerca...).

Cosa contiene la STL

La STL ha tre componenti principali:

- **Contenitori:** strutture per memorizzare dati (`vector` , `map` , `set` , ...)
- **Algoritmi:** funzioni per operare sui dati (`sort` , `find` , `count` , ...)
- **Iteratori:** oggetti per scorrere i contenitori (trattati nella sezione apposita)

I contenitori principali

vector — Lista dinamica

Già trattato nella sezione dedicata. È il contenitore più usato in assoluto.

```
#include <vector>
vector<int> v = {1, 2, 3, 4, 5};
v.push_back(6);
```

map — Dizionario (chiave → valore)

Un `map` associa ogni chiave a un valore. È come un dizionario: cerchi una parola (chiave) e trovi la sua definizione (valore). Le chiavi sono mantenute in ordine alfabetico/numerico.

```

#include <map>
using namespace std;

map<string, int> eta;
eta["Alice"] = 16;
eta["Bob"] = 17;
eta["Carlo"] = 15;

// Leggi il valore associato a una chiave
cout << eta["Alice"] << endl;    // 16

// Scorri tutte le coppie
for (auto& coppia : eta) {
    // .first è la chiave, .second è il valore
    cout << coppia.first << ": " << coppia.second << endl;
}
// Output (in ordine alfabetico):
// Alice: 16
// Bob: 17
// Carlo: 15

// Controlla se una chiave esiste
if (eta.count("Alice") > 0) {
    cout << "Alice è nel registro" << endl;
}

// Rimuovi una coppia
eta.erase("Bob");

```

unordered_map — Dizionario veloce

Come `map`, ma non mantiene l'ordine delle chiavi. In cambio è più veloce:

```
#include <unordered_map>
using namespace std;

unordered_map<string, int> contaParole;
contaParole["ciao"]++; // se la chiave non esiste, viene creata con
valore 0
contaParole["mondo"]++;
contaParole["ciao"]++;

for (auto& p : contaParole) {
    cout << p.first << ": " << p.second << " volte" << endl;
}
```

Usa `map` se ti serve l'ordine, `unordered_map` se ti serve la velocità.

set — Insieme (senza duplicati)

Un `set` memorizza elementi unici, in ordine. Se inserisci un valore già presente, non cambia nulla.

```
#include <set>
using namespace std;

set<int> numeri = {5, 2, 8, 2, 1, 9, 1}; // i duplicati vengono ignorati
// numeri contiene: {1, 2, 5, 8, 9}

numeri.insert(3); // aggiunge 3
numeri.erase(8); // rimuove 8

if (numeri.count(5) > 0) {
    cout << "5 è nel set" << endl;
}

for (int n : numeri) cout << n << " ";
// Output: 1 2 3 5 9
```

list — Lista collegata

Una `list` è efficiente quando aggiungi e rimuovi elementi in mezzo alla lista molto spesso:

```
#include <list>
using namespace std;

list<int> l = {1, 2, 4, 5};
l.push_front(0); // aggiunge all'inizio
l.push_back(6);  // aggiunge alla fine
l.pop_front();   // rimuove il primo

for (int n : l) cout << n << " ";
```

Per la maggior parte dei casi usa `vector`. Usa `list` solo se fai molte inserzioni/rimozioni nel mezzo.

stack — Pila (ultimo entrato, primo uscito)

Una pila (stack) funziona come una pila di piatti: metti sempre sopra, e prendi sempre dall'alto. L'ultimo elemento aggiunto è il primo a uscire (LIFO: Last In, First Out).

```
#include <stack>
using namespace std;

stack<int> pila;
pila.push(1); // metti 1 in cima
pila.push(2); // metti 2 sopra all'1
pila.push(3); // metti 3 sopra al 2

cout << pila.top() << endl; // 3 - elemento in cima
pila.pop();                 // rimuovi l'elemento in cima
cout << pila.top() << endl; // 2
```

queue — Coda (primo entrato, primo uscito)

Una coda (queue) funziona come la cassa al supermercato: il primo ad arrivare è il primo ad essere servito (FIFO: First In, First Out).

```
#include <queue>
using namespace std;

queue<string> coda;
coda.push("Alice");    // Alice arriva per prima
coda.push("Bob");
coda.push("Carlo");

cout << coda.front() << endl; // "Alice" – la prima ad essere servita
coda.pop();              // Alice viene servita e va via
cout << coda.front() << endl; // ora tocca a "Bob"
```

Riepilogo: quale contenitore usare?

Contenitore	Quando usarlo
<code>vector</code>	Uso generale, accesso per posizione
<code>map</code>	Associare chiavi a valori, con ordine
<code>unordered_map</code>	Associare chiavi a valori, più veloce
<code>set</code>	Memorizzare valori unici
<code>list</code>	Molte inserzioni/rimozioni in mezzo
<code>stack</code>	Algoritmi che tornano indietro (backtracking)
<code>queue</code>	Code di elaborazione

Gli algoritmi della STL

Con `#include <algorithm>` hai accesso a molti algoritmi già pronti:

```

#include <algorithm>
#include <vector>
#include <numeric>
using namespace std;

vector<int> v = {5, 2, 8, 1, 9, 3};

// Ordina in ordine crescente
sort(v.begin(), v.end());

// Cerca se un valore è presente (richiede il vector ordinato)
bool trovato = binary_search(v.begin(), v.end(), 5);

// Conta quante volte appare un valore
int quanti = count(v.begin(), v.end(), 3);

// Trova il massimo e il minimo
auto massimo = *max_element(v.begin(), v.end());
auto minimo = *min_element(v.begin(), v.end());
cout << "Max: " << massimo << ", Min: " << minimo << endl;

// Somma tutti gli elementi (da <numeric>)
int somma = accumulate(v.begin(), v.end(), 0);

```

Vettori (vector)

Come usare il contenitore vector della STL in C++.

Cos'è un vector?

Un array normale ha un problema fondamentale: la dimensione deve essere decisa prima di eseguire il programma. Se non sai in anticipo quanti dati avrai, sei bloccato.

Il `vector` risolve questo problema: è come un array, ma **cambia dimensione automaticamente** mentre il programma gira. Aggiungi elementi e lui cresce. Rimuovili e lui si riduce.

Per usarlo, includi `<vector>` :

```
#include <vector>
using namespace std;
```

Creare un vector

```
// Vector vuoto – aggiungeremo elementi dopo
vector<int> v1;

// Vector con 5 elementi, tutti a zero
vector<int> v2(5);           // {0, 0, 0, 0, 0}

// Vector con valori già impostati
vector<int> v3 = {1, 2, 3, 4, 5};

// Vector con 4 elementi, tutti uguali a 7
vector<int> v4(4, 7);        // {7, 7, 7, 7}

// Vector di stringhe
vector<string> nomi = {"Alice", "Bob", "Carlo"};
```

Il tipo tra `<>` indica cosa contiene il vector. `vector<int>` contiene interi, `vector<string>` contiene stringhe, e così via.

Accedere agli elementi

Come gli array, si usa l'indice `[]` (parte da 0):

```
vector<int> v = {10, 20, 30, 40, 50};

cout << v[0] << endl;    // 10 – primo elemento
cout << v[2] << endl;    // 30
cout << v[4] << endl;    // 50 – ultimo elemento

// Modifica di un elemento
v[1] = 99;
cout << v[1] << endl;    // 99
```


Per un accesso più sicuro (il programma ti avvisa se esci dai limiti), usa `.at()` :

```
cout << v.at(2) << endl;    // 30
// v.at(10);                // lancia un errore se 10 è fuori range
```

Aggiungere e rimuovere elementi

```
vector<int> v;

// Aggiunge in coda – il vector cresce automaticamente
v.push_back(10);
v.push_back(20);
v.push_back(30);
// v è ora {10, 20, 30}

// Rimuove l'ultimo elemento
v.pop_back();
// v è ora {10, 20}
```

Informazioni sul vector

```
vector<int> v = {1, 2, 3, 4, 5};

cout << v.size() << endl;    // 5 – quanti elementi contiene
cout << v.empty() << endl;   // false (0) – è vuoto?

v.pop_back();
cout << v.size() << endl;    // 4

v.clear();                   // svuota completamente il vector
cout << v.size() << endl;    // 0
cout << v.empty() << endl;   // true (1)
```

Scorrere un vector

Con indice (come un array):

```
vector<int> v = {10, 20, 30, 40, 50};

for (int i = 0; i < v.size(); i++) {
    cout << v[i] << " ";
}
```

Con range-based for (il modo preferito — più semplice):

```
for (int n : v) {
    cout << n << " ";
}
```

Per modificare gli elementi mentre scorri, usa `&` :

```
// Moltiplica ogni elemento per 2
for (int& n : v) {
    n *= 2;
}
```

Primo e ultimo elemento

```
vector<int> v = {10, 20, 30};

cout << v.front() << endl;    // 10 - primo elemento
cout << v.back() << endl;     // 30 - ultimo elemento
```

Inserire e rimuovere in posizioni specifiche

```
vector<int> v = {1, 2, 4, 5};

// Inserisce 3 alla posizione 2 (tra il 2 e il 4)
v.insert(v.begin() + 2, 3);
// v è ora {1, 2, 3, 4, 5}

// Rimuove l'elemento alla posizione 1 (il 2)
v.erase(v.begin() + 1);
// v è ora {1, 3, 4, 5}
```

Ordinare un vector

Per ordinare, includi anche `<algorithm>` :

```
#include <algorithm>

vector<int> v = {5, 2, 8, 1, 9, 3};

sort(v.begin(), v.end());           // ordine crescente → {1, 2, 3, 5, 8, 9}
sort(v.begin(), v.end(), greater<int>()); // ordine decrescente → {9, 8, 5, 3, 2, 1}
```

Esempio pratico: raccolta voti interattiva

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> voti;
    int voto;

    cout << "Inserisci i voti (0 per terminare):" << endl;
    while (true) {
        cin >> voto;
        if (voto == 0) break;

        if (voto >= 1 && voto <= 10) {
            voti.push_back(voto); // aggiungiamo il voto al vector
        } else {
            cout << "Voto non valido, ignorato." << endl;
        }
    }

    if (voti.empty()) {
        cout << "Nessun voto inserito." << endl;
        return 0;
    }

    // Calcoliamo le statistiche
    int somma = 0;
    for (int v : voti) somma += v;
    double media = (double)somma / voti.size();

    // Ordiniamo per trovare min e max
    sort(voti.begin(), voti.end());

    cout << "\nNumero di voti: " << voti.size() << endl;
    cout << "Media: " << media << endl;
    cout << "Minimo: " << voti.front() << endl; // primo dopo ordinamento
    cout << "Massimo: " << voti.back() << endl; // ultimo dopo
ordinamento

    cout << "Voti ordinati: ";
    for (int v : voti) cout << v << " ";
    cout << endl;
```

```
}  
    return 0;  
}
```

break e continue

Come controllare il flusso dei cicli con break e continue in C++.

Controllare il flusso dentro un ciclo

A volte, mentre stai scorrendo una lista, vuoi fermarti non appena hai trovato quello che cercavi. Oppure vuoi saltare certi elementi e andare direttamente al prossimo.

`break` e `continue` sono due istruzioni che ti danno questo controllo preciso sul comportamento dei cicli.

break : esci dal ciclo

`break` interrompe immediatamente il ciclo. L'esecuzione continua con la prima istruzione dopo il ciclo, come se il ciclo fosse finito.

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        break; // usciamo quando i vale 5  
    }  
    cout << i << " ";  
}  
cout << "fine";  
// Output: 0 1 2 3 4 fine
```

Quando usare break

Il caso più comune: cercare qualcosa in una lista. Appena lo trovi, non ha senso continuare a cercare:

```

int numeri[] = {10, 25, 3, 47, 8, 15};
int dimensione = 6;
int daQuerare = 47;
bool trovato = false;

for (int i = 0; i < dimensione; i++) {
    if (numeri[i] == daQuerare) {
        cout << "Trovato " << daQuerare << " alla posizione " << i << endl;
        trovato = true;
        break; // trovato! inutile continuare a cercare
    }
}

if (!trovato) {
    cout << daQuerare << " non trovato." << endl;
}

```

break nei cicli annidati

break esce solo dal ciclo più interno in cui si trova. Il ciclo esterno continua normalmente:

```

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) {
            break; // esce solo dal ciclo interno (j)
        }
        cout << "(" << i << "," << j << ") ";
    }
    cout << endl;
}

```

Output:

```

(0,0)
(1,0)
(2,0)

```

continue : salta al prossimo giro

`continue` non esce dal ciclo. Salta solo il resto del blocco corrente e passa direttamente alla prossima iterazione.

```
for (int i = 0; i < 10; i++) {  
    if (i % 2 == 0) {  
        continue; // se il numero è pari, saltiamo questa iterazione  
    }  
    cout << i << " "; // stampiamo solo i dispari  
}  
// Output: 1 3 5 7 9
```

Quando usare `continue`

Utile per saltare certi casi senza dover annidare tutto il codice dentro un `if` :

```
// Stampa solo i numeri non divisibili per 3  
for (int i = 1; i <= 20; i++) {  
    if (i % 3 == 0) {  
        continue; // saltiamo i multipli di 3  
    }  
    cout << i << " ";  
}  
// Output: 1 2 4 5 7 8 10 11 13 14 16 17 19 20
```

Un altro esempio: ignorare i dati non validi e continuare con quelli validi:

```

int voti[] = {8, -1, 7, 150, 6, 9};
int n = 6;
int somma = 0;
int contatore = 0;

for (int i = 0; i < n; i++) {
    if (voti[i] < 0 || voti[i] > 10) {
        cout << "Voto non valido ignorato: " << voti[i] << endl;
        continue; // saltiamo questo voto e passiamo al prossimo
    }
    somma += voti[i];
    contatore++;
}

if (contatore > 0) {
    cout << "Media voti validi: " << (double)somma / contatore << endl;
}

```

break e continue nel while

Funzionano allo stesso modo anche in `while` e `do...while` :

```

// Ciclo "infinito" controllato da break
int i = 0;
while (true) {
    if (i >= 5) {
        break; // uscita manuale dal ciclo
    }
    cout << i << " ";
    i++;
}
// Output: 0 1 2 3 4

```



```
int i = 0;
while (i < 10) {
    i++;
    if (i == 5) {
        continue; // salta il 5
    }
    cout << i << " ";
}
// Output: 1 2 3 4 6 7 8 9 10
```

Consigli pratici

- Usa `break` e `continue` con moderazione. Troppi rendono il codice difficile da seguire.
- Se un ciclo ha molti `break`, potrebbe essere riscrivibile con una condizione più precisa:

```
// Con break (meno chiaro)
for (int i = 0; i < 100; i++) {
    if (i * i > 50) {
        break;
    }
    cout << i << " ";
}

// Alternativa più chiara: la condizione di uscita è nella testa del for
for (int i = 0; i < 100 && i * i <= 50; i++) {
    cout << i << " ";
}
```

- Se usi `break` o `continue` per un motivo non ovvio, aggiungi un commento che spiega il perché.

Condizioni in C++

Prendere decisioni nel codice con `if`, `else if`, `else` e `switch` in C++.

Perché le condizioni?

Un programma che fa sempre la stessa cosa non è molto utile. I programmi interessanti prendono **decisioni**: se l'utente è maggiorenne, mostra il contenuto; se il voto è sufficiente, passa alla classe successiva; se piove, porta l'ombrello.

Le condizioni permettono al programma di scegliere tra strade diverse in base a una situazione.

L'istruzione **if**

if significa "se". Se la condizione tra parentesi è vera, il blocco di codice viene eseguito. Altrimenti viene saltato.

```
if (condizione) {  
    // questo codice viene eseguito solo se la condizione è vera  
}
```

```
int eta = 18;  
  
if (eta >= 18) {  
    cout << "Sei maggiorenne." << endl;  
}  
// Se eta fosse 15, questa riga non verrebbe mai stampata
```

Nota: in C++ la condizione **deve** stare tra parentesi tonde **()**.

L'istruzione **else**

else significa "altrimenti". Definisce cosa fare quando la condizione è falsa:

```
int eta = 15;

if (eta >= 18) {
    cout << "Sei maggiorenne." << endl;
} else {
    cout << "Sei minorenni." << endl;
}

// Stampa: "Sei minorenni."
```

L'istruzione **else if**

Quando hai più di due possibilità, usa **else if** per controllare più condizioni in sequenza:

```
int voto = 75;

if (voto >= 90) {
    cout << "Ottimo!" << endl;
} else if (voto >= 70) {
    cout << "Buono!" << endl;      // ← questa viene stampata
} else if (voto >= 60) {
    cout << "Sufficiente." << endl;
} else {
    cout << "Insufficiente." << endl;
}
```

Il programma controlla le condizioni dall'alto verso il basso e si ferma alla prima che risulta vera. Le successive vengono ignorate.

Condizioni annidate

Puoi mettere un **if** dentro un altro **if**. Pensa a esso come a una serie di domande: prima la domanda principale, poi le sotto-domande.

```

int eta = 20;
bool haPatente = true;

if (eta >= 18) {
    // siamo maggiorenni – ora controlliamo la patente
    if (haPatente) {
        cout << "Puoi guidare." << endl;
    } else {
        cout << "Sei maggiorenne ma non hai la patente." << endl;
    }
} else {
    cout << "Sei minorenne, non puoi guidare." << endl;
}

```

Usare gli operatori logici

Puoi combinare più condizioni in una sola usando `&&` (E), `||` (O) e `!` (NON):

```

int eta = 16;
bool conAccompagnatore = true;

// Può entrare se è maggiorenne OPPURE se ha un accompagnatore
if (eta >= 18 || conAccompagnatore) {
    cout << "Puoi entrare." << endl;
}

```

```

int temperatura = 22;
bool piove = false;

// Perfetto per uscire se è caldo E non piove
if (temperatura > 20 && !piove) {
    cout << "Perfetto per uscire!" << endl;
}

```

Il costrutto `switch`

Quando devi confrontare una variabile con molti valori fissi, `switch` è più leggibile di una lunga serie di `else if`. Funziona come un selettore: vai direttamente al caso corrispondente.

```
int giorno = 3;

switch (giorno) {
    case 1:
        cout << "Lunedì" << endl;
        break;
    case 2:
        cout << "Martedì" << endl;
        break;
    case 3:
        cout << "Mercoledì" << endl; // ← questo viene eseguito
        break;
    case 4:
        cout << "Giovedì" << endl;
        break;
    case 5:
        cout << "Venerdì" << endl;
        break;
    default:
        cout << "Weekend" << endl; // eseguito se nessun case
        corrisponde
}
```

Output: Mercoledì

I componenti di `switch`

- **case valore:** — esegue il codice se la variabile vale quel numero
- **break** — esce dallo switch. Senza di esso, l'esecuzione continua nei case successivi!
- **default** — viene eseguito se nessun case corrisponde (facoltativo)

Attenzione al `break` !

Dimenticare il `break` è un errore comune. Senza di esso, dopo aver trovato il case giusto il programma continua ad eseguire anche i case successivi:

```

int x = 2;
switch (x) {
    case 1:
        cout << "uno" << endl;
    case 2:
        cout << "due" << endl;    // eseguito (corretto)
    case 3:
        cout << "tre" << endl;    // eseguito anche questo! (manca break)
        break;
}
// Output: due
//         tre

```

A volte questo comportamento è intenzionale. Per esempio, se vuoi che più case eseguano lo stesso codice:

```

char voto = 'B';
switch (voto) {
    case 'A':
    case 'B':
        // sia A che B arrivano qui
        cout << "Promosso con lode" << endl;
        break;
    case 'C':
        cout << "Promosso" << endl;
        break;
    default:
        cout << "Bocciato" << endl;
}

```

L'operatore ternario

Per condizioni semplici con solo due risultati, esiste una forma compatta su una riga sola:

```

// condizione ? valore_se_vero : valore_se_falso
int eta = 20;
string stato = (eta >= 18) ? "maggiorenne" : "minorenne";
cout << stato << endl;    // maggiorenne

```

Usa questa forma solo quando è chiara da leggere. Se la condizione è complessa, preferisci un normale `if / else`.

Esempio pratico

```
#include <iostream>
using namespace std;

int main() {
    int punteggio;
    cout << "Inserisci il punteggio (0-100): ";
    cin >> punteggio;

    // Prima controlliamo che il valore sia valido
    if (punteggio < 0 || punteggio > 100) {
        cout << "Punteggio non valido!" << endl;
    } else if (punteggio >= 90) {
        cout << "Voto: A – Eccellente!" << endl;
    } else if (punteggio >= 80) {
        cout << "Voto: B – Molto buono" << endl;
    } else if (punteggio >= 70) {
        cout << "Voto: C – Buono" << endl;
    } else if (punteggio >= 60) {
        cout << "Voto: D – Sufficiente" << endl;
    } else {
        cout << "Voto: F – Insufficiente" << endl;
    }

    return 0;
}
```

Ciclo do...while

Il ciclo do...while in C++, che esegue il blocco almeno una volta.

do-while : quando eseguire almeno una volta

A volte vuoi che il tuo programma faccia qualcosa almeno una volta, e poi decida se ripeterlo.

Per esempio: mostra il menu all'utente, poi chiedi se vuole continuare. Il menu deve apparire sempre almeno una volta, anche prima di sapere cosa vuole l'utente.

Il ciclo `do...while` fa esattamente questo: **esegue il blocco, poi controlla la condizione**.

La struttura del `do...while`

```
do {  
    // codice da eseguire almeno una volta  
} while (condizione);
```

Nota il **punto e virgola** dopo la parentesi tonda finale — è obbligatorio e facile da dimenticare.

Confronto con il `while` normale

Il `while` controlla la condizione **prima** di eseguire il blocco. Se la condizione è già falsa, non esegue mai nulla:

```
int x = 10;  
while (x < 5) {  
    cout << "while: " << x << endl; // non viene mai stampato  
}
```

Il `do...while` esegue il blocco **almeno una volta**, poi controlla:

```
int y = 10;  
do {  
    cout << "do-while: " << y << endl; // viene stampato una volta  
} while (y < 5);
```

Output:

```
do-while: 10
```

Anche se `y = 10` non è minore di 5, il blocco viene eseguito lo stesso la prima volta.

Il caso d'uso classico: validare l'input

Il `do...while` è perfetto quando vuoi chiedere qualcosa all'utente e ripetere finché non inserisce un valore corretto:

```
int numero;

do {
    cout << "Inserisci un numero tra 1 e 10: ";
    cin >> numero;

    if (numero < 1 || numero > 10) {
        cout << "Valore non valido, riprova." << endl;
    }
} while (numero < 1 || numero > 10);

cout << "Hai inserito: " << numero << endl;
```

Con il `while` normale, dovresti leggere l'input anche prima del ciclo (duplicando il codice). Con `do...while` il codice è più pulito.

Menu interattivo

Il `do...while` è ideale per i menu: il menu deve sempre apparire almeno una volta, e si ripete finché l'utente non sceglie di uscire.

```

#include <iostream>
using namespace std;

int main() {
    int scelta;

    do {
        // Il menu viene mostrato all'inizio di ogni giro
        cout << "\n=== Menu ===" << endl;
        cout << "1. Saluta" << endl;
        cout << "2. Conta fino a 5" << endl;
        cout << "3. Esci" << endl;
        cout << "Scelta: ";
        cin >> scelta;

        switch (scelta) {
            case 1:
                cout << "Ciao! Come stai?" << endl;
                break;
            case 2:
                for (int i = 1; i <= 5; i++) {
                    cout << i << " ";
                }
                cout << endl;
                break;
            case 3:
                cout << "Arrivederci!" << endl;
                break;
            default:
                cout << "Scelta non valida." << endl;
        }

    } while (scelta != 3); // ripete finché l'utente non sceglie "Esci"

    return 0;
}

```

Contatore con **do...while**

Funziona anche per contare, come **while** e **for** :

```
int i = 1;

do {
    cout << i << " ";
    i++;
} while (i <= 5);

// Output: 1 2 3 4 5
```

Riepilogo dei tre tipi di ciclo

Ciclo	La condizione viene controllata	Esecuzioni minime
while	Prima del blocco	0 — potrebbe non eseguire mai
for	Prima del blocco	0 — potrebbe non eseguire mai
do...while	Dopo il blocco	1 — esegue almeno una volta

Il `do...while` è meno frequente degli altri due, ma è la scelta più naturale quando sai con certezza che il blocco deve essere eseguito almeno una volta.

Esempio pratico: indovina il numero

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

int main() {
    srand(time(0)); // inizializza il generatore di numeri casuali con
    l'ora attuale
    int numeroDaIndovinare = rand() % 100 + 1; // numero casuale tra 1 e
    100
    int tentativo;
    int contatore = 0;

    cout << "Ho pensato un numero tra 1 e 100. Indovinalo!" << endl;

    do {
        cout << "Tentativo " << (contatore + 1) << ": ";
        cin >> tentativo;
        contatore++;

        if (tentativo < numeroDaIndovinare) {
            cout << "Troppo basso!" << endl;
        } else if (tentativo > numeroDaIndovinare) {
            cout << "Troppo alto!" << endl;
        }

    } while (tentativo != numeroDaIndovinare); // ripeti finché non
    indovina

    cout << "Bravo! Hai indovinato in " << contatore << " tentativi!" <<
    endl;

    return 0;
}
```

Ciclo for

Come usare il ciclo for in C++ per iterazioni controllate.

A cosa serve il ciclo **for** ?

Immagina di dover stampare i numeri da 1 a 100. Scriveresti 100 righe di codice? Certo che no.

Il ciclo **for** ti permette di ripetere un blocco di codice un numero preciso di volte. Scrivi il codice una volta sola, e il programma lo esegue quante volte vuoi.

La struttura del ciclo **for**

```
for (inizio; condizione; aggiornamento) {  
    // codice da ripetere  
}
```

Un esempio concreto:

```
for (int i = 0; i < 5; i++) {  
    cout << i << endl;  
}
```

Output:

```
0  
1  
2  
3  
4
```

Le tre parti del **for**

```
for (int i = 0; i < 5; i++) {  
    //      ^^^^^^^^^  ^^^^^  ^^^  
    //      inizio    cond.   aggiorn.
```

1. **Inizio** (`int i = 0`): viene eseguita una sola volta, all'inizio. Crea il "contatore" del ciclo.
2. **Condizione** (`i < 5`): viene controllata prima di ogni ripetizione. Se è falsa, il ciclo si ferma.
3. **Aggiornamento** (`i++`): viene eseguito alla fine di ogni ripetizione. Di solito aumenta il contatore.

Pensa al ciclo `for` come a un cronometro: parti da zero, vai avanti di uno in uno, ti fermi quando arrivi al limite.

Contare in avanti

```
// Da 1 a 10
for (int i = 1; i <= 10; i++) {
    cout << i << " ";
}
// Output: 1 2 3 4 5 6 7 8 9 10
```

Contare all'indietro

```
// Da 10 a 1 – basta cambiare inizio, condizione e aggiornamento
for (int i = 10; i >= 1; i--) {
    cout << i << " ";
}
// Output: 10 9 8 7 6 5 4 3 2 1
```

Passo diverso da 1

Non devi necessariamente aumentare di uno alla volta:

```
// Solo i numeri pari da 0 a 10
for (int i = 0; i <= 10; i += 2) {
    cout << i << " ";
}
// Output: 0 2 4 6 8 10

// Multipli di 5
for (int i = 5; i <= 25; i += 5) {
    cout << i << " ";
}
// Output: 5 10 15 20 25
```

Cicli annidati

Puoi mettere un ciclo dentro un altro. Utile per lavorare con griglie, tabelle o schemi:

```
// Per ogni riga (i) eseguiamo tutto il ciclo delle colonne (j)
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        cout << i * j << "\t"; // \t è il tabulatore (allinea le colonne)
    }
    cout << endl; // vai a capo dopo ogni riga
}
```

Output (tabellina pitagorica 3x3):

```
1  2  3
2  4  6
3  6  9
```

Il for su una collezione (range-based for)

In C++ moderno puoi usare una forma più semplice di `for` per scorrere gli elementi di una lista o di una stringa, senza gestire manualmente l'indice:

```
// Scorre ogni carattere della stringa nome
string nome = "Alice";
for (char c : nome) {
    cout << c << " ";
}
// Output: A l i c e

// Scorre ogni numero dell'array
int numeri[] = {10, 20, 30, 40, 50};
for (int n : numeri) {
    cout << n << " ";
}
// Output: 10 20 30 40 50
```

La sintassi è: `for (tipo elemento : collezione)` . Si legge "per ogni elemento nella collezione".

Usare **auto** per non specificare il tipo

Se non vuoi scrivere il tipo dell'elemento, puoi usare `auto` e lasciare che il compilatore lo capisca da solo:

```
int numeri[] = {1, 2, 3, 4, 5};
for (auto n : numeri) {
    cout << n * 2 << " "; // stampa il doppio di ogni numero
}
// Output: 2 4 6 8 10
```


Esempio pratico: tabella delle potenze

```
#include <iostream>
using namespace std;

int main() {
    // Intestazione della tabella
    cout << "n\tn2\tn3" << endl;
    cout << "----\t----\t----" << endl;

    // Calcoliamo n2, n3 per ogni numero da 1 a 10
    for (int n = 1; n <= 10; n++) {
        cout << n << "\t" << n*n << "\t" << n*n*n << endl;
    }

    return 0;
}
```

Output:

n	n ²	n ³
----	----	----
1	1	1
2	4	8
3	9	27
4	16	64
5	25	125
...		

Ciclo while

Come ripetere istruzioni con il ciclo while in C++.

A cosa serve il ciclo **while** ?

A volte non sai in anticipo quante volte dovrai ripetere qualcosa. Per esempio: continua a chiedere all'utente un numero finché non inserisce uno valido. Oppure: elabora dati finché non hai finito la lista.

Il ciclo `while` ripete un blocco di codice **finché una condizione rimane vera**. Si ferma solo quando la condizione diventa falsa.

La struttura del ciclo `while`

```
while (condizione) {  
    // codice da ripetere  
}
```

Un esempio concreto:

```
int i = 1;  
  
while (i <= 5) {  
    cout << i << endl;  
    i++; // aumentiamo i di 1 ad ogni giro  
}
```

Output:

```
1  
2  
3  
4  
5
```

Come funziona, passo per passo:

1. Controlla la condizione `i <= 5`
2. Se è vera, esegue il blocco
3. Torna al punto 1
4. Quando la condizione è falsa, si ferma

Il contatore: non dimenticarlo!

Di solito si usa una variabile **contatore** per tenere traccia dei giri. Devi:

1. Inizializzarla **prima** del ciclo
2. Aggiornarla **dentro** il ciclo

```
int contatore = 0;

while (contatore < 3) {
    cout << "Esecuzione " << contatore + 1 << endl;
    contatore++; // senza questa riga: ciclo infinito!
}
```

Il ciclo infinito: cosa evitare

Se la condizione non diventa mai falsa, il ciclo gira per sempre. Il programma si blocca:

```
// ATTENZIONE: questo è un ciclo infinito!
int i = 1;
while (i <= 5) {
    cout << i << endl;
    // manca i++ - i rimane sempre 1, la condizione non cambia mai
}
```

Se ti capita, puoi fermare il programma premendo **Ctrl+C** nel terminale.

Quando il **while** è più adatto del **for**

Il **while** eccelle quando non sai in anticipo quante ripetizioni ci saranno. Per esempio, continuare a leggere input finché l'utente non decide di uscire:

```

int numero;
cout << "Inserisci un numero (0 per uscire): ";
cin >> numero;

while (numero != 0) {
    // calcoliamo il quadrato del numero inserito
    cout << "Il quadrato di " << numero << " è " << numero * numero <<
endl;

    cout << "Inserisci un numero (0 per uscire): ";
    cin >> numero;
}

cout << "Arrivederci!" << endl;

```

Ciclo con accumulatore

Un pattern molto comune: sommare o raccogliere valori uno per uno:

```

int somma = 0;    // accumulatore – parte da zero
int numero;
int n = 5;        // quanti numeri vogliamo leggere

cout << "Inserisci " << n << " numeri:" << endl;

int i = 0;
while (i < n) {
    cout << "Numero " << (i + 1) << ": ";
    cin >> numero;
    somma += numero; // aggiungiamo il numero alla somma totale
    i++;
}

cout << "Somma: " << somma << endl;
cout << "Media: " << (double)somma / n << endl;

```

Condizioni complesse

Puoi combinare più condizioni con `&&` e `||` :

```

int tentativi = 0;
int password;
const int PASSWORD_CORRETTA = 1234;

// Continua finché la password è sbagliata E i tentativi sono meno di 3
while (password != PASSWORD_CORRETTA && tentativi < 3) {
    cout << "Inserisci la password: ";
    cin >> password;
    tentativi++;

    if (password != PASSWORD_CORRETTA) {
        cout << "Password errata. Tentativi rimasti: " << (3 - tentativi)
<< endl;
    }
}

if (password == PASSWORD_CORRETTA) {
    cout << "Accesso consentito!" << endl;
} else {
    cout << "Accesso bloccato dopo 3 tentativi." << endl;
}

```

Esempio pratico: calcolatrice interattiva

```
#include <iostream>
using namespace std;

int main() {
    char continua = 's';

    // Continuiamo finché l'utente vuole fare calcoli
    while (continua == 's' || continua == 'S') {
        double a, b;
        char operazione;

        cout << "Inserisci il primo numero: ";
        cin >> a;
        cout << "Operazione (+, -, *, /): ";
        cin >> operazione;
        cout << "Inserisci il secondo numero: ";
        cin >> b;

        if (operazione == '+') {
            cout << "Risultato: " << a + b << endl;
        } else if (operazione == '-') {
            cout << "Risultato: " << a - b << endl;
        } else if (operazione == '*') {
            cout << "Risultato: " << a * b << endl;
        } else if (operazione == '/') {
            if (b != 0) {
                cout << "Risultato: " << a / b << endl;
            } else {
                cout << "Errore: divisione per zero!" << endl;
            }
        }

        cout << "Vuoi continuare? (s/n): ";
        cin >> continua;
    }

    cout << "Ciao!" << endl;
    return 0;
}
```

Funzioni in C++

Come definire e usare le funzioni in C++.

Cos'è una funzione?

Immagina di dover calcolare l'area di un cerchio in dieci punti diversi del tuo programma. Scriveresti la stessa formula dieci volte? E se poi vuoi cambiarla, la aggiorni in dieci posti?

Le **funzioni** risolvono questo problema: scrivi il codice una volta, gli dai un nome, e lo richiami ogni volta che ne hai bisogno. È come salvare una ricetta: la scrivi una volta, poi la segui ogni volta che vuoi cucinare quel piatto.

Definire una funzione

Una funzione si scrive così:

```
tipo_ritorno nomeFunzione(tipo parametro1, tipo parametro2) {  
    // codice della funzione  
    return valore; // se il tipo di ritorno non è void  
}
```

La funzione più semplice: non riceve nulla e non restituisce nulla:

```
void saluta() {  
    cout << "Ciao!" << endl;  
}
```

`void` significa "niente": questa funzione non restituisce nessun valore.

Chiamare una funzione

Una volta definita, puoi "chiamarla" (cioè eseguirla) scrivendo il suo nome seguito dalle parentesi:

```

void saluta() {
    cout << "Ciao!" << endl;
}

int main() {
    saluta();    // esegue la funzione – stampa "Ciao!"
    saluta();    // la puoi chiamare quante volte vuoi
    return 0;
}

```

Passare informazioni: i parametri

I **parametri** sono informazioni che puoi passare alla funzione quando la chiami:

```

// Questa funzione riceve un nome e lo usa per salutare
void saluta(string nome) {
    cout << "Ciao, " << nome << "!" << endl;
}

int main() {
    saluta("Alice");    // stampa: Ciao, Alice!
    saluta("Bob");      // stampa: Ciao, Bob!
    return 0;
}

```

Puoi avere più parametri separati da virgola:

```

// Riceve base e altezza, calcola area e perimetro
void stampaRettangolo(int base, int altezza) {
    cout << "Area: " << base * altezza << endl;
    cout << "Perimetro: " << 2 * (base + altezza) << endl;
}

int main() {
    stampaRettangolo(5, 3);
    stampaRettangolo(10, 7);
    return 0;
}

```


Ricevere un risultato: il valore di ritorno

Una funzione può anche **restituire** un valore al codice che la ha chiamata. Si usa `return` :

```
// Riceve due numeri e restituisce la loro somma
int somma(int a, int b) {
    return a + b;
}

int main() {
    int risultato = somma(3, 5);
    cout << risultato << endl;    // 8

    // Puoi usare la funzione direttamente nelle espressioni
    cout << somma(10, 20) * 2 << endl;    // 60

    return 0;
}
```

Quando il compilatore incontra `return` , la funzione si ferma subito e restituisce il valore.

Il prototipo: dichiarare prima di usare

In C++, una funzione deve essere **dichiarata prima del punto in cui la chiami**. Se vuoi definire le funzioni dopo il `main` , devi prima scrivere il loro **prototipo** (una dichiarazione anticipata):

```
// Prototipo – dice al compilatore che questa funzione esisterà
int somma(int a, int b);

int main() {
    cout << somma(3, 5) << endl;    // funziona!
    return 0;
}

// Definizione completa – scritta dopo il main
int somma(int a, int b) {
    return a + b;
}
```

Il prototipo ha la stessa firma della funzione ma termina con `;` invece del corpo.

Valori di default

Puoi dare un valore predefinito a un parametro. Se chi chiama la funzione non lo specifica, viene usato il valore di default:

```
void saluta(string nome, string saluto = "Ciao") {  
    cout << saluto << ", " << nome << "!" << endl;  
}  
  
int main() {  
    saluta("Alice");           // usa il default – stampa: Ciao, Alice!  
    saluta("Bob", "Buongiorno"); // usa il valore specificato – stampa:  
    Buongiorno, Bob!  
    return 0;  
}
```

I parametri con valori di default devono stare sempre in fondo alla lista.

Più istruzioni **return**

Una funzione può avere più punti di uscita. Appena il programma incontra **return**, la funzione si ferma:

```
string classificaVoto(int voto) {  
    if (voto >= 9) return "Ottimo";  
    if (voto >= 7) return "Buono";  
    if (voto >= 6) return "Sufficiente";  
    return "Insufficiente"; // raggiunto solo se nessun return precedente  
    è scattato  
}
```

Passaggio per valore: la funzione riceve una copia

Di default, una funzione riceve una **copia** del valore. Le modifiche che fa non cambiano la variabile originale:

```
void raddoppia(int x) {  
    x = x * 2;    // modifica solo la copia - l'originale non cambia  
    cout << "Dentro la funzione: " << x << endl;  
}  
  
int main() {  
    int numero = 5;  
    raddoppia(numero);  
    cout << "Fuori dalla funzione: " << numero << endl;    // ancora 5!  
    return 0;  
}
```

Output:

```
Dentro la funzione: 10  
Fuori dalla funzione: 5
```

Per modificare la variabile originale, si usano i riferimenti (trattati nella sezione memoria).

Esempio pratico

```
#include <iostream>
using namespace std;

// Calcola l'area di un cerchio dato il raggio
double calcolaAreaCerchio(double raggio) {
    const double PI = 3.14159265358979;
    return PI * raggio * raggio;
}

// Restituisce true se il numero è pari, false altrimenti
bool isPari(int numero) {
    return numero % 2 == 0;
}

// Restituisce il maggiore dei due numeri
int massimo(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    double raggio = 5.0;
    cout << "Area cerchio (r=" << raggio << "): " <<
    calcolaAreaCerchio(raggio) << endl;

    int n = 7;
    cout << n << " è " << (isPari(n) ? "pari" : "dispari") << endl;

    cout << "Massimo tra 15 e 23: " << massimo(15, 23) << endl;

    return 0;
}
```

Overloading di Funzioni

Come definire più funzioni con lo stesso nome ma parametri diversi in C++.

Più versioni della stessa funzione

Immagina di voler calcolare l'area di forme diverse: un cerchio, un rettangolo, un triangolo. Potresti chiamare le funzioni `areaDelCerchio`, `areaDelRettangolo`, `areaDelTriangolo`. Ma sarebbe più comodo chiamarle tutte `area` e lasciare al compilatore il compito di capire quale usare.

In C++ questo si chiama **overloading** (sovraccarico): puoi avere più funzioni con lo stesso nome, purché abbiano parametri diversi.

Come funziona

Il compilatore distingue le funzioni guardando il numero e il tipo dei parametri. Quando chiami la funzione, sceglie automaticamente quella giusta:

```
void stampa(int x) {
    cout << "Intero: " << x << endl;
}

void stampa(double x) {
    cout << "Decimale: " << x << endl;
}

void stampa(string s) {
    cout << "Stringa: " << s << endl;
}

int main() {
    stampa(42);           // il compilatore sceglie stampa(int)
    stampa(3.14);         // il compilatore sceglie stampa(double)
    stampa("Ciao");       // il compilatore sceglie stampa(string)
    return 0;
}
```

Output:

```
Intero: 42
Decimale: 3.14
Stringa: Ciao
```

Overloading con numero diverso di parametri

Puoi anche avere versioni con un numero diverso di parametri:

```
int somma(int a, int b) {  
    return a + b;  
}  
  
int somma(int a, int b, int c) {  
    return a + b + c;  
}  
  
int somma(int a, int b, int c, int d) {  
    return a + b + c + d;  
}  
  
int main() {  
    cout << somma(1, 2) << endl;           // 3  
    cout << somma(1, 2, 3) << endl;        // 6  
    cout << somma(1, 2, 3, 4) << endl;    // 10  
    return 0;  
}
```

Esempio pratico: area di figure diverse

Un uso classico è calcolare l'area di figure geometriche diverse con lo stesso nome `area` :

```

#include <iostream>
using namespace std;

const double PI = 3.14159265358979;

// Area del cerchio – riceve solo il raggio
double area(double raggio) {
    return PI * raggio * raggio;
}

// Area del rettangolo – riceve base e altezza
double area(double base, double altezza) {
    return base * altezza;
}

// Area del triangolo – riceve base, altezza e un flag per distinguerla dal
// rettangolo
double area(double base, double altezza, bool triangolo) {
    return (base * altezza) / 2;
}

int main() {
    cout << "Area cerchio (r=5): " << area(5.0) << endl;
    cout << "Area rettangolo (4x6): " << area(4.0, 6.0) << endl;
    cout << "Area triangolo (3x8): " << area(3.0, 8.0, true) << endl;
    return 0;
}

```

Cosa non funziona: il tipo di ritorno non basta

Il compilatore distingue le funzioni solo dai parametri, non dal tipo di ritorno. Questo causa un errore:

```

int calcola(int x) { return x * 2; }
double calcola(int x) { return x * 2.0; } // ERRORE: i parametri sono
identici!

```

Il compilatore non sa quale delle due chiamare, e lo segnala come errore.

Overloading o parametri di default?

A volte si può ottenere lo stesso risultato con entrambi gli approcci:

```
// Con overloading – due funzioni separate
void saluta() { cout << "Ciao, ospite!" << endl; }
void saluta(string nome) { cout << "Ciao, " << nome << "!" << endl; }

// Con parametro di default – una sola funzione
void saluta(string nome = "ospite") {
    cout << "Ciao, " << nome << "!" << endl;
}
```

La regola generale:

- Usa i **parametri di default** quando la funzione fa la stessa cosa con valori diversi
- Usa l'**overloading** quando la funzione fa cose diverse per tipi diversi di input

Esempio pratico: confronto flessibile

```
#include <iostream>
#include <string>
using namespace std;

// Confronta due interi
bool confronta(int a, int b) {
    return a == b;
}

// Confronta due decimali con una piccola tolleranza
// (i numeri decimali raramente sono esattamente uguali)
bool confronta(double a, double b) {
    return (a - b < 0.0001) && (b - a < 0.0001);
}

// Confronta due stringhe
bool confronta(string a, string b) {
    return a == b;
}

int main() {
    cout << boolalpha;
    cout << confronta(5, 5) << endl;           // true
    cout << confronta(3.14, 3.14) << endl;      // true
    cout << confronta("ciao", "ciao") << endl;  // true
    cout << confronta("ciao", "Ciao") << endl;  // false (maiuscole
    contano)
    return 0;
}
```

Scope delle Variabili

La visibilità e la durata delle variabili in C++.

Cos'è lo scope?

Immagina un edificio con stanze. Quello che metti in una stanza non è visibile dalle altre stanze. Se esci dalla stanza, le cose che hai lasciato lì scompaiono.

In C++, le variabili funzionano allo stesso modo: esistono solo nel "posto" in cui vengono dichiarate, e spariscono quando quel posto termina.

Questo "posto" si chiama **scope** (visibilità).

Le parentesi graffe creano lo scope

In C++, ogni coppia di `{ }` crea una nuova zona di codice con il proprio scope. Le variabili dichiarate dentro quella zona esistono solo lì:

```
int main() {  
    int x = 10;  
  
    {  
        int y = 20;    // y esiste solo in questo blocco interno  
        cout << x;     // OK - x è visibile anche qui  
        cout << y;     // OK - y è visibile qui  
    }  
  
    cout << x;         // OK - x esiste ancora  
    // cout << y; // ERRORE - y non esiste più, il blocco è finito  
}
```

Variabili locali: vivono dentro la funzione

Una **variabile locale** è dichiarata dentro una funzione. Esiste solo finché quella funzione è in esecuzione, e non è visibile dall'esterno:

```

void miaFunzione() {
    int x = 10;    // x esiste solo dentro miaFunzione
    cout << x;    // OK
}

int main() {
    miaFunzione();
    // cout << x;  // ERRORE - x non esiste qui
    return 0;
}

```

Ogni chiamata alla funzione crea una nuova copia delle variabili locali, che poi sparisce quando la funzione termina.

Variabili globali: visibili ovunque

Una **variabile globale** è dichiarata fuori da qualsiasi funzione. È visibile da tutto il codice nel file:

```

int contatore = 0;    // variabile globale - visibile ovunque

void incrementa() {
    contatore++;      // può accedere e modificare la variabile globale
}

int main() {
    cout << contatore << endl;  // 0
    incrementa();
    incrementa();
    cout << contatore << endl;  // 2
    return 0;
}

```

Usa le variabili globali con cautela. Il fatto che qualsiasi funzione possa modificarle le rende difficili da controllare nei programmi grandi.

Le variabili dei cicli

Le variabili dichiarate dentro un `for` esistono solo per la durata del ciclo:

```
for (int i = 0; i < 5; i++) {  
    cout << i << " ";    // OK  
}  
// cout << i;    // ERRORE – i non esiste più fuori dal ciclo
```

Questo è un vantaggio: il contatore `i` non inquina il resto del codice.

Shadowing: quando due variabili hanno lo stesso nome

Se dichiari una variabile locale con lo stesso nome di una globale, la locale "nasconde" la globale dentro il suo scope. Si chiama **shadowing**:

```
int x = 100;    // variabile globale  
  
void funzione() {  
    int x = 50;    // variabile locale – nasconde quella globale  
    cout << x;    // stampa 50 (usa la locale)  
}  
  
int main() {  
    cout << x;    // stampa 100 (usa la globale)  
    funzione();  
    cout << x;    // stampa 100 (la globale non è cambiata)  
    return 0;  
}
```

Evita lo shadowing: usa nomi diversi per variabili diverse. Il codice è molto più chiaro.

La variabile `static` : sopravvive tra le chiamate

Normalmente, una variabile locale ricomincia da zero ogni volta che la funzione viene chiamata. Con la parola chiave `static`, il valore viene conservato tra una chiamata e l'altra:

```

void contaChiamate() {
    int locale = 0;           // ricomincia da 0 ad ogni chiamata
    static int statica = 0;   // conserva il valore tra le chiamate

    locale++;
    statica++;

    cout << "Locale: " << locale << ", Statica: " << statica << endl;
}

int main() {
    contaChiamate(); // Locale: 1, Statica: 1
    contaChiamate(); // Locale: 1, Statica: 2
    contaChiamate(); // Locale: 1, Statica: 3
    return 0;
}

```

I parametri delle funzioni

I parametri di una funzione si comportano come variabili locali: esistono solo dentro la funzione.

```

void funzione(int a, int b) {
    // a e b sono locali a questa funzione
    int somma = a + b;
    cout << somma;
}
// a, b e somma non esistono qui

```

Consigli pratici

- **Dichiara le variabili vicino a dove le usi:** non in cima alla funzione se le usi solo alla fine.
- **Evita le variabili globali** quando possibile. Preferisci passare i valori come parametri.
- **Evita lo shadowing:** dai nomi diversi alle variabili in scope diversi.

```
// Meno chiaro: variabile dichiarata lontano dall'uso
int risultato;
// ... molte righe di codice ...
risultato = a + b;

// Più chiaro: dichiarazione e uso insieme
int risultato = a + b;
```

Input/Output Formattato

Come formattare l'output in C++ con `setw`, `setprecision` e altre opzioni.

Controllare l'aspetto dell'output

Stampare "3.14159265358979" va bene in alcuni casi, ma se stai mostrando un prezzo vuoi "3.14". Se stai costruendo una tabella vuoi che le colonne siano allineate.

Il C++ offre strumenti per controllare con precisione come appaiono i dati sullo schermo. Per usarli, includi `<iomanip>` :

```
#include <iostream>
#include <iomanip>
using namespace std;
```

Quante cifre decimali: **setprecision**

Di default, `cout` mostra 6 cifre significative. Con `setprecision` puoi cambiarle:

```
double pi = 3.14159265358979;

cout << pi << endl;           // 3.14159 (default: 6 cifre)
cout << setprecision(3) << pi << endl; // 3.14
cout << setprecision(10) << pi << endl; // 3.141592654
```

Per controllare le cifre **dopo la virgola** (non le cifre significative), aggiungi **fixed** :

```
double x = 1234.5678;

cout << fixed << setprecision(2) << x << endl; // 1234.57
cout << fixed << setprecision(4) << x << endl; // 1234.5678
cout << fixed << setprecision(0) << x << endl; // 1235 (arrotondato)
```

Per la notazione scientifica (es. per numeri molto grandi o piccoli):

```
double grande = 1234567.89;
cout << scientific << setprecision(3) << grande << endl; // 1.235e+06
```

Larghezza minima: **setw**

setw(n) riserva almeno **n** spazi per il prossimo valore stampato. Se il valore è più corto, vengono aggiunti spazi davanti (a destra di default):

```
cout << setw(10) << "Nome" << setw(10) << "Età" << endl;
cout << setw(10) << "Alice" << setw(10) << 16 << endl;
cout << setw(10) << "Bob" << setw(10) << 17 << endl;
```

Output:

Nome	Età
Alice	16
Bob	17

Attenzione: `setw` è **temporaneo**. Vale solo per il prossimo valore stampato, poi va reimpostato.

Allineamento: `left` e `right`

Di default i valori vengono allineati a destra. Puoi cambiarlo con `left` :

```
cout << left << setw(10) << "Alice" << endl; // Alice
cout << right << setw(10) << "Alice" << endl; //      Alice
```

Output:

```
Alice
      Alice
```

`left` e `right` sono **persistenti**: rimangono attivi finché non li cambi.

Carattere di riempimento: `setfill`

Di default `setw` riempie con spazi. Puoi usare un altro carattere:

```
cout << setfill('*') << setw(10) << "ciao" << endl; // *****ciao
cout << setfill('0') << setw(5) << 42 << endl;      // 00042
```

Utile per creare separatori o codici con zeri iniziali.

Stampare bool come parole

Di default `true` e `false` vengono stampati come `1` e `0` . Con `boolalpha` vengono stampate le parole:


```

bool b = true;
cout << b << endl;           // 1
cout << boolalpha << b << endl; // true

bool f = false;
cout << boolalpha << f << endl; // false

```

Riepilogo: persistente vs temporaneo

Modificatore	Dura fino a...
<code>fixed</code> , <code>scientific</code>	Finché non lo cambi
<code>left</code> , <code>right</code>	Finché non lo cambi
<code>boolalpha</code>	Finché non lo cambi
<code>setprecision(n)</code>	Finché non lo cambi
<code>setfill(c)</code>	Finché non lo cambi
<code>setw(n)</code>	Solo il prossimo valore

Esempio pratico: tabella formattata

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

int main() {
    // Intestazione della tabella
    cout << left
         << setw(15) << "Studente"
         << setw(8)  << "Età"
         << setw(10) << "Voto"
         << endl;

    // Riga separatrice
    cout << setfill('-') << setw(33) << "" << endl;
    cout << setfill(' '); // ripristina il riempimento con spazio

    // Dati degli studenti
    struct Studente { string nome; int eta; double voto; };
    Studente studenti[] = {
        {"Alice", 16, 8.5},
        {"Bob", 17, 7.2},
        {"Carlo Verdi", 15, 9.1}
    };

    for (const auto& s : studenti) {
        cout << left
             << setw(15) << s.nome
             << setw(8)  << s.eta
             << fixed << setprecision(1) << setw(10) << s.voto
             << endl;
    }

    return 0;
}
```

Output:

Studente	Età	Voto
Alice	16	8.5
Bob	17	7.2
Carlo Verdi	15	9.1

Input con cin

Come leggere dati inseriti dall'utente con cin in C++.

Come si legge l'input dell'utente?

Un programma che fa sempre la stessa cosa con gli stessi dati non è molto utile. `cin` ti permette di leggere quello che l'utente scrive dalla tastiera, rendendo il programma interattivo.

Cos'è `cin` ?

`cin` è lo strumento del C++ per leggere l'input dall'utente tramite tastiera. Come `cout`, fa parte della libreria `<iostream>`.

L'operatore `>>`

L'operatore `>>` "estrae" un valore dall'input e lo salva in una variabile:

```
int x;  
cin >> x;
```

Quando il programma arriva a questa riga, si ferma e aspetta che l'utente scriva qualcosa e prema Invio. Il valore inserito viene salvato nella variabile `x`.

Leggere un numero intero

```
#include <iostream>
using namespace std;

int main() {
    int eta;
    cout << "Inserisci la tua età: "; // spiega cosa vuoi
    cin >> eta;                       // leggi il valore
    cout << "Hai " << eta << " anni." << endl;
    return 0;
}
```

Esempio di esecuzione:

```
Inserisci la tua età: 16
Hai 16 anni.
```

Leggere un numero decimale

```
double altezza;
cout << "Inserisci la tua altezza in metri: ";
cin >> altezza;
cout << "Alta/o " << altezza << " m." << endl;
```

Leggere una parola

Con `cin >>` puoi leggere una parola sola. Il problema: si ferma al primo spazio.

```
string nome;
cout << "Inserisci il tuo nome: ";
cin >> nome;
cout << "Ciao, " << nome << "!" << endl;
```

```
Inserisci il tuo nome: Alice
Ciao, Alice!
```

Se l'utente scrive "Alice Bianchi", `cin >> nome` legge solo "Alice" e scarta "Bianchi".

Leggere una riga intera (con spazi)

Per leggere tutta la riga compreso gli spazi, usa `getline()` :

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nome_completo;
    cout << "Inserisci il tuo nome completo: ";
    getline(cin, nome_completo); // legge tutta la riga
    cout << "Ciao, " << nome_completo << "!" << endl;
    return 0;
}
```

```
Inserisci il tuo nome completo: Alice Bianchi
Ciao, Alice Bianchi!
```

Leggere più valori

Puoi leggere più valori in una volta sola, separati da spazio:

```
int a, b;
cout << "Inserisci due numeri separati da spazio: ";
cin >> a >> b;
cout << "Somma: " << a + b << endl;
```

Inserisci due numeri separati da spazio: 5 3
Somma: 8

Attenzione: `cin` e `getline` insieme

Se usi `cin >>` e poi `getline()`, c'è una trappola: quando l'utente preme Invio dopo `cin >>`, il carattere "a capo" rimane nel buffer e viene letto subito da `getline()`, saltando l'input dell'utente.

La soluzione è aggiungere `cin.ignore()` per scartare quel carattere "a capo":

```
int eta;
string nome;

cout << "Età: ";
cin >> eta;

cin.ignore(); // scarta il 'Invio' rimasto nel buffer

cout << "Nome completo: ";
getline(cin, nome); // ora funziona correttamente

cout << nome << " ha " << eta << " anni." << endl;
```

Esempio pratico

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nome;
    int eta;
    double altezza;

    cout << "Come ti chiami? ";
    cin >> nome;

    cout << "Quanti anni hai? ";
    cin >> eta;

    cout << "Quanto sei alto/a (in metri)? ";
    cin >> altezza;

    cout << endl;
    cout << "=== I tuoi dati ===" << endl;
    cout << "Nome:    " << nome << endl;
    cout << "Età:      " << eta << " anni" << endl;
    cout << "Altezza: " << altezza << " m" << endl;

    return 0;
}
```

Esempio di esecuzione:

```
Come ti chiami? Alice
Quanti anni hai? 16
Quanto sei alto/a (in metri)? 1.65

=== I tuoi dati ===
Nome:    Alice
Età:     16 anni
Altezza: 1.65 m
```

Cosa succede se l'utente sbaglia tipo?

Se l'utente scrive del testo dove il programma si aspetta un numero, `cin` entra in uno stato di errore e le letture successive non funzionano. Per ora, puoi supporre che l'utente inserisca sempre il tipo corretto. La gestione degli errori di input è un argomento più avanzato.

Output con cout

Come visualizzare testo e valori sullo schermo con cout in C++.

Come si stampa a schermo in C++?

Un programma che non mostra niente all'utente è inutile. `cout` è il modo principale per comunicare con chi usa il tuo programma: mostrare risultati, messaggi, errori, menu.

È una delle prime cose che impari in C++ e la userai in ogni programma che scrivi.

Cos'è `cout` ?

`cout` è lo strumento del C++ per stampare testo e valori sullo schermo. Fa parte della libreria `<iostream>` che devi includere all'inizio di ogni programma:

```
#include <iostream>
using namespace std;
```

L'operatore `<<`

L'operatore `<<` manda qualcosa a `cout`. Pensa a esso come a una freccia: "manda questo testo allo schermo".

```
cout << "Ciao, Mondo!";
```


Stampa di testo

Metti il testo tra virgolette doppie:

```
cout << "Ciao!";  
cout << "Questa è una frase.";
```

Andare a capo

Per andare a capo, hai due opzioni:

```
cout << "Prima riga" << endl;    // usando endl  
cout << "Seconda riga\n";        // usando \n nella stringa
```

Entrambe vanno a capo. La differenza è che `endl` è leggermente più lento (svuota il buffer). Per la maggior parte dei programmi, sono equivalenti.

Stampare più cose nella stessa riga

Puoi concatenare più `<<` per stampare più valori insieme:

```
cout << "Nome: " << "Alice" << endl;  
cout << "Età: " << 16 << endl;  
cout << "Altezza: " << 1.65 << " m" << endl;
```

Output:

```
Nome: Alice  
Età: 16  
Altezza: 1.65 m
```

Stampare variabili

`cout` funziona con qualsiasi tipo di dato:

```
int x = 42;
double pi = 3.14;
char lettera = 'A';
bool vero = true;

cout << x << endl;           // 42
cout << pi << endl;          // 3.14
cout << lettera << endl;     // A
cout << vero << endl;        // 1 (true viene stampato come 1 di default)
```

Caratteri speciali nelle stringhe

Puoi inserire caratteri speciali nelle stringhe usando il simbolo `\` :

Sequenza	Significato
<code>\n</code>	Nuova riga
<code>\t</code>	Tabulazione (tab) — crea uno spazio largo
<code>\\</code>	Il carattere backslash <code>\</code>
<code>\"</code>	Il carattere virgolette doppie <code>"</code>
<code>\'</code>	Il carattere virgolette singole <code>'</code>

```
cout << "Riga 1\nRiga 2\nRiga 3" << endl;
cout << "Nome:\tAlice" << endl;
cout << "Dice: \"Ciao!\"" << endl;
```

Output:

```
Riga 1
Riga 2
Riga 3
Nome:  Alice
Dice: "Ciao!"
```

Calcoli direttamente nell'output

Puoi fare calcoli direttamente dentro il `cout` :

```
int a = 5, b = 3;
cout << "Somma: " << a + b << endl;    // 8
cout << "Prodotto: " << a * b << endl;  // 15
```

Esempio pratico

```
#include <iostream>
using namespace std;

int main() {
    string nome = "Alice";
    int eta = 16;
    double altezza = 1.65;

    cout << "=== Scheda Studente ===" << endl;
    cout << "Nome:    " << nome << endl;
    cout << "Età:      " << eta << " anni" << endl;
    cout << "Altezza: " << altezza << " m" << endl;

    return 0;
}
```

Output:

```
=== Scheda Studente ===  
Nome:    Alice  
Età:     16 anni  
Altezza: 1.65 m
```

Stringhe in C++

Come usare le stringhe in C++ con il tipo `string`.

Lavorare con il testo in C++

I programmi lavorano con parole, nomi, frasi, URL, messaggi... non solo numeri. Le **stringhe** sono il modo in cui il programma lavora con il testo.

In C++ moderno si usa il tipo `string`, che rende il lavoro con il testo semplice e sicuro.

Includere la libreria

```
#include <iostream>  
#include <string>  
using namespace std;
```

Creare una stringa

```
string nome = "Alice";  
string saluto = "Ciao, mondo!";  
string vuota = ""; // stringa senza nessun carattere
```

Le stringhe usano le **virgolette doppie** `"`. Le virgolette singole `'` sono per i caratteri singoli (`char`).

Unire stringhe: concatenazione

Puoi unire due stringhe con `+` :

```
string nome = "Alice";
string cognome = "Bianchi";
string nomeCompleto = nome + " " + cognome;
cout << nomeCompleto << endl; // Alice Bianchi
```

Per aggiungere qualcosa in coda a una stringa:

```
string frase = "Ciao";
frase += ", mondo";
frase += "!";
cout << frase << endl; // Ciao, mondo!
```

Lunghezza

Il metodo `.length()` restituisce quanti caratteri ha la stringa:

```
string nome = "Alice";
cout << nome.length() << endl; // 5
```

Accedere a un singolo carattere

Come gli array, puoi leggere o modificare un singolo carattere usando l'indice tra `[]` . Gli indici partono da 0:

```

string nome = "Alice";
cout << nome[0] << endl; // A – primo carattere
cout << nome[1] << endl; // l
cout << nome[4] << endl; // e – quinto (e ultimo) carattere

// Modifica del primo carattere
nome[0] = 'E';
cout << nome << endl; // Elice

```

Confrontare stringhe

Puoi confrontare stringhe con gli operatori normali:

```

string a = "ciao";
string b = "ciao";
string c = "hello";

cout << (a == b) << endl; // 1 – sono uguali
cout << (a == c) << endl; // 0 – sono diverse
cout << (a != c) << endl; // 1 – sono diverse

```

Attenzione: il confronto distingue maiuscole e minuscole. `"Ciao"` e `"ciao"` sono considerate diverse.

Cercare dentro una stringa: `find()`

Il metodo `find()` cerca una parola o un carattere e restituisce la posizione in cui si trova:

```

string frase = "Ciao, mondo!";
int pos = frase.find("mondo");
cout << pos << endl; // 6 – "mondo" inizia alla posizione 6

// Se non trova niente, restituisce string::npos
if (frase.find("xyz") == string::npos) {
    cout << "Non trovato" << endl;
}

```

Estrarre una parte: `substr()`

Il metodo `substr()` estrae una porzione di stringa:

```
string frase = "Ciao, mondo!";  
// substr(posizione_di_inizio, lunghezza)  
string parte = frase.substr(6, 5);  
cout << parte << endl; // mondo
```

Convertire numeri in stringhe

Per trasformare un numero in stringa (utile per concatenarlo):

```
int eta = 16;  
string etaStr = to_string(eta);  
cout << "Hai " + etaStr + " anni." << endl;
```

Convertire stringhe in numeri

Per trasformare una stringa che contiene un numero in un valore numerico:

```
string numStr = "42";  
int num = stoi(numStr);    // da stringa a int  
cout << num + 1 << endl;   // 43  
  
string decimaleStr = "3.14";  
double dec = stod(decimaleStr); // da stringa a double
```

Leggere stringhe con spazi

`cin >>` si ferma al primo spazio. Per leggere tutta la riga usa `getline()` :

```
string nome;
cout << "Inserisci il tuo nome completo: ";
getline(cin, nome);
cout << "Ciao, " << nome << "!" << endl;
```

Altri metodi utili

```
// Controlla se la stringa è vuota
string vuota = "";
cout << vuota.empty() << endl;    // 1 (true)

// Trova la posizione di un carattere
string nome = "Alice";
cout << nome.find('l') << endl;    // 1

// Sostituisce una porzione
string s = "Ciao, mondo!";
s.replace(6, 5, "Alice");           // sostituisce 5 caratteri dalla
posizione 6
cout << s << endl;                 // Ciao, Alice!

// Inserisce del testo in una posizione
string str = "Ciao!";
str.insert(4, ", mondo");
cout << str << endl;              // Ciao, mondo!
```


Esempio pratico

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nome, cognome;

    cout << "Inserisci il nome: ";
    cin >> nome;
    cout << "Inserisci il cognome: ";
    cin >> cognome;

    string nomeCompleto = nome + " " + cognome;

    cout << endl;
    cout << "Nome completo: " << nomeCompleto << endl;
    cout << "Lunghezza: " << nomeCompleto.length() << " caratteri" << endl;
    cout << "Iniziale nome: " << nome[0] << endl;
    cout << "Iniziale cognome: " << cognome[0] << endl;

    // Conta le vocali nel nome completo
    string vocali = "aeiouAEIOU";
    int contaVocali = 0;
    for (char c : nomeCompleto) {
        if (vocali.find(c) != string::npos) {
            contaVocali++;
        }
    }
    cout << "Numero di vocali: " << contaVocali << endl;

    return 0;
}
```

Gestione della Memoria

Introduzione all'allocazione dinamica della memoria con new e delete in C++.

Come funziona la memoria in C++?

Immagina di organizzare una festa. Non sai quante persone verranno. Se prenoti 10 sedie prima, e vengono in 50, hai un problema. Se ne prenoti 200, e vengono in 10, hai sprecato spazio.

La **memoria dinamica** risolve questo problema: puoi "prenotare" esattamente la quantità di memoria che ti serve, nel momento in cui sai quanta ne hai bisogno.

Due tipi di memoria

In C++, la memoria funziona in due modi diversi:

Memoria automatica (stack): le variabili normali. Il computer le crea quando entri in una funzione e le distrugge da solo quando esci. La quantità deve essere nota in anticipo.

```
int x = 5;           // creato automaticamente, distrutto automaticamente
int arr[10];         // array di dimensione fissa – deve essere nota prima
```

Memoria dinamica (heap): tu decidi quanta memoria serve, nel momento in cui ti serve. Devi gestirla tu — compresa la "restituzione" quando hai finito.

```
int* ptr = new int;   // crei uno spazio nella memoria heap
int* arr = new int[100]; // un array di 100 interi – deciso a runtime
```

Pensa all'heap come a un magazzino enorme. Puoi affittare uno spazio, usarlo, e poi restituirlo. Se non lo restituisci, rimane occupato inutilmente.

Allocare memoria: **new**

L'operatore **new** affitta uno spazio nell'heap e ti restituisce l'indirizzo (sotto forma di puntatore):

```
// Alloca spazio per un singolo intero
int* ptr = new int;
*ptr = 42;
cout << *ptr << endl;    // 42

// Alloca e inizializza subito
int* ptr2 = new int(100);
cout << *ptr2 << endl;    // 100

// Alloca un array di 5 interi
int* arr = new int[5];
arr[0] = 10;
arr[1] = 20;
// ...
```

Liberare memoria: delete

Ogni volta che usi `new`, devi usare `delete` quando hai finito. Altrimenti la memoria rimane "affittata" per sempre — si chiama **memory leak** (perdita di memoria):

```
int* ptr = new int(42);
cout << *ptr << endl;    // 42

delete ptr;                // "restituisce" la memoria affittata
ptr = nullptr;             // buona abitudine: azzera il puntatore dopo delete
```

Per un array allocato con `new[]`, usa `delete[]` (con le parentesi quadre):

```
int* arr = new int[10];
// ... usa arr ...
delete[] arr;              // IMPORTANTE: delete[] e non solo delete
arr = nullptr;
```

Il memory leak: la perdita di memoria

Se dimentichi il `delete`, la memoria resta occupata finché il programma non chiude:

```
// Ogni volta che chiami questa funzione, perdi 4 byte di memoria
void funzione() {
    int* ptr = new int(42);
    // ... usa ptr ...
    // manca delete ptr! - il ptr viene distrutto ma la memoria no
}

// Se la chiami mille volte, hai perso 4000 byte senza saperlo
for (int i = 0; i < 1000; i++) {
    funzione();
}
```

In un programma piccolo non si nota. In un programma che gira per ore o giorni, i leak si accumulano fino a esaurire la RAM.

Il dangling pointer: il puntatore "fantasma"

Dopo aver chiamato `delete`, il puntatore esiste ancora ma punta a memoria che non è più tua. Usarlo è un errore grave:

```
int* ptr = new int(42);
delete ptr;
// ptr è ora un "puntatore fantasma": punta a memoria liberata

cout << *ptr;    // ERRORE: comportamento non definito (possibile crash)

// Soluzione: dopo delete, metti sempre nullptr
delete ptr;
ptr = nullptr;

if (ptr != nullptr) {
    cout << *ptr;    // non verrà mai eseguito, ma è sicuro
}
```

Esempio pratico: array di dimensione variabile

Il caso più comune in cui serve la memoria dinamica è quando non sai in anticipo quanti elementi hai bisogno:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Quanti numeri vuoi inserire? ";
    cin >> n;

    // Alloca esattamente n interi – deciso a runtime!
    int* numeri = new int[n];

    // Inserisci i numeri
    for (int i = 0; i < n; i++) {
        cout << "Numero " << (i + 1) << ": ";
        cin >> numeri[i];
    }

    // Calcola la media
    int somma = 0;
    for (int i = 0; i < n; i++) {
        somma += numeri[i];
    }
    cout << "Media: " << (double)somma / n << endl;

    // Libera la memoria!
    delete[] numeri;
    numeri = nullptr;

    return 0;
}
```

La soluzione moderna: puntatori smart

Gestire `new` e `delete` manualmente è complicato e soggetto a errori. In C++ moderno si usano i **puntatori smart**, che si occupano di liberare la memoria automaticamente quando non serve più:

```
#include <memory>
using namespace std;

// unique_ptr: possiede la memoria e la libera automaticamente
unique_ptr<int> ptr = make_unique<int>(42);
cout << *ptr << endl;    // 42
// Non serve delete: viene chiamato automaticamente quando ptr esce dal
// blocco

// shared_ptr: più puntatori condividono la stessa memoria
shared_ptr<int> sptr = make_shared<int>(100);
```

I puntatori smart eliminano quasi tutti i problemi di memory leak e dangling pointer.

La soluzione ancora più semplice: **vector**

Per gli array dinamici, la scelta migliore è usare **vector** dalla libreria standard. Gestisce la memoria per te:

```
#include <vector>
using namespace std;

// Invece di:
int* arr = new int[n];
// ... usa arr ...
delete[] arr;

// Usa semplicemente:
vector<int> arr(n);
// ... usa arr ...
// nessun delete! viene liberato automaticamente
```

Regole pratiche

- Se usi **new**, usa sempre **delete** nella stessa funzione (o usa uno smart pointer)
- Se usi **new[]**, usa sempre **delete[]** (non **delete**)
- Dopo ogni **delete**, imposta il puntatore a **nullptr**
- Quando puoi, usa **vector** invece di array dinamici manuali

- Per oggetti singoli, usa `make_unique` invece di `new`

Puntatori

Il concetto di puntatori in C++, come funzionano e come usarli.

Cos'è un puntatore?

Immagina di avere un quaderno con le istruzioni per trovare un tesoro. Il quaderno non contiene il tesoro — contiene l'**indirizzo** dove trovarlo.

Un **puntatore** funziona allo stesso modo: non contiene un valore direttamente, ma contiene la **posizione in memoria** dove quel valore si trova.

Questo ti permette di fare cose che altrimenti non potresti fare, come modificare una variabile dall'interno di una funzione.

La memoria del computer come una via di appartamenti

Pensa alla RAM come a una lunghissima via con migliaia di appartamenti. Ogni appartamento ha un **numero civico** (l'indirizzo) e può contenere un valore.

Quando dichiari `int x = 42;`, il computer affitta un appartamento per `x` e ci mette dentro il numero 42.

Un puntatore è come un bigliettino su cui scrivi il numero civico di quell'appartamento. Con il bigliettino, puoi andare a vedere cosa c'è dentro — o addirittura cambiarlo.

Come vedere l'indirizzo di una variabile

Usa l'operatore `&` davanti a una variabile per leggere il suo indirizzo:

```
int x = 42;
cout << x << endl;    // stampa 42 (il valore)
cout << &x << endl;    // stampa qualcosa come 0x7fff5c1a2b40 (l'indirizzo)
```

Quell'indirizzo strano con lettere e numeri è scritto in **esadecimale** — è il numero del "cassetto" nella RAM.

Dichiarare un puntatore

Si usa il simbolo `*` dopo il tipo:

```
int* ptr;          // puntatore a int (un bigliettino con l'indirizzo di un
intero)
double* dptr;      // puntatore a double
char* cptr;        // puntatore a char
```

Assegnare un indirizzo al puntatore

Per far "puntare" il puntatore a una variabile, assegnagli il suo indirizzo con `&` :

```
int x = 42;
int* ptr = &x;    // ptr contiene l'indirizzo di x (il "numero civico" di
x)

cout << ptr << endl;    // stampa l'indirizzo di x
cout << *ptr << endl;    // stampa 42 (il valore dentro quell'indirizzo)
```

Leggere e modificare il valore: l'operatore `*`

Mettere `*` davanti a un puntatore significa "vai all'indirizzo e leggi (o scrivi) il valore lì dentro". Si chiama **dereferenziazione**.

```
int x = 42;
int* ptr = &x;

cout << *ptr << endl;    // legge il valore di x → stampa 42

*ptr = 100;              // scrive un nuovo valore nella posizione di x
cout << x << endl;      // x è cambiata! stampa 100
```

È come andare all'appartamento indicato sul bigliettino e sostituire il contenuto.

Riepilogo dei simboli

```
int x = 42;
int* ptr = &x;

// & davanti a una variabile = "dammi l'indirizzo di questa variabile"
cout << &x << endl;    // indirizzo di x

// * nella dichiarazione = "questa variabile è un puntatore"
int* ptr = &x;          // ptr è un puntatore a int

// * davanti a un puntatore = "vai all'indirizzo e leggi/scrivi il valore"
cout << *ptr << endl;  // il valore a quell'indirizzo (42)
```

Il puntatore nullo

Se un puntatore non deve puntare a niente, assegnagli `nullptr` :

```
int* ptr = nullptr;    // non punta a nessun indirizzo

// Prima di usare un puntatore, controlla che non sia nullo!
if (ptr != nullptr) {
    cout << *ptr;    // sicuro solo se ptr non è nullptr
}
```

Regola d'oro: non usare mai un puntatore nullo senza averlo controllato prima — causerebbe il crash del programma.

Passare variabili alle funzioni tramite puntatore

Il caso d'uso più utile dei puntatori è permettere a una funzione di modificare una variabile che si trova fuori da essa:

```
// La funzione riceve il "bigliettino" con l'indirizzo di x
void raddoppia(int* ptr) {
    *ptr = *ptr * 2;    // va all'indirizzo e modifica il valore lì
}

int main() {
    int x = 5;
    cout << x << endl;    // 5

    raddoppia(&x);        // passa l'indirizzo di x (il "bigliettino")

    cout << x << endl;    // 10 - x è stata modificata dalla funzione!
    return 0;
}
```

Senza i puntatori, la funzione riceverebbe solo una **copia** di `x` e le modifiche andrebbero perse.

Puntatori e array

Il nome di un array è già un puntatore al suo primo elemento. Puoi muoverti tra gli elementi sommando numeri all'indirizzo:

```
int numeri[] = {10, 20, 30, 40, 50};
int* ptr = numeri;    // punta al primo elemento (numeri[0])

cout << *ptr << endl;    // 10 (primo elemento)
cout << *(ptr + 1) << endl;    // 20 (secondo elemento)
cout << *(ptr + 2) << endl;    // 30 (terzo elemento)

// Equivalente all'accesso normale con []
cout << ptr[0] << endl;    // 10
cout << ptr[1] << endl;    // 20
```

Esempio pratico: scambiare due variabili

```
#include <iostream>
using namespace std;

// Riceve i bigliettini con gli indirizzi di a e b
void scambia(int* a, int* b) {
    int temp = *a;    // salva il valore di a in temp
    *a = *b;          // copia il valore di b in a
    *b = temp;        // copia temp (vecchio a) in b
}

int main() {
    int x = 10, y = 20;
    cout << "Prima: x=" << x << ", y=" << y << endl;

    scambia(&x, &y);    // passa gli indirizzi di x e y

    cout << "Dopo:  x=" << x << ", y=" << y << endl;
    return 0;
}
```

Output:

```
Prima: x=10, y=20
Dopo:  x=20, y=10
```

Errori comuni da evitare

```
int* ptr;        // NON inizializzato: contiene un indirizzo casuale,
                 // pericoloso!
*ptr = 42;       // ERRORE: modifica una zona di memoria a caso → crash

int* ptr2 = nullptr;
*ptr2 = 42;      // ERRORE: dereferenziare nullptr → crash garantito
```

In C++ moderno, si preferisce usare i **riferimenti** quando possibile (vedi la sezione successiva) e i **puntatori smart** per la memoria dinamica.

Riferimenti

Come usare i riferimenti in C++ per creare alias alle variabili.

Cos'è un riferimento?

Supponi di chiamarti "Marco" ma tutti gli amici ti chiamano "Max". Sei sempre la stessa persona — solo con due nomi diversi.

Un **riferimento** in C++ funziona così: è un secondo nome per la stessa variabile. Se modifichi il valore tramite un nome, l'altro nome vede subito la modifica — perché si riferiscono alla stessa cosa.

I riferimenti sono più semplici e sicuri dei puntatori, e si usano moltissimo in C++.

Come creare un riferimento

Si usa `&` dopo il tipo nella dichiarazione:

```
int x = 42;
int& ref = x;    // ref è un altro nome per x

cout << x << endl;    // 42
cout << ref << endl;  // 42 (stessa variabile, stesso valore)

ref = 100;        // modifica il valore tramite ref
cout << x << endl;    // 100 – anche x è cambiata!
```

`ref` e `x` puntano alla stessa casella in memoria. Cambiare uno cambia l'altro.

Differenze tra riferimenti e puntatori

Caratteristica	Riferimento (&)	Puntatore (*)
Come si usa	<code>ref</code> (direttamente)	<code>*ptr</code> (con la stella)
Può essere nullo?	No	Sì (<code>nullptr</code>)
Può cambiare a cosa si riferisce?	No	Sì
Deve essere inizializzato subito?	Sì	No (ma è pericoloso)

I riferimenti sono più comodi e sicuri: una volta creati, puntano sempre alla stessa variabile.

Il caso d'uso principale: funzioni che modificano variabili

Normalmente, quando passi una variabile a una funzione, la funzione ne riceve una **copia**. Le modifiche alla copia non cambiano l'originale:

```
void raddoppiaValore(int x) {
    x = x * 2;    // modifica solo la copia
}

int main() {
    int numero = 5;
    raddoppiaValore(numero);
    cout << numero << endl;    // 5 - non è cambiato!
}
```

Con un riferimento, la funzione lavora direttamente sull'originale:

```

void raddoppiaRiferimento(int& x) {
    x = x * 2;    // modifica la variabile originale
}

int main() {
    int numero = 5;
    raddoppiaRiferimento(numero);    // nessun &, si passa direttamente
    cout << numero << endl;        // 10 – è cambiato!
}

```

Il `&` nella **dichiarazione del parametro** dice: "non fare una copia, lavora sull'originale".

Scambiare due variabili

Con i riferimenti, il classico scambio di due variabili diventa molto leggibile:

```

void scambia(int& a, int& b) {
    int temp = a;    // salva a
    a = b;           // a prende il valore di b
    b = temp;        // b prende il vecchio valore di a
}

int main() {
    int x = 10, y = 20;
    cout << x << " " << y << endl;    // 10 20

    scambia(x, y);

    cout << x << " " << y << endl;    // 20 10
    return 0;
}

```

Riferimento costante: leggere senza copiare

Se passi una stringa o un oggetto grande a una funzione, il computer ne fa una copia — spreco di tempo e memoria.

Con `const &` passi il riferimento (nessuna copia) ma impedisce le modifiche:

```

// Senza const ref: copia l'intera stringa (lento per stringhe lunghe)
void stampa1(string s) {
    cout << s << endl;
}

// Con const ref: nessuna copia, non si può modificare (veloce e sicuro)
void stampa2(const string& s) {
    cout << s << endl;
    // s = "altro"; // ERRORE: è const, non si può modificare
}

int main() {
    string testo = "Una stringa molto lunga...";
    stampa1(testo); // fa una copia
    stampa2(testo); // non fa nessuna copia – più efficiente!
}

```

La regola pratica: usa `const string&` (e in generale `const Tipo&`) per passare oggetti grandi in sola lettura.

Riferimenti nel ciclo for

I riferimenti sono utili anche nei cicli `for` quando vuoi modificare gli elementi di una sequenza:

```

int numeri[] = {1, 2, 3, 4, 5};

// Senza riferimento: modifica solo una copia, l'array non cambia
for (int n : numeri) {
    n *= 2; // cambia la copia locale, non l'array
}

// Con riferimento: modifica l'array originale
for (int& n : numeri) {
    n *= 2; // modifica l'elemento vero dell'array
}

for (int n : numeri) {
    cout << n << " "; // 2 4 6 8 10
}

```

Esempio pratico completo

```
#include <iostream>
#include <string>
using namespace std;

// Riceve nome per riferimento costante: nessuna copia, non modificabile
void stampaInfo(const string& nome, int eta) {
    cout << "Nome: " << nome << ", Eta: " << eta << endl;
}

// Modifica il punteggio direttamente
void aggiungiPunti(int& punteggio, int bonus) {
    punteggio += bonus;
}

int main() {
    string giocatore = "Alice";
    int punti = 100;

    stampaInfo(giocatore, 16); // passa il riferimento costante

    aggiungiPunti(punti, 50); // punti viene modificato direttamente
    cout << "Punti aggiornati: " << punti << endl; // 150

    return 0;
}
```

Output:

```
Nome: Alice, Eta: 16
Punti aggiornati: 150
```

Classi e Oggetti

Introduzione alla programmazione orientata agli oggetti con classi e oggetti in C++.

Cosa sono le classi e gli oggetti?

Immagina di dover gestire le informazioni di una classe scolastica con 30 studenti. Ogni studente ha un nome, un'età e dei voti.

Senza classi, avresti 30 variabili per i nomi, 30 per le età, 30 per i voti — un caos totale. Con le classi, puoi creare un "modello Studente" e poi creare 30 studenti, ognuno con i suoi dati ben organizzati.

Classe vs Oggetto: lo stampo e il prodotto

Pensa a una **classe** come allo **stampo per fare i biscotti**. Lo stampo definisce la forma — ma non è un biscotto.

Un **oggetto** è il biscotto vero, creato usando quello stampo. Puoi usare lo stesso stampo per fare decine di biscotti, ognuno diverso (con cioccolato, con glassa, ecc.).

- **Classe** = lo stampo (il progetto)
- **Oggetto** = il prodotto concreto (l'istanza)

Come si definisce una classe

```
class Cane {  
public:  
    string nome;    // caratteristiche (dati)  
    string razza;  
    int eta;  
  
    void abbaia() {                                // azioni (funzioni)  
        cout << nome << " dice: Bau!" << endl;  
    }  
  
    void descrivi() {  
        cout << nome << " è un/a " << razza  
            << " di " << eta << " anni." << endl;  
    }  
};
```

La parola **public:** significa che questi dati e queste funzioni sono accessibili dall'esterno della classe.

Come si creano oggetti

Una volta definita la classe, puoi creare quanti oggetti vuoi:

```
Cane c1;           // crea un cane (oggetto vuoto)
c1.nome = "Fido";  // imposta i dati con il punto .
c1.razza = "Labrador";
c1.eta = 3;

Cane c2;
c2.nome = "Rex";
c2.razza = "Pastore Tedesco";
c2.eta = 5;

c1.abbaia();      // Fido dice: Bau!
c2.descrivi();    // Rex è un/a Pastore Tedesco di 5 anni.
```

Il punto `.` si usa per accedere sia ai dati che alle funzioni di un oggetto.

Un esempio più completo: Rettangolo

```
class Rettangolo {
public:
    double base;
    double altezza;

    // Funzione che calcola l'area
    double area() {
        return base * altezza;
    }

    // Funzione che calcola il perimetro
    double perimetro() {
        return 2 * (base + altezza);
    }
};

int main() {
    Rettangolo r;          // crea un rettangolo
    r.base = 5.0;           // imposta i dati
    r.altezza = 3.0;

    cout << "Area: " << r.area() << endl;          // 15
    cout << "Perimetro: " << r.perimetro() << endl; // 16

    return 0;
}
```

La cosa bella è che `area()` usa automaticamente `base` e `altezza` dell'oggetto su cui viene chiamata. Non devi passarle come parametri.

Differenza tra `class` e `struct`

In C++, `class` e `struct` sono quasi identiche. L'unica differenza:

- In `struct` tutto è **pubblico** per default (accessibile dall'esterno)
- In `class` tutto è **privato** per default (non accessibile dall'esterno)

In pratica, si usa `struct` per semplici raggruppamenti di dati, e `class` per oggetti più complessi con logica interna.

Passare oggetti alle funzioni

Puoi passare un oggetto a una funzione come qualsiasi altra variabile. Per evitare di copiarlo, usa `const &` :

```
// Legge i dati del cane senza modificarli
void stampaInfoCane(const Cane& c) {
    cout << "Nome: " << c.nome << endl;
    cout << "Razza: " << c.razza << endl;
    cout << "Eta: " << c.eta << " anni" << endl;
}

int main() {
    Cane fido;
    fido.nome = "Fido";
    fido.razza = "Labrador";
    fido.eta = 3;

    stampaInfoCane(fido); // passa il cane alla funzione
    return 0;
}
```

Esempio pratico: classe Studente

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Studente {
public:
    string nome;
    vector<int> voti;    // lista di voti

    // Aggiunge un voto alla lista
    void aggiungiVoto(int voto) {
        voti.push_back(voto);
    }

    // Calcola la media dei voti
    double calcolaMedia() {
        if (voti.empty()) return 0.0;
        int somma = 0;
        for (int v : voti) {
            somma += v;
        }
        return (double)somma / voti.size();
    }

    // Stampa la scheda completa dello studente
    void stampaScheda() {
        cout << "Studente: " << nome << endl;
        cout << "Voti: ";
        for (int v : voti) {
            cout << v << " ";
        }
        cout << endl;
        cout << "Media: " << calcolaMedia() << endl;
    }
};

int main() {
    Studente alice;
    alice.nome = "Alice";
    alice.aggiungiVoto(8);
    alice.aggiungiVoto(9);
    alice.aggiungiVoto(7);
    alice.aggiungiVoto(10);
}
```

```
alice.stampaScheda();  
  
return 0;  
}
```

Output:

```
Studente: Alice  
Voti: 8 9 7 10  
Media: 8.5
```

Costruttori

Come usare i costruttori per inizializzare gli oggetti in C++.

Cos'è un costruttore?

Ogni volta che crei un oggetto, i suoi campi inizialmente contengono valori casuali. Se dimentichi di impostarli prima di usarli, il programma si comporta in modo imprevedibile.

Il **costruttore** risolve questo: è una funzione speciale che viene chiamata **automaticamente** ogni volta che crei un oggetto, e il suo compito è impostare i valori iniziali. Come il modulo di registrazione che compili quando apri un conto in banca.

Come funziona un costruttore

Un costruttore ha tre caratteristiche speciali:

1. Ha lo **stesso nome della classe**
2. **Non ha tipo di ritorno** (nemmeno `void`)
3. Viene chiamato **automaticamente** alla creazione dell'oggetto

```
class Persona {
public:
    string nome;
    int eta;

    // Questo è il costruttore (stesso nome della classe, nessun tipo di
    ritorno)
    Persona() {
        nome = "Sconosciuto"; // imposta valori di default
        eta = 0;
    }
};

int main() {
    Persona p; // il costruttore viene chiamato qui, automaticamente
    cout << p.nome << ", " << p.eta << endl; // Sconosciuto, 0
    return 0;
}
```

Costruttore con parametri

Puoi passare dati al costruttore per inizializzare l'oggetto con valori specifici:

```

class Persona {
public:
    string nome;
    int eta;

    // Costruttore con parametri
    Persona(string n, int e) {
        nome = n;
        eta = e;
    }
};

int main() {
    Persona p1("Alice", 16);    // crea Alice di 16 anni
    Persona p2("Bob", 17);      // crea Bob di 17 anni

    cout << p1.nome << ", " << p1.eta << endl;    // Alice, 16
    cout << p2.nome << ", " << p2.eta << endl;    // Bob, 17
    return 0;
}

```

La lista di inizializzazione

Un modo più efficiente di impostare i campi nel costruttore è la **lista di inizializzazione**, che si scrive dopo i due punti

:

```

class Rettangolo {
private:
    double base;
    double altezza;

public:
    // base(b) significa: inizializza base con il valore di b
    Rettangolo(double b, double a) : base(b), altezza(a) {
        // il corpo può restare vuoto
    }

    double area() const { return base * altezza; }
};

```

È preferita perché inizializza i campi direttamente, senza prima crearli vuoti e poi assegnarli.

Più costruttori per la stessa classe

Puoi avere più costruttori con parametri diversi — il compilatore sceglie quello giusto in base a come crei l'oggetto:

```
class Punto {  
public:  
    double x;  
    double y;  
  
    // Costruttore senza parametri: crea il punto nell'origine  
    Punto() : x(0.0), y(0.0) {}  
  
    // Costruttore con un valore: stessa coordinata per x e y  
    Punto(double v) : x(v), y(v) {}  
  
    // Costruttore con due valori  
    Punto(double x, double y) : x(x), y(y) {}  
};  
  
int main() {  
    Punto p1;           // (0, 0) – usa il primo costruttore  
    Punto p2(5.0);      // (5, 5) – usa il secondo costruttore  
    Punto p3(3.0, 4.0); // (3, 4) – usa il terzo costruttore  
    return 0;  
}
```

Valori di default nei parametri

Puoi dare valori predefiniti ai parametri del costruttore, così non sei obbligato a fornirli tutti:

```

class Cane {
public:
    string nome;
    string razza;
    int eta;

    // Se non specifichi razza o eta, usano i valori di default
    Cane(string n, string r = "Meticcio", int e = 0)
        : nome(n), razza(r), eta(e) {}

    void descrivi() {
        cout << nome << " (" << razza << ", " << eta << " anni)" << endl;
    }
};

int main() {
    Cane d1("Fido", "Labrador", 3);    // tutti i parametri
    Cane d2("Rex", "Pastore");          // eta usa il default (0)
    Cane d3("Luna");                    // razza e eta usano i default

    d1.descrivi(); // Fido (Labrador, 3 anni)
    d2.descrivi(); // Rex (Pastore, 0 anni)
    d3.descrivi(); // Luna (Meticcio, 0 anni)
    return 0;
}

```

Il distruttore: il contrario del costruttore

Come il costruttore crea e inizializza, il **distruttore** fa la pulizia quando l'oggetto non serve più. Si definisce con il simbolo `~` davanti al nome della classe:

```

class Risorsa {
public:
    Risorsa() {
        cout << "Risorsa creata" << endl;
    }

    ~Risorsa() {
        cout << "Risorsa distrutta" << endl;
    }
};

int main() {
    cout << "Inizio" << endl;
    {
        Risorsa r;                // costruttore chiamato qui
        cout << "In uso" << endl;
    }                             // qui r esce dal blocco → distruttore
    chiamato
    cout << "Fine" << endl;
    return 0;
}

```

Output:

```

Inizio
Risorsa creata
In uso
Risorsa distrutta
Fine

```

Il distruttore è utile soprattutto per liberare risorse (memoria, file aperti, connessioni di rete) quando l'oggetto viene eliminato.

Esempio pratico: conto bancario

```
#include <iostream>
#include <string>
using namespace std;

class ContoBancario {
private:
    string intestatario;
    double saldo;

public:
    // Costruttore: apre il conto
    ContoBancario(string nome, double saldoIniziale = 0.0)
        : intestatario(nome), saldo(saldoIniziale) {
        cout << "Conto aperto per " << intestatario << endl;
    }

    // Distruttore: chiude il conto
    ~ContoBancario() {
        cout << "Conto di " << intestatario << " chiuso." << endl;
    }

    void deposita(double importo) { saldo += importo; }
    double getSaldo() const { return saldo; }
    string getNome() const { return intestatario; }
};

int main() {
    ContoBancario c1("Alice", 1000.0); // saldo iniziale 1000
    ContoBancario c2("Bob");           // saldo iniziale 0 (default)

    c1.deposita(500.0);
    cout << c1.getNome() << ": " << c1.getSaldo() << " euro" << endl;
    cout << c2.getNome() << ": " << c2.getSaldo() << " euro" << endl;

    return 0; // qui i distruttori vengono chiamati automaticamente
}
```

Incapsulamento

Come usare `public` e `private` per proteggere i dati nelle classi C++.

Cos'è l'incapsulamento?

Immagina un distributore automatico. Puoi premere i pulsanti (interfaccia pubblica), ma non puoi mettere le mani nel meccanismo interno (dati privati). Questo è l'**incapsulamento**: nascondere i dettagli interni e offrire solo un accesso controllato dall'esterno.

In pratica: i dati di una classe vengono dichiarati **privati**, così nessuno può modificarli direttamente dall'esterno. Per accedervi, si usano metodi pubblici appositamente creati.

Il problema senza incapsulamento

Quando i dati sono pubblici, chiunque può modificarli — anche con valori assurdi:

```
class Eta {
public:
    int valore;    // pubblico: chiunque può modificarlo
};

int main() {
    Eta e;
    e.valore = -5;    // nessuno impedisce valori assurdi!
    e.valore = 9999;  // nessun controllo!
    return 0;
}
```

La soluzione: dati privati, metodi pubblici

```
class Eta {
private:
    int valore;    // privato: nessuno può accedervi direttamente
                  // dall'esterno

public:
    // Setter: imposta il valore, ma solo se è valido
    void setValore(int v) {
        if (v >= 0 && v <= 120) {
            valore = v;
        } else {
            cout << "Eta non valida!" << endl;
        }
    }

    // Getter: restituisce il valore corrente
    int getValore() const {
        return valore;
    }
};

int main() {
    Eta e;
    e.setValore(16);    // OK
    e.setValore(-5);    // "Eta non valida!" – il controllo blocca il
    valore
    e.setValore(9999);  // "Eta non valida!"

    cout << e.getValore() << endl;    // 16

    // e.valore = 5; // ERRORE di compilazione: valore è privato!
    return 0;
}
```

I tre livelli di accesso

In C++ ci sono tre parole chiave per controllare chi può accedere ai dati:

Parola chiave	Chi può accedere
<code>public</code>	Tutti — dall'interno della classe e dall'esterno
<code>private</code>	Solo dall'interno della classe
<code>protected</code>	Dalla classe e dalle classi che la estendono (ereditarietà)

Esempio completo: classe Persona

```
#include <iostream>
#include <string>
using namespace std;

class Persona {
private:
    string nome;    // dati privati
    int eta;
    string email;

public:
    // Costruttore: usa i setter per validare i valori fin dall'inizio
    Persona(string n, int e, string em) {
        nome = n;
        setEta(e);    // passa attraverso il setter
        email = em;
    }

    // Getter: leggono i dati privati (sola lettura)
    string getName() const { return nome; }
    int getEta() const { return eta; }
    string getEmail() const { return email; }

    // Setter con validazione
    void setEta(int e) {
        if (e >= 0 && e <= 150) {
            eta = e;
        } else {
            cout << "Eta non valida!" << endl;
            eta = 0;
        }
    }

    void setEmail(string em) {
        if (em.find('@') != string::npos) {    // deve contenere @
            email = em;
        } else {
            cout << "Email non valida!" << endl;
        }
    }

    // Metodo per stampare le informazioni
    void stampa() const {
        cout << "Nome: " << nome << endl;
    }
}
```



```

        cout << "Eta: " << eta << endl;
        cout << "Email: " << email << endl;
    }
};

int main() {
    Persona p("Alice", 16, "alice@esempio.it");
    p.stampa();

    p.setEta(17); // OK
    p.setEmail("non-una-email"); // Email non valida!
    p.setEmail("alice.nuovo@esempio.it"); // OK

    cout << "Nuova eta: " << p.getEta() << endl;
    cout << "Nuova email: " << p.getEmail() << endl;

    return 0;
}

```

Il vantaggio nascosto: libertà di cambiare l'interno

L'incapsulamento ti permette di modificare come funziona internamente una classe **senza dover cambiare il codice che la usa**:

```

// Versione 1: memorizza la temperatura in Celsius
class Termometro {
private:
    double celsius;
public:
    void setTemperatura(double t) { celsius = t; }
    double getTemperatura() const { return celsius; }
};

// Versione 2: internamente usa Kelvin, ma l'interfaccia è identica!
// Chi usa questa classe non sa (e non deve sapere) del cambiamento
// interno.
class Termometro {
private:
    double kelvin; // cambiato internamente
public:
    void setTemperatura(double celsius) { kelvin = celsius + 273.15; }
    double getTemperatura() const { return kelvin - 273.15; }
};

```

Chi scrive `t.setTemperatura(100)` non deve cambiare nulla — l'interfaccia è la stessa.

Ricorda: nelle `class`, tutto è privato per default

```
class Esempio {  
    int x;    // privato per default (nessun public/private specificato)  
  
public:  
    int y;    // pubblico (esplicitamente)  
};
```

È buona abitudine specificare sempre `public:` e `private:` per chiarezza, anche se tecnicamente non sarebbe necessario.

Enumerazioni (enum)

Come usare le enumerazioni in C++ per definire insiemi di costanti con nome.

Cos'è un enum?

Supponi di dover rappresentare i giorni della settimana nel tuo programma. Potresti usare i numeri 0, 1, 2... ma chi si ricorda che 3 è giovedì?

Un'enumerazione (`enum`) ti permette di dare nomi significativi a un insieme fisso di valori. Invece di `3`, scrivi `GIOVEDI`. Il codice diventa leggibile come l'italiano.

Definire un `enum`

```
enum NomeEnum {  
    VALORE1,  
    VALORE2,  
    VALORE3  
};
```

Esempio concreto:

```
enum GiornoSettimana {  
    LUNEDI,  
    MARTEDI,  
    MERCOLEDI,  
    GIOVEDI,  
    VENERDI,  
    SABATO,  
    DOMENICA  
};
```

Per convenzione, i valori si scrivono tutti in **MAIUSCOLO**.

Usare un `enum`

```
GiornoSettimana oggi = MERCOLEDI;    // dichiara una variabile di tipo  
GiornoSettimana  
  
if (oggi == SABATO || oggi == DOMENICA) {  
    cout << "Weekend!" << endl;  
} else {  
    cout << "Giorno lavorativo." << endl;  
}
```

I valori numerici nascosti

Internamente, ogni valore dell'enum è un numero. Per default, il primo vale 0, il secondo 1, e così via:

```
enum Stagione {  
    PRIMAVERA, // 0  
    ESTATE,    // 1  
    AUTUNNO,   // 2  
    INVERNO    // 3  
};  
  
Stagione s = ESTATE;  
cout << s << endl; // stampa 1 (il numero interno)
```

Puoi assegnare valori specifici:

```
enum Mese {  
    GENNAIO = 1, // parte da 1 invece di 0  
    FEBBRAIO,    // 2  
    MARZO,       // 3  
    // ...  
    DICEMBRE = 12  
};  
  
Mese m = MARZO;  
cout << m << endl; // 3
```

Abbinare enum e switch

enum e switch si abbinano perfettamente — il codice è chiaro e privo di ambiguità:

```

enum Semaforo {
    ROSSO,
    GIALLO,
    VERDE
};

void istruzione(Semaforo s) {
    switch (s) {
        case ROSSO:
            cout << "STOP!" << endl;
            break;
        case GIALLO:
            cout << "Attenzione, rallentare." << endl;
            break;
        case VERDE:
            cout << "Via libera!" << endl;
            break;
    }
}

int main() {
    istruzione(ROSSO);    // STOP!
    istruzione(VERDE);    // Via libera!
    return 0;
}

```

enum class : la versione moderna e più sicura

Il C++ moderno introduce `enum class`, che evita un problema degli enum classici: i nomi possono andare in conflitto tra enum diversi.

```
// Problema con enum classico:
enum Colore { ROSSO, VERDE, BLU };
enum Frutto { ROSSO, VERDE, GIALLO }; // ERRORE: ROSSO è già definito!

// Soluzione con enum class:
enum class Colore { ROSSO, VERDE, BLU };
enum class Frutto { ROSSO, VERDE, GIALLO }; // nessun conflitto!

int main() {
    Colore c = Colore::ROSSO; // devi specificare il nome dell'enum
    Frutto f = Frutto::ROSSO; // completamente separato da Colore::ROSSO

    if (c == Colore::ROSSO) {
        cout << "Il colore è rosso" << endl;
    }
    return 0;
}
```

Con `enum class` devi sempre scrivere `NomeEnum::VALORE` — è più lungo, ma non ci sono mai ambiguità. È la scelta preferita nel C++ moderno.

Convertire tra `enum` e numeri

```
enum Livello { FACILE, MEDIO, DIFFICILE };

Livello l = MEDIO;
int n = static_cast<int>(l); // da enum a numero → 1
cout << n << endl;

Livello l2 = static_cast(2); // da numero a enum → DIFFICILE
```

Esempio pratico: sistema di voti

```
#include <iostream>
using namespace std;

enum class Giudizio {
    OTTIMO,
    BUONO,
    SUFFICIENTE,
    INSUFFICIENTE
};

// Converte un voto numerico nel giudizio corrispondente
Giudizio classificaVoto(int voto) {
    if (voto >= 9) return Giudizio::OTTIMO;
    if (voto >= 7) return Giudizio::BUONO;
    if (voto >= 6) return Giudizio::SUFFICIENTE;
    return Giudizio::INSUFFICIENTE;
}

// Stampa il giudizio in parole
void stampaGiudizio(Giudizio g) {
    switch (g) {
        case Giudizio::OTTIMO:      cout << "Ottimo!" << endl;
        break;
        case Giudizio::BUONO:       cout << "Buono" << endl;
        break;
        case Giudizio::SUFFICIENTE: cout << "Sufficiente" << endl;
        break;
        case Giudizio::INSUFFICIENTE: cout << "Insufficiente" << endl;
        break;
    }
}

int main() {
    int voti[] = {9, 7, 5, 10, 6};
    for (int voto : voti) {
        cout << "Voto " << voto << ": ";
        stampaGiudizio(classificaVoto(voto));
    }
    return 0;
}
```

Output:

Voto 9: Ottimo!
Voto 7: Buono
Voto 5: Insufficiente
Voto 10: Ottimo!
Voto 6: Sufficiente

Ereditarietà in C++

Introduzione all'ereditarietà in C++, come creare classi derivate da classi esistenti.

Cos'è l'ereditarietà?

Immagina di dover creare le classi `Cane`, `Gatto` e `Coniglio`. Tutte e tre hanno un nome, un'età, mangiano e dormono. Riscrivere le stesse cose tre volte è una perdita di tempo.

L'**ereditarietà** risolve questo: scrivi una classe `Animale` con le caratteristiche comuni, e poi `Cane`, `Gatto` e `Coniglio` **ereditano** tutto da essa, aggiungendo solo quello che hanno in più.

Come in natura: un cane è un animale. Ha tutto quello che ha un animale, più le caratteristiche proprie del cane.

Come funziona l'ereditarietà

La classe "figlia" (derivata) eredita tutto dalla classe "madre" (base):


```

// Classe base (la "madre")
class Animale {
public:
    string nome;
    int eta;

    void mangia() {
        cout << nome << " sta mangiando." << endl;
    }

    void dormi() {
        cout << nome << " sta dormendo." << endl;
    }
};

// Classe derivata (la "figlia") – eredita da Animale
class Cane : public Animale {
public:
    string razza;    // campo aggiuntivo specifico del Cane

    void abbaia() {
        cout << nome << " dice: Bau!" << endl;    // usa il campo di
Animale!
    }
};

// Un'altra classe derivata
class Gatto : public Animale {
public:
    bool viveInCasa;

    void faSeLeFusa() {
        cout << nome << " fa le fusa." << endl;
    }
};

```

Usare le classi derivate

```
int main() {
    Cane fido;
    fido.nome = "Fido";      // ereditato da Animale
    fido.eta = 3;
    fido.razza = "Labrador"; // specifico di Cane

    fido.mangia();           // metodo ereditato da Animale
    fido.abbaia();           // metodo proprio di Cane

    Gatto luna;
    luna.nome = "Luna";
    luna.dormi();            // metodo ereditato
    luna.faSeLeFusa();       // metodo proprio di Gatto

    return 0;
}
```

Il costruttore nelle classi derivate

Quando crei un oggetto `Cane`, il C++ chiama prima il costruttore di `Animale` e poi quello di `Cane`. Per passare dati al costruttore della classe madre, usa `:` nella lista di inizializzazione:

```

class Animale {
public:
    string nome;
    int eta;

    Animale(string n, int e) : nome(n), eta(e) {
        cout << "Animale creato: " << nome << endl;
    }
};

class Cane : public Animale {
public:
    string razza;

    // Chiama il costruttore di Animale passandogli n e e
    Cane(string n, int e, string r) : Animale(n, e), razza(r) {
        cout << "Cane creato: " << nome << " (" << razza << ")" << endl;
    }
};

int main() {
    Cane fido("Fido", 3, "Labrador");
    // Stampa:
    // Animale creato: Fido
    // Cane creato: Fido (Labrador)
    return 0;
}

```

Sovrascrivere un metodo della classe madre

Una classe figlia può **ridefinire** un metodo della madre per dargli un comportamento diverso:

```

class Animale {
public:
    string nome;

    void emettiSuono() {
        cout << nome << " emette un suono generico." << endl;
    }
};

class Cane : public Animale {
public:
    // Sovrascrive il metodo della classe madre
    void emettiSuono() {
        cout << nome << " dice: Bau!" << endl;
    }
};

class Gatto : public Animale {
public:
    void emettiSuono() {
        cout << nome << " dice: Miao!" << endl;
    }
};

int main() {
    Animale a; a.nome = "Generico";
    Cane c;    c.nome = "Fido";
    Gatto g;   g.nome = "Luna";

    a.emettiSuono(); // emette un suono generico
    c.emettiSuono(); // Bau!
    g.emettiSuono(); // Miao!
    return 0;
}

```

Chiamare il metodo della classe madre

Se hai bisogno di usare anche il metodo originale della madre dentro quello della figlia:

```

class Animale {
public:
    virtual void descrivi() {
        cout << "Sono un animale." << endl;
    }
};

class Cane : public Animale {
public:
    void descrivi() {
        Animale::descrivi();    // chiama prima il metodo della madre
        cout << "Sono un cane." << endl;    // poi aggiunge qualcosa
    }
};

```

protected : il livello intermedio

private significa che nemmeno le classi figlie possono accedere al dato. A volte vuoi qualcosa di più accessibile, ma non completamente pubblico: usa **protected** :

```

class Animale {
protected:
    string nome;    // le classi figlie possono usarlo
private:
    int codiceInterno;    // le classi figlie NON possono vederlo
public:
    Animale(string n) : nome(n) {}
};

class Cane : public Animale {
public:
    Cane(string n) : Animale(n) {}

    void abbaia() {
        cout << nome << " dice: Bau!";    // OK: nome è protected
        cout << codiceInterno;    // ERRORE: è private
    }
};

```

Catene di ereditarietà

Puoi creare gerarchie con più livelli:

```
class Veicolo {
public:
    int velocitaMassima;
    void muoviti() { cout << "Mi muovo." << endl; }
};

class Auto : public Veicolo {
public:
    int numeroPosti;
};

class SportCar : public Auto {
public:
    bool haIlTurbo;
    void sgomma() { cout << "Sgommata!" << endl; }
};
```

`SportCar` eredita da `Auto`, che eredita da `Veicolo`. Quindi `SportCar` ha `velocitaMassima`, `numeroPosti`, `haIlTurbo`, più i metodi `muoviti()` e `sgomma()`.

Metodi

Come definire e usare i metodi nelle classi C++.

Cosa sono i metodi?

Un oggetto non è solo un contenitore di dati — può anche fare cose. I **metodi** sono le azioni che un oggetto sa compiere.

Un oggetto `Cane` ha dati (nome, razza, età) ma può anche abbaiare, mangiare, correre. Un oggetto `ContoBancario` ha un saldo, ma può anche ricevere depositi e prelievi.

I metodi sono semplicemente **funzioni scritte dentro una classe**.

Definire metodi dentro la classe

```
class Cerchio {
public:
    double raggio;

    // Metodo che calcola l'area
    double area() {
        return 3.14159 * raggio * raggio;
    }

    // Metodo che calcola il perimetro
    double perimetro() {
        return 2 * 3.14159 * raggio;
    }
};

int main() {
    Cerchio c;
    c.raggio = 5.0;

    cout << "Area: " << c.area() << endl;          // usa automaticamente
c.raggio
    cout << "Perimetro: " << c.perimetro() << endl;
}
```

Il metodo ha accesso diretto ai dati dell'oggetto su cui viene chiamato.

Separare dichiarazione e definizione

Per classi grandi, puoi dichiarare il metodo dentro la classe e scriverne il corpo fuori, usando `NomeClasse::` :

```

class Cerchio {
public:
    double raggio;

    double area();          // solo la firma (dichiarazione)
    double perimetro();     // solo la firma
};

// Il corpo viene scritto fuori dalla classe
double Cerchio::area() {
    return 3.14159 * raggio * raggio;
}

double Cerchio::perimetro() {
    return 2 * 3.14159 * raggio;
}

```

L'operatore `::` dice "questo metodo appartiene alla classe Cerchio".

Metodi che non modificano i dati: **const**

Se un metodo legge solo i dati senza cambiarli, aggiungere `const` alla fine è una buona abitudine. Serve come promessa al compilatore (e al lettore del codice):

```

class Rettangolo {
public:
    double base;
    double altezza;

    double area() const {          // const = non modifica i dati
        return base * altezza;
    }

    void ridimensiona(double b, double a) { // nessun const = può
        modificare
        base = b;
        altezza = a;
    }
};

```


Se provi a modificare un campo dentro un metodo `const`, il compilatore ti segnala l'errore.

Risolvere conflitti di nome con `this`

Dentro un metodo, `this` è un puntatore speciale che indica "l'oggetto corrente". Si usa raramente, ma è utile quando il nome di un parametro coincide con il nome di un campo:

```
class Persona {  
public:  
    string nome;  
  
    void setName(string nome) {  
        // "nome" da solo si riferisce al parametro  
        // "this->nome" si riferisce al campo della classe  
        this->nome = nome;  
    }  
};
```

Getter e Setter: porte controllate per i dati privati

Quando i dati di una classe sono privati (non accessibili dall'esterno), si usano metodi appositi per leggerli e modificarli:

- **Getter:** legge un dato privato
- **Setter:** modifica un dato privato (può anche validare)

```

class Temperatura {
private:
    double gradi;    // non accessibile dall'esterno

public:
    // Setter: imposta la temperatura, ma solo se valida
    void setGradi(double t) {
        if (t >= -273.15) {    // non può scendere sotto lo zero assoluto
            gradi = t;
        } else {
            cout << "Temperatura non valida!" << endl;
        }
    }

    // Getter: legge la temperatura
    double getGradi() const {
        return gradi;
    }

    // Metodo utile: converte in Fahrenheit
    double inFahrenheit() const {
        return gradi * 9.0/5.0 + 32;
    }
};

int main() {
    Temperatura t;
    t.setGradi(100.0);
    cout << t.getGradi() << "°C = " << t.inFahrenheit() << "°F" << endl;
    // 100°C = 212°F

    t.setGradi(-300.0);    // Temperatura non valida!
    return 0;
}

```

Esempio pratico: conto bancario

```
#include <iostream>
#include <string>
using namespace std;

class ContoBancario {
private:
    string intestatario;
    double saldo;

public:
    // Inizializza il conto
    void apri(string nome, double saldoIniziale) {
        intestatario = nome;
        saldo = saldoIniziale;
    }

    // Aggiunge denaro al saldo
    void deposita(double importo) {
        if (importo > 0) {
            saldo += importo;
            cout << "Depositati " << importo
                  << " euro. Saldo: " << saldo << " euro." << endl;
        }
    }

    // Toglie denaro dal saldo (solo se ci sono fondi)
    bool preleva(double importo) {
        if (importo > 0 && importo <= saldo) {
            saldo -= importo;
            cout << "Prelevati " << importo
                  << " euro. Saldo: " << saldo << " euro." << endl;
            return true;
        }
        cout << "Fondi insufficienti!" << endl;
        return false;
    }

    // Getter per leggere i dati privati
    double getSaldo() const { return saldo; }
    string getIntestatario() const { return intestatario; }
};

int main() {
    ContoBancario conto;
```

```
conto.apri("Alice", 1000.0);

conto.deposita(500.0);    // Depositati 500 euro. Saldo: 1500.
conto.preleva(200.0);    // Prelevati 200 euro. Saldo: 1300.
conto.preleva(2000.0);   // Fondi insufficienti!

cout << "Saldo finale: " << conto.getSaldo() << " euro" << endl;

return 0;
}
```

Polimorfismo

Il concetto di polimorfismo in C++, con funzioni virtuali e override.

Cos'è il polimorfismo?

Immagina di avere una fattoria con cani, gatti e mucche. Vuoi far "parlare" tutti gli animali, uno per uno. Senza il polimorfismo, dovresti scrivere un ciclo separato per ogni tipo di animale.

Con il **polimorfismo**, puoi mettere tutti gli animali in una stessa lista e far sì che ognuno risponda al comando "emetti suono" a modo suo — automaticamente, senza controllare di che tipo è.

Polimorfismo significa "molte forme": lo stesso comando, comportamenti diversi a seconda dell'oggetto.

Il problema senza polimorfismo

```
class Cane {
public:
    void suona() { cout << "Bau!" << endl; }
};

class Gatto {
public:
    void suona() { cout << "Miao!" << endl; }
};

// Senza polimorfismo, serve una funzione per ogni tipo
void faiSuonare(Cane& c) { c.suona(); }
void faiSuonare(Gatto& g) { g.suona(); }
// E se aggiungi Mucca? Devi aggiungere un'altra funzione...
```

La parola chiave **virtual**

Aggiungendo **virtual** davanti a un metodo nella classe madre, dici al C++ di usare il comportamento della classe figlia, non della madre:

```

class Animale {
public:
    string nome;

    virtual void emettiSuono() { // virtual = usa il metodo della figlia
        cout << nome << " emette un suono." << endl;
    }
};

class Cane : public Animale {
public:
    void emettiSuono() override { // override = sovrascrive il metodo
base
        cout << nome << " dice: Bau!" << endl;
    }
};

class Gatto : public Animale {
public:
    void emettiSuono() override {
        cout << nome << " dice: Miao!" << endl;
    }
};

```

La parola **override** non è obbligatoria, ma è una buona abitudine: il compilatore ti avvisa se stai cercando di sovrascrivere un metodo che non esiste nella classe madre.

Il polimorfismo in azione

La "magia" si vede quando usi un **puntatore alla classe madre** per gestire oggetti di classi figlie diverse:

```

Animale* a1 = new Cane();    // puntatore ad Animale, ma è un Cane
Animale* a2 = new Gatto();   // puntatore ad Animale, ma è un Gatto

a1->nome = "Fido";
a2->nome = "Luna";

a1->emettiSuono();    // Fido dice: Bau!    (usa il metodo di Cane!)
a2->emettiSuono();    // Luna dice: Miao!    (usa il metodo di Gatto!)

delete a1;
delete a2;

```

Senza `virtual`, entrambi avrebbero chiamato il metodo di `Animale` (suono generico). Con `virtual`, il C++ sa di dover usare il metodo del tipo reale dell'oggetto.

Gestire una lista mista di oggetti

Questo è il punto di forza del polimorfismo: puoi mettere tipi diversi in una stessa lista e trattarli tutti allo stesso modo:

```

#include <iostream>
#include <vector>
using namespace std;

class Animale {
public:
    string nome;
    Animale(string n) : nome(n) {}
    virtual void emettiSuono() = 0;    // obbliga le classi figlie a
    implementarlo
    virtual ~Animale() {}             // distruttore virtuale (importante!)
};

class Cane : public Animale {
public:
    Cane(string n) : Animale(n) {}
    void emettiSuono() override { cout << nome << ": Bau!" << endl; }
};

class Gatto : public Animale {
public:
    Gatto(string n) : Animale(n) {}
    void emettiSuono() override { cout << nome << ": Miao!" << endl; }
};

class Mucca : public Animale {
public:
    Mucca(string n) : Animale(n) {}
    void emettiSuono() override { cout << nome << ": Mu!" << endl; }
};

int main() {
    vector fattoria;
    fattoria.push_back(new Cane("Fido"));
    fattoria.push_back(new Gatto("Luna"));
    fattoria.push_back(new Mucca("Bessie"));
    fattoria.push_back(new Cane("Rex"));

    // Un solo ciclo gestisce tutti i tipi di animali!
    for (Animale* a : fattoria) {
        a->emettiSuono();
    }

    // Libera la memoria
    for (Animale* a : fattoria) {
        delete a;
    }
}

```



```
    return 0;  
}
```

Output:

```
Fido: Bau!  
Luna: Miao!  
Bessie: Mu!  
Rex: Bau!
```

Classi astratte: il modello obbligatorio

Quando scrivi `= 0` dopo un metodo virtuale, quel metodo diventa **puro virtuale**: non ha corpo nella classe madre e ogni classe figlia è obbligata a implementarlo.

Una classe con almeno un metodo puro virtuale è **astratta**: non puoi creare oggetti direttamente da essa.

```

class Forma {
public:
    // Ogni forma DEVE implementare questi due metodi
    virtual double area() const = 0;
    virtual double perimetro() const = 0;

    // Questo metodo usa i precedenti (funziona grazie al polimorfismo)
    void stampa() const {
        cout << "Area: " << area() << ", Perimetro: " << perimetro() <<
endl;
    }
};

class Cerchio : public Forma {
private:
    double raggio;
public:
    Cerchio(double r) : raggio(r) {}
    double area() const override { return 3.14159 * raggio * raggio; }
    double perimetro() const override { return 2 * 3.14159 * raggio; }
};

class Rettangolo : public Forma {
private:
    double base, altezza;
public:
    Rettangolo(double b, double a) : base(b), altezza(a) {}
    double area() const override { return base * altezza; }
    double perimetro() const override { return 2 * (base + altezza); }
};

int main() {
    // Forma f; // ERRORE: Forma è astratta, non puoi creare oggetti diretti

    Cerchio c(5.0);
    Rettangolo r(4.0, 6.0);

    c.stampa(); // Area: 78.5397, Perimetro: 31.4159
    r.stampa(); // Area: 24, Perimetro: 20

    return 0;
}

```

Il distruttore virtuale

Quando usi il polimorfismo con puntatori alla classe madre, dichiara sempre il distruttore della classe madre come `virtual`. Altrimenti, quando elimini un oggetto con `delete`, viene chiamato solo il distruttore della madre — non quello della figlia:

```
class Base {
public:
    virtual ~Base() {           // distruttore virtuale
        cout << "Base distrutta" << endl;
    }
};

class Derivata : public Base {
public:
    ~Derivata() override {
        cout << "Derivata distrutta" << endl;
    }
};

Base* ptr = new Derivata();
delete ptr;    // con virtual: chiama Derivata prima, poi Base
               // senza virtual: chiama solo Base (bug!)
```

Strutture (struct)

Come raggruppare dati correlati in strutture in C++.

Cos'è una struct?

Immagina di dover tenere traccia di 30 studenti. Ogni studente ha un nome, un'età e un voto. Potresti usare tre array separati — ma è un disastro da gestire.

Una **struttura** (`struct`) ti permette di raggruppare tutte le informazioni di uno studente in un unico "pacchetto". Così hai un array di studenti, non tre array separati.

Definire una struttura

```
struct NomeStruttura {  
    tipo1 campo1;  
    tipo2 campo2;  
    // ...  
}; // nota il punto e virgola finale!
```

Esempio concreto:

```
struct Studente {  
    string nome;  
    int eta;  
    double media;  
};
```

Creare e usare una struttura

```
Studente s1;           // crea una variabile di tipo Studente  
  
// Accedi ai campi con il punto .  
s1.nome = "Alice";  
s1.eta = 16;  
s1.media = 8.5;  
  
cout << s1.nome << endl; // Alice  
cout << s1.media << endl; // 8.5
```

Inizializzazione rapida

Puoi impostare tutti i valori in una sola riga:

```
Studente s1 = {"Alice", 16, 8.5}; // ordine: nome, eta, media

// C++11: puoi anche specificare il nome dei campi
Studente s2 = {.nome = "Bob", .eta = 17, .media = 7.2};
```

Strutture come parametri di funzioni

Le strutture si passano alle funzioni esattamente come le variabili normali:

```
struct Punto {
    double x;
    double y;
};

// Calcola la distanza tra due punti
double distanza(Punto p1, Punto p2) {
    double dx = p1.x - p2.x;
    double dy = p1.y - p2.y;
    return sqrt(dx*dx + dy*dy);
}

int main() {
    Punto a = {0.0, 0.0};
    Punto b = {3.0, 4.0};

    cout << "Distanza: " << distanza(a, b) << endl; // 5
    return 0;
}
```

Per modificare una struttura dentro una funzione, passa per riferimento (&):

```
struct Contatore {  
    int valore;  
    int passi;  
};  
  
void incrementa(Contatore& c, int n) {  
    c.valore += n;    // modifica l'originale, non una copia  
    c.passi++;  
}  
  
int main() {  
    Contatore c = {0, 0};  
    incrementa(c, 10);  
    incrementa(c, 5);  
    cout << "Valore: " << c.valore << ", Passi: " << c.passi << endl;  
    // Valore: 15, Passi: 2  
    return 0;  
}
```

Array di strutture

Puoi creare array di strutture per gestire più elementi dello stesso tipo:

```

struct Prodotto {
    string nome;
    double prezzo;
    int quantita;
};

int main() {
    Prodotto magazzino[3] = {
        {"Mela", 0.50, 100},
        {"Banana", 0.30, 150},
        {"Arancia", 0.80, 80}
    };

    cout << "Prodotti in magazzino:" << endl;
    for (int i = 0; i < 3; i++) {
        cout << magazzino[i].nome << " - "
            << magazzino[i].prezzo << " euro"
            << " (quantita: " << magazzino[i].quantita << ")" << endl;
    }

    // Calcola il valore totale
    double totale = 0;
    for (int i = 0; i < 3; i++) {
        totale += magazzino[i].prezzo * magazzino[i].quantita;
    }
    cout << "Valore totale: " << totale << " euro" << endl;

    return 0;
}

```

Strutture annidate

Una struttura può contenere un'altra struttura come campo:

```

struct Indirizzo {
    string via;
    string citta;
    string cap;
};

struct Persona {
    string nome;
    int eta;
    Indirizzo indirizzo; // struttura dentro struttura
};

int main() {
    Persona p;
    p.nome = "Alice";
    p.eta = 16;
    p.indirizzo.via = "Via Roma 10";
    p.indirizzo.citta = "Milano";
    p.indirizzo.cap = "20100";

    cout << p.nome << " vive a " << p.indirizzo.citta << endl;
    return 0;
}

```

Differenza tra struct e class

In C++, `struct` e `class` sono quasi identiche. L'unica differenza:

- In `struct` tutto è **pubblico** per default
- In `class` tutto è **privato** per default

Nella pratica, si usa `struct` per raggruppare dati semplici senza logica, e `class` per oggetti più complessi con comportamenti e dati protetti.

Esempio pratico: registro studenti

```
#include <iostream>
#include <string>
using namespace std;

struct Studente {
    string nome;
    int eta;
    double voto;
};

// Passa per const & per leggere senza copiare
void stampa(const Studente& s) {
    cout << s.nome << " (" << s.eta << " anni) - Voto: " << s.voto << endl;
}

int main() {
    const int N = 3;
    Studente classe[N] = {
        {"Alice", 16, 8.5},
        {"Bob", 17, 7.0},
        {"Carlo", 16, 9.2}
    };

    cout << "=== Registro ===" << endl;
    double somma = 0;
    for (int i = 0; i < N; i++) {
        stampa(classe[i]);
        somma += classe[i].voto;
    }

    cout << "Media della classe: " << somma / N << endl;

    return 0;
}
```

Output:

```
=== Registro ===  
Alice (16 anni) – Voto: 8.5  
Bob (17 anni) – Voto: 7  
Carlo (16 anni) – Voto: 9.2  
Media della classe: 8.23333
```

Algoritmi di Base

Introduzione agli algoritmi di ricerca e ordinamento più comuni, implementati in C++.

Cos'è un algoritmo?

Un **algoritmo** è una sequenza di passi ben definiti per risolvere un problema — come una ricetta di cucina.

Capire gli algoritmi di base ti insegna a pensare in modo strutturato e a valutare quanto "veloce" o "lento" è il tuo codice. Sono fondamentali in informatica.

Vediamo i quattro algoritmi più importanti per chi inizia: ricerca lineare, ricerca binaria, bubble sort e selection sort.

Ricerca Lineare

Come funziona

La **ricerca lineare** è la più semplice: guardi gli elementi uno per uno dall'inizio alla fine, finché non trovi quello che cerchi.

È come cercare il tuo nome in un elenco scritto a mano: leggi il primo nome, poi il secondo, poi il terzo... finché lo trovi o arrivi in fondo.

Implementazione

```
#include <iostream>
using namespace std;

// Cerca target nell'array. Restituisce la posizione (indice) oppure -1 se
// non trovato.
int ricercaLineare(int arr[], int dimensione, int target) {
    for (int i = 0; i < dimensione; i++) {
        if (arr[i] == target) {
            return i;    // trovato! restituisce la posizione
        }
    }
    return -1;          // non trovato
}

int main() {
    int numeri[] = {15, 3, 8, 42, 7, 19, 11};
    int dim = 7;
    int cercato = 42;

    int risultato = ricercaLineare(numeri, dim, cercato);

    if (risultato != -1) {
        cout << "Trovato " << cercato << " alla posizione " << risultato <<
endl;
    } else {
        cout << cercato << " non trovato" << endl;
    }

    return 0;
}
```

Output:

```
Trovato 42 alla posizione 3
```

Caratteristiche

- Funziona su array **non ordinati** — l'ordine non importa
- Nel caso peggiore (elemento non presente o all'ultimo posto): controlla tutti gli **n** elementi

- Semplicissimo da scrivere e capire

Ricerca Binaria

Come funziona

La **ricerca binaria** è molto più veloce, ma richiede che l'array sia **già ordinato**.

Pensa a quando cerchi una parola nel dizionario. Non inizi dalla prima pagina — apri il dizionario a metà. Se la parola viene prima, guardi nella prima metà; se viene dopo, guardi nella seconda metà. Ripeti, dimezzando ogni volta lo spazio di ricerca.

Passi:

1. Guarda l'elemento al centro dell'array
2. Se è quello cercato, hai finito
3. Se quello cercato è più piccolo, continua solo nella metà sinistra
4. Se quello cercato è più grande, continua solo nella metà destra
5. Ripeti finché trovi l'elemento o lo spazio si esaurisce

Implementazione

```
#include <iostream>
using namespace std;

int ricercaBinaria(int arr[], int dimensione, int target) {
    int sinistra = 0;
    int destra = dimensione - 1;

    while (sinistra <= destra) {
        int centro = sinistra + (destra - sinistra) / 2;

        if (arr[centro] == target) {
            return centro;           // trovato!
        } else if (arr[centro] < target) {
            sinistra = centro + 1;    // cerca nella metà destra
        } else {
            destra = centro - 1;      // cerca nella metà sinistra
        }
    }

    return -1; // non trovato
}

int main() {
    // IMPORTANTE: l'array deve essere ordinato!
    int numeri[] = {2, 5, 8, 12, 16, 23, 38, 45, 72, 91};
    int dim = 10;
    int cercato = 23;

    int risultato = ricercaBinaria(numeri, dim, cercato);

    if (risultato != -1) {
        cout << "Trovato " << cercato << " alla posizione " << risultato <<
endl;
    } else {
        cout << cercato << " non trovato" << endl;
    }

    return 0;
}
```

Output:

Trovato 23 alla posizione 5

Visualizzazione passo per passo

Cerchiamo 23 in {2, 5, 8, 12, 16, 23, 38, 45, 72, 91} :

Passo 1: centro = 4 $\rightarrow$ arr[4] = 16 < 23 $\rightarrow$ vai a destra (sinistra = 5)

Passo 2: centro = 7 $\rightarrow$ arr[7] = 45 > 23 $\rightarrow$ vai a sinistra (destra = 6)

Passo 3: centro = 5 $\rightarrow$ arr[5] = 23 $\rightarrow$ TROVATO!

Soli 3 passi su un array di 10 elementi. Con un milione di elementi, ne basterebbero circa 20.

Bubble Sort

Come funziona

Il **bubble sort** (ordinamento a bolla) scorre l'array più volte. A ogni passata, confronta gli elementi vicini a coppie e li scambia se sono nell'ordine sbagliato. I valori grandi "salgono" verso la fine, come bolle d'acqua.

Implementazione

```
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int dimensione) {
    for (int i = 0; i < dimensione - 1; i++) {
        // Ogni passata sposta il valore più grande in fondo
        for (int j = 0; j < dimensione - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                // Scambia i due elementi vicini
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void stampaArray(int arr[], int dimensione) {
    for (int i = 0; i < dimensione; i++) {
        cout << arr[i];
        if (i < dimensione - 1) cout << ", ";
    }
    cout << endl;
}

int main() {
    int numeri[] = {64, 34, 25, 12, 22, 11, 90};
    int dim = 7;

    cout << "Prima dell'ordinamento: ";
    stampaArray(numeri, dim);

    bubbleSort(numeri, dim);

    cout << "Dopo l'ordinamento: ";
    stampaArray(numeri, dim);

    return 0;
}
```

Output:

Prima dell'ordinamento: 64, 34, 25, 12, 22, 11, 90
Dopo l'ordinamento: 11, 12, 22, 25, 34, 64, 90

Visualizzazione di una passata

Array: {64, 34, 25, 12}

64 e 34: $64 > 34 \rightarrow$ scambia $\rightarrow$ {34, 64, 25, 12}
64 e 25: $64 > 25 \rightarrow$ scambia $\rightarrow$ {34, 25, 64, 12}
64 e 12: $64 > 12 \rightarrow$ scambia $\rightarrow$ {34, 25, 12, 64}

Il 64 è finito in fondo. Si ripete per il resto.

Selection Sort

Come funziona

Il **selection sort** (ordinamento per selezione) funziona in modo diverso:

1. Trova il valore più piccolo e mettilo in prima posizione
2. Trova il secondo valore più piccolo e mettilo in seconda posizione
3. Ripeti fino alla fine

Come ordinare le carte: ogni volta "scegli" la carta più bassa e la metti al suo posto.

Implementazione

```
#include <iostream>
using namespace std;

void selectionSort(int arr[], int dimensione) {
    for (int i = 0; i < dimensione - 1; i++) {
        // Trova il minimo nella parte non ancora ordinata
        int indiceMinimo = i;
        for (int j = i + 1; j < dimensione; j++) {
            if (arr[j] < arr[indiceMinimo]) {
                indiceMinimo = j;
            }
        }

        // Scambia il minimo trovato con l'elemento in posizione i
        if (indiceMinimo != i) {
            int temp = arr[i];
            arr[i] = arr[indiceMinimo];
            arr[indiceMinimo] = temp;
        }
    }
}

int main() {
    int numeri[] = {64, 25, 12, 22, 11};
    int dim = 5;

    cout << "Prima: ";
    for (int i = 0; i < dim; i++) cout << numeri[i] << " ";
    cout << endl;

    selectionSort(numeri, dim);

    cout << "Dopo: ";
    for (int i = 0; i < dim; i++) cout << numeri[i] << " ";
    cout << endl;

    return 0;
}
```

Output:

Prima: 64 25 12 22 11
Dopo: 11 12 22 25 64

Confronto tra gli algoritmi

Algoritmo	Array ordinato richiesto?	Velocità (caso peggiore)	Difficoltà
Ricerca Lineare	No	N confronti	Facilissima
Ricerca Binaria	Sì	~20 confronti per 1 milione di elementi	Moderata
Bubble Sort	No (ordina lui)	N ² confronti	Facile
Selection Sort	No (ordina lui)	N ² confronti	Facile

- Usa la **ricerca lineare** su array piccoli o non ordinati
- Usa la **ricerca binaria** su array grandi e già ordinati
- **Bubble sort** e **selection sort** sono utili per capire i concetti, ma lenti su array grandi

La soluzione pronta in C++: `std::sort`

Nella pratica, non scrivere mai bubble sort o selection sort in un programma vero. La libreria standard ha `sort()` che è molto più veloce:

```

#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

int main() {
    vector<int> numeri = {64, 25, 12, 22, 11, 90, 38};

    sort(numeri.begin(), numeri.end()); // ordina in modo efficiente

    cout << "Ordinato: ";
    for (int n : numeri) cout << n << " ";
    cout << endl;

    // Ricerca binaria già pronta
    if (binary_search(numeri.begin(), numeri.end(), 22)) {
        cout << "22 trovato!" << endl;
    }

    return 0;
}

```

Output:

```

Ordinato: 11 12 22 25 38 64 90
22 trovato!

```

Studiare gli algoritmi "di base" ti insegna a ragionare passo per passo e a capire il concetto di efficienza — due delle basi più importanti dell'informatica.

Buone Pratiche di Codice

Come scrivere codice C++ leggibile, manutenibile e professionale.

Scrivere codice che si capisce

Scrivere codice che funziona è il primo traguardo. Scrivere codice che **si capisce, si modifica facilmente e non crea problemi nel tempo** è il secondo — ed è quello che distingue un programmatore che cresce da uno che rimane bloccato.

Queste abitudini ti aiutano a:

- Trovare i bug più in fretta
- Capire il tuo stesso codice dopo settimane senza toccarlo
- Lavorare con altri programmatori senza causare confusione
- Evitare gli errori classici che fanno perdere tempo

Usa nomi che spiegano da soli

Il nome di una variabile o di una funzione deve dire **cosa rappresenta** o **cosa fa**:

```
// Scadente: cosa sono x, y, b?  
int x, y;  
double d;  
bool b;  
  
// Buono: si capisce subito  
int larghezza, altezza;  
double prezzoUnitario;  
bool maggiorenne;
```

Lo stesso vale per le funzioni:

```
// Scadente  
void f(int a, int b) { return a + b; }  
  
// Buono: si capisce cosa fa senza leggere il corpo  
int calcolaSomma(int primo, int secondo) { return primo + secondo; }
```

Commenta il "perché", non il "cosa"

Il codice mostra già **cosa** fa. I commenti devono spiegare **perché** lo fa:

```
// Scadente: ridondante, il codice lo dice già
tentativi = tentativi + 1; // incrementa tentativi

// Buono: spiega il ragionamento
tentativi = tentativi + 1; // ogni risposta sbagliata aggiunge un tentativo
```

Documenta le funzioni complesse:

```
// Calcola il massimo comun divisore di due interi positivi.
// Precondizione: a > 0 e b > 0
// Usa l'algoritmo di Euclide (sottrazione ripetuta del resto).
int mcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```

Ogni funzione fa una sola cosa

Se una funzione è troppo lunga o fa troppe cose, spezzala. Una funzione ben fatta si capisce in pochi secondi:

```

// Scadente: una funzione enorme che legge, calcola e stampa tutto insieme
void elaboraStudenti() {
    // 100 righe con tutto mescolato...
}

// Buono: ogni funzione ha una responsabilità precisa
void leggiVoti(vector<int>& voti);
double calcolaMedia(const vector<int>& voti);
int trovaMassimo(const vector<int>& voti);
void stampaReport(const string& nome, double media, int max);

void elaboraStudenti() {
    vector<int> voti;
    leggiVoti(voti);
    double media = calcolaMedia(voti);
    int max = trovaMassimo(voti);
    stampaReport("Alice", media, max);
}

```

Evita i "numeri magici"

Un numero scritto direttamente nel codice senza spiegazione si chiama "numero magico". Chi lo legge non capisce cosa rappresenta:

```

// Scadente: cosa sono 86400 e 3.14159?
if (secondi > 86400) {
    area = 3.14159 * r * r;
}

// Buono: i nomi spiegano tutto
const int SECONDI_PER_GIORNO = 86400;
const double PI = 3.14159265358979;

if (secondi > SECONDI_PER_GIORNO) {
    area = PI * r * r;
}

```

Inizializza sempre le variabili

In C++, una variabile non inizializzata contiene un valore casuale (qualunque cosa ci fosse prima in quella zona di memoria):

```
// Pericoloso: valore casuale!
int x;
cout << x;    // stampa qualcosa di imprevedibile

// Sicuro
int x = 0;
cout << x;    // stampa 0
```

Valida l'input dell'utente

Non fidarti mai di quello che l'utente inserisce:

```
int eta;
cout << "Inserisci eta: ";
cin >> eta;

// Scadente: accetta qualsiasi valore, anche -500 o 9999
cout << "Hai " << eta << " anni";

// Buono: controlla prima di usare
if (eta < 0 || eta > 150) {
    cout << "Eta non valida!" << endl;
    return 1;
}
cout << "Hai " << eta << " anni";
```

Evita le variabili globali

Le variabili globali rendono il codice difficile da capire: chiunque, in qualunque punto del programma, può modificarle. Meglio passare i dati come parametri:

```
// Scadente: la variabile globale può essere modificata ovunque
int contatore = 0;
void incrementa() { contatore++; }

// Meglio: i dati vengono passati esplicitamente
void incrementa(int& contatore) { contatore++; }
```

Usa **const** per i dati che non cambiano

const comunica le intenzioni e previene modifiche accidentali:

```
// Parametri che la funzione non deve modificare
void stampa(const string& nome) {
    cout << nome << endl;
    // nome = "altro"; // il compilatore ti ferma se provi
}

// Valori fissi
const int MAX_STUDENTI = 30;
const double PI = 3.14159;
```

Verifica sempre le operazioni che possono fallire

Non assumere che un'operazione sia andata a buon fine:


```
// Scadente: apre il file senza verificare
ifstream file("dati.txt");
string riga;
getline(file, riga); // e se il file non esiste?

// Buono: controlla prima di usare
ifstream file("dati.txt");
if (!file.is_open()) {
    cerr << "Errore: impossibile aprire 'dati.txt'" << endl;
    return 1;
}
string riga;
getline(file, riga); // ora è sicuro
```

Non ripetere il codice (DRY)

DRY significa "Don't Repeat Yourself" — non ripeterti. Se scrivi lo stesso codice in più punti, estrailo in una funzione:

```
// Scadente: stessa struttura ripetuta
cout << "=====" << endl;
cout << "Alice: 8.5" << endl;
cout << "=====" << endl;
cout << "=====" << endl;
cout << "Bob: 7.2" << endl;
cout << "=====" << endl;

// Buono: una funzione riutilizzabile
void stampaScheda(const string& nome, double voto) {
    cout << "=====" << endl;
    cout << nome << ": " << voto << endl;
    cout << "=====" << endl;
}

stampaScheda("Alice", 8.5);
stampaScheda("Bob", 7.2);
```

Testa con casi diversi

Quando scrivi una funzione, testala con:

- Valori normali (caso standard)
- Valori limite (il minimo e il massimo consentiti)
- Valori non validi (cosa succede se riceve dati sbagliati?)

```
// Testa la funzione massimo con vari casi
cout << massimo(5, 3) << endl;    // 5 – caso normale
cout << massimo(-1, -5) << endl;  // -1 – numeri negativi
cout << massimo(7, 7) << endl;    // 7 – valori uguali
```

Debugging di Base

Tecniche e strumenti per trovare e correggere gli errori nei programmi C++.

Cosa significa fare debugging?

Tutti i programmatori scrivono codice con bug. Non è una questione di bravura — è normale. La differenza tra un programmatore esperto e uno alle prime armi spesso sta nella **velocità con cui trovano e correggono** gli errori.

Il debugging (eliminazione dei bug) è un'abilità che si impara con la pratica. Queste tecniche ti aiuteranno a trovare i problemi più velocemente.

I tre tipi di errori

- **Errori di compilazione:** il compilatore si rifiuta di creare il programma. Ti dice esattamente dove c'è il problema.
- **Errori di runtime:** il programma parte ma poi si blocca o crasha. Devi capire perché.
- **Errori logici:** il programma gira, non crasha, ma produce risultati sbagliati. I più subdoli.

Leggere i messaggi di errore del compilatore

Quando il compilatore trova un errore, ti dice esattamente dove:

```
hello.cpp:5:5: error: 'cout' was not declared in this scope
  cout << "Ciao!" << endl;
  ^~~~
```

Il formato è: `file:riga:colonna: tipo: messaggio`

- `hello.cpp` — il file con l'errore
- `5` — la riga dell'errore
- `error` — tipo di problema (`error` = grave, `warning` = avviso)
- Il messaggio spiega cosa c'è di sbagliato

Consiglio importante: correggi sempre il **primo** errore nella lista. Spesso un errore ne genera altri a cascata. Una volta sistemato il primo, molti degli altri spariscono.

I warning: segnali di pericolo

I warning non impediscono la compilazione, ma segnalano codice potenzialmente problematico:

```
warning: unused variable 'x' [-Wunused-variable]
```

Non ignorarli — spesso indicano errori logici nascosti.

Compila sempre con `-Wall` per vedere tutti i warning:

```
g++ programma.cpp -o programma -Wall
```

La tecnica più semplice: stampe di debug

Il modo più diretto per capire cosa fa il programma è aggiungere `cout` per stampare i valori delle variabili nei punti chiave:

```
double calcola(int a, int b) {
    cout << "[DEBUG] a=" << a << ", b=" << b << endl; // stampa i valori
    in entrata

    double risultato = a / b; // possibile divisione intera!

    cout << "[DEBUG] risultato=" << risultato << endl;

    return risultato;
}
```

Aggiungi il prefisso `[DEBUG]` così quando hai finito sai quali `cout` eliminare.

Gli errori più comuni

Variabile non inizializzata

```
int x;  
cout << x;    // stampa un numero casuale – bug!  
  
// Soluzione: inizializza sempre  
int x = 0;
```

Divisione intera indesiderata

```
int a = 7, b = 2;  
double risultato = a / b;    // 3.0, non 3.5! (divisione intera)  
  
// Soluzione: converti uno dei due prima di dividere  
double risultato = (double)a / b;    // 3.5
```

Errore di indice (off-by-one)

```
int arr[5] = {1, 2, 3, 4, 5};  
for (int i = 0; i <= 5; i++) {    // ERRORE: i arriva a 5, ma arr[5] non  
    cout << arr[i];  
}  
  
// Soluzione: usa < invece di <=  
for (int i = 0; i < 5; i++) {  
    cout << arr[i];  
}
```

= invece di **==** nelle condizioni

```
int x = 5;
if (x = 10) {    // ERRORE: assegna 10 a x, non confronta!
    cout << "uguale";
}

// Soluzione: usa == per confrontare
if (x == 10) {
    cout << "uguale";
}
```

Ciclo infinito

```
int i = 0;
while (i < 10) {
    cout << i;
    // manca i++! il ciclo non finisce mai
}

// Soluzione: aggiorna sempre la variabile del ciclo
while (i < 10) {
    cout << i;
    i++;
}
```

Il debugger in VS Code

VS Code ha un **debugger** integrato che ti permette di:

- Fermare il programma a una riga specifica (**breakpoint**)
- Vedere il valore di tutte le variabili in quel momento
- Eseguire il codice **una riga alla volta**

Come usarlo:

1. Clicca sul margine sinistro accanto al numero di riga per impostare un breakpoint (appare un punto rosso)
2. Premi **F5** per avviare il debug

3. Usa **F10** per eseguire riga per riga, **F11** per entrare dentro una funzione

Strategia di debugging

Quando hai un bug, segui questi passi:

1. **Riproduci il bug:** assicurati di riuscire a farlo apparire in modo affidabile
2. **Isola il problema:** dove esattamente inizia ad andare storto? Aggiungi stampe per scoprirlo
3. **Formula un'ipotesi:** cosa pensi stia causando il bug?
4. **Testa l'ipotesi:** modifica il codice e verifica se il bug sparisce
5. **Verifica che non hai rotto altro:** testa anche gli altri casi

Esempio: trovare un bug

```
// Programma con due bug: calcola la media di un array
#include <iostream>
using namespace std;

int main() {
    int voti[] = {8, 7, 9, 6, 10};
    int n = 5;
    int somma = 0;

    for (int i = 0; i <= n; i++) {    // BUG 1: <= invece di < → legge fuori
dall'array
        cout << "[DEBUG] i=" << i << ", voto=" << voti[i] << endl;
        somma += voti[i];
    }

    double media = somma / n;    // BUG 2: divisione intera → risultato
arrotondato
    cout << "Media: " << media << endl;
    return 0;
}

// Versione corretta:
// for (int i = 0; i < n; i++) {        ← usa < invece di <=
// double media = (double)somma / n;    ← converti prima di dividere
```

Con le stampe di debug, il bug 1 è visibile subito: quando `i=5`, `voti[5]` stampa un numero strano (non appartiene all'array).

Gestione degli Errori

Come gestire gli errori di base in C++ senza eccezioni avanzate.

Cosa succede quando il programma va in errore?

Tutti i programmi possono ricevere input sbagliati, trovare file mancanti o fare operazioni impossibili (come dividere per zero). Un programma che "esplode" senza spiegazioni è inutilizzabile.

La **gestione degli errori** è l'arte di intercettare queste situazioni e rispondere in modo sensato, senza far crashare il programma.

I tre tipi di errori

- **Errori di compilazione:** il codice non è scritto correttamente. Il compilatore li trova prima ancora di eseguire il programma (es. hai dimenticato un `;`).
- **Errori di runtime:** il codice è scritto bene, ma succede qualcosa di sbagliato durante l'esecuzione (es. divisione per zero, file non trovato).
- **Errori logici:** il programma gira senza crashare, ma produce risultati sbagliati. Sono i più difficili da trovare.

Gestione semplice: codici di ritorno

Un approccio comune è far restituire alle funzioni `true` se tutto è andato bene, `false` se c'è stato un errore:

```

// La funzione restituisce true se riesce, false se c'è un errore
bool dividi(int a, int b, int& risultato) {
    if (b == 0) {
        return false;    // errore: non si può dividere per zero
    }
    risultato = a / b;
    return true;
}

int main() {
    int risultato;

    if (dividi(10, 2, risultato)) {
        cout << "Risultato: " << risultato << endl;    // 5
    } else {
        cout << "Errore: divisione per zero!" << endl;
    }

    if (dividi(10, 0, risultato)) {
        cout << "Risultato: " << risultato << endl;
    } else {
        cout << "Errore: divisione per zero!" << endl;    // questo viene
stampato
    }

    return 0;
}

```

Validare l'input dell'utente

L'errore più comune è ricevere dati non validi dall'utente. Controlla sempre prima di usare:


```
int leggiEtaValida() {
    int eta;
    do {
        cout << "Inserisci la tua eta (0-120): ";
        cin >> eta;
        if (eta < 0 || eta > 120) {
            cout << "Eta non valida. Riprova." << endl;
        }
    } while (eta < 0 || eta > 120);
    return eta;
}
```

Controllare se `cin` ha letto correttamente

Se l'utente inserisce una lettera dove ti aspetti un numero, `cin` va in errore. Puoi verificarlo:

```
int numero;
cout << "Inserisci un numero: ";

if (cin >> numero) {
    cout << "Hai inserito: " << numero << endl;
} else {
    cout << "Errore: non hai inserito un numero valido." << endl;
    cin.clear(); // resetta lo stato di errore di cin
    cin.ignore(1000, '\n'); // scarta il testo non valido
}
```

Le eccezioni: per errori gravi

Le **eccezioni** sono il meccanismo del C++ per gestire errori che interrompono il flusso normale. Funzionano con tre parole chiave: `try`, `catch`, `throw`.

- `throw` lancia un errore
- `try` racchiude il codice da monitorare
- `catch` intercetta l'errore e gestisce la situazione

```

#include <stdexcept>
using namespace std;

double dividi(double a, double b) {
    if (b == 0) {
        throw runtime_error("Divisione per zero!");    // lancia l'errore
    }
    return a / b;
}

int main() {
    try {
        cout << dividi(10, 2) << endl;    // 5 - va bene
        cout << dividi(10, 0) << endl;    // lancia l'eccezione
    } catch (runtime_error& e) {
        // Il programma non crasha - gestiamo l'errore qui
        cout << "Errore catturato: " << e.what() << endl;
    }

    cout << "Il programma continua normalmente." << endl;
    return 0;
}

```

Tipi di eccezioni già pronti

La libreria `<stdexcept>` fornisce tipi di errore già pronti da usare:

```

#include <stdexcept>

throw runtime_error("Errore generico durante l'esecuzione");
throw invalid_argument("L'argomento passato non è valido");
throw out_of_range("Il valore è fuori dall'intervallo consentito");

```

Esempio pratico: radice quadrata sicura

```
#include <iostream>
#include <cmath>
#include <stdexcept>
using namespace std;

// Calcola la radice quadrata, ma solo per numeri non negativi
double radiceQuadrata(double x) {
    if (x < 0) {
        throw invalid_argument("Impossibile calcolare la radice di un
numero negativo");
    }
    return sqrt(x);
}

int main() {
    double valori[] = {16.0, -4.0, 25.0, 0.0};

    for (double v : valori) {
        try {
            cout << "sqrt(" << v << ") = " << radiceQuadrata(v) << endl;
        } catch (invalid_argument& e) {
            cout << "Errore per " << v << ": " << e.what() << endl;
        }
    }

    return 0;
}
```

Output:

```
sqrt(16) = 4
Errore per -4: Impossibile calcolare la radice di un numero negativo
sqrt(25) = 5
sqrt(0) = 0
```

Controllare l'apertura di un file

Prima di leggere o scrivere un file, verifica sempre che sia stato aperto:

```
#include <fstream>

ifstream file("dati.txt");

if (!file.is_open()) {
    cerr << "Errore: impossibile aprire il file 'dati.txt'" << endl;
    return 1;    // restituisce 1 per indicare che c'è stato un errore
}

// Qui sei sicuro che il file è aperto correttamente
```

`cerr` funziona come `cout`, ma manda il messaggio al canale degli errori — utile per distinguere i messaggi di errore dall'output normale.

Regole pratiche

- Valida sempre l'input dell'utente prima di usarlo
- Controlla sempre il risultato delle operazioni che possono fallire (apertura file, divisioni, ecc.)
- Non ignorare mai un errore in silenzio — il programma sembrerà funzionare ma darà risultati sbagliati
- Scrivi messaggi di errore chiari: l'utente deve capire cosa è andato storto
- Usa `cerr` per i messaggi di errore, non `cout`

Leggere e Scrivere File

Come leggere e scrivere file in C++ con `fstream`.

Leggere e scrivere file in C++

Quando il tuo programma termina, tutte le variabili spariscono. Se vuoi **conservare i dati** — i voti degli studenti, un punteggio di un gioco, una lista della spesa — devi salvarli su un file.

Con i file puoi anche leggere dati preparati in anticipo, senza dover inserire tutto a mano ogni volta.

Le tre classi per i file

In C++ si usa la libreria `<fstream>` (abbreviazione di "file stream"):

Classe	Per cosa serve
<code>ofstream</code>	Scrivere su file
<code>ifstream</code>	Leggere da un file
<code>fstream</code>	Leggere E scrivere

Scrivere su un file

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream file("saluto.txt");    // apre (o crea) il file in scrittura

    if (file.is_open()) {           // controlla sempre che sia aperto!
        file << "Ciao, mondo!" << endl;
        file << "Questa è la seconda riga." << endl;
        file.close();               // chiudi il file quando hai finito
        cout << "File scritto con successo." << endl;
    } else {
        cout << "Errore: impossibile aprire il file." << endl;
    }

    return 0;
}
```

Questo crea il file `saluto.txt` nella stessa cartella del programma. Se esiste già, viene **sovrascritto**.

Aggiungere testo senza cancellare

Per aggiungere righe in fondo a un file già esistente (senza cancellarlo), usa `ios::app` :

```
ofstream file("diario.txt", ios::app); // "app" = append (aggiungi in coda)

if (file.is_open()) {
    file << "Oggi ho imparato i file in C++." << endl;
    file.close();
}
```

Leggere da un file

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream file("saluto.txt"); // apre il file in lettura

    if (file.is_open()) {
        string riga;
        while (getline(file, riga)) { // legge una riga alla volta
            cout << riga << endl;
        }
        file.close();
    } else {
        cout << "Errore: file non trovato." << endl;
    }

    return 0;
}
```

`getline(file, riga)` legge una riga intera. Quando non ci sono più righe, il ciclo si ferma da solo.

Leggere parola per parola o numero per numero

Se il file contiene dati separati da spazi o da righe, puoi leggere con `>>` :

```

// Leggere parole
ifstream file("parole.txt");
string parola;
while (file >> parola) {    // legge una parola alla volta
    cout << parola << endl;
}
file.close();

// Leggere numeri e calcolare la media
ifstream file2("numeri.txt");
int numero, somma = 0, contatore = 0;
while (file2 >> numero) {
    somma += numero;
    contatore++;
}
if (contatore > 0) {
    cout << "Media: " << (double)somma / contatore << endl;
}
file2.close();

```

Leggere più valori per riga

Se ogni riga del file contiene più valori (es. nome e voto separati da spazio):

```

// Il file "dati.txt" contiene:
// Alice 8.5
// Bob 7.2
// Carlo 9.0

ifstream file("dati.txt");
string nome;
double voto;

while (file >> nome >> voto) {    // legge nome e voto insieme
    cout << nome << ": " << voto << endl;
}
file.close();

```

Esempio pratico: registro voti su file

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

// Chiede i voti all'utente e li salva nel file
void salvaVoti(const string& nomefile) {
    ofstream file(nomefile);
    if (!file.is_open()) {
        cout << "Errore nella creazione del file." << endl;
        return;
    }

    int n;
    cout << "Quanti studenti? ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        string nome;
        double voto;
        cout << "Nome studente " << (i+1) << ": ";
        cin >> nome;
        cout << "Voto: ";
        cin >> voto;

        file << nome << " " << voto << endl;    // scrive una riga per
    studente
    }

    file.close();
    cout << "Dati salvati in " << nomefile << endl;
}

// Legge il file e stampa i voti con la media
void leggiVoti(const string& nomefile) {
    ifstream file(nomefile);
    if (!file.is_open()) {
        cout << "File non trovato: " << nomefile << endl;
        return;
    }

    cout << "\n=== Voti salvati ===" << endl;
    string nome;
    double voto, somma = 0;
```



```

int contatore = 0;

while (file >> nome >> voto) {
    cout << nome << ": " << voto << endl;
    somma += voto;
    contatore++;
}

if (contatore > 0) {
    cout << "Media classe: " << somma / contatore << endl;
}

file.close();
}

int main() {
    string nomefile = "registro.txt";
    salvaVoti(nomefile);    // salva i voti su file
    leggiVoti(nomefile);    // rilegge e stampa
    return 0;
}

```

Regole pratiche

- Verifica **sempre** `file.is_open()` prima di usare il file
- Chiudi il file con `file.close()` quando hai finito
- Se dimentichi di chiudere, il file viene chiuso automaticamente quando l'oggetto viene distrutto (ma è buona abitudine chiuderlo esplicitamente)
- Per aggiungere contenuto senza cancellare, usa `ios::app`
- Usa `cerr` per i messaggi di errore relativi ai file, non `cout`

Librerie Standard

Le librerie standard del C++ più utili per iniziare a programmare.

Non devi inventare tutto da zero

Non devi inventare tutto da zero. Il C++ include una collezione enorme di strumenti già pronti — funzioni matematiche, gestione dei file, algoritmi di ordinamento e molto altro.

Si chiama **libreria standard** (Standard Library) ed è disponibile su ogni computer con un compilatore C++. Basta includerla con `#include` .

<iostream> — Input e Output

La libreria indispensabile: serve per stampare a schermo e leggere da tastiera.

```
#include <iostream>
using namespace std;

int main() {
    cout << "Ciao!" << endl;    // stampa a schermo

    int x;
    cin >> x;                    // legge un valore da tastiera

    cerr << "Errore!" << endl;  // stampa un messaggio di errore

    return 0;
}
```

- `cout` — stampa a schermo
- `cin` — legge da tastiera
- `cerr` — stampa messaggi di errore (canale separato)
- `endl` — va a capo

<string> — Testo

Per usare il tipo `string` e tutte le sue funzionalità:

```
#include <string>
using namespace std;

string nome = "Alice";
int lunghezza = nome.length();    // 5
string inizio = nome.substr(0, 3); // "Ali"

// Converta numero in stringa e viceversa
string s = to_string(42);    // "42"
int n = stoi("123");        // 123
double d = stod("3.14");    // 3.14
```

<cmath> — Funzioni Matematiche

Contiene funzioni matematiche avanzate (potenza, radice, arrotondamento, trigonometria):

```
#include <cmath>
using namespace std;

// Potenza: 2 elevato alla 10
cout << pow(2, 10) << endl;    // 1024

// Radice quadrata
cout << sqrt(25) << endl;      // 5
cout << sqrt(2) << endl;      // 1.41421

// Valore assoluto
cout << abs(-5) << endl;      // 5
cout << fabs(-3.14) << endl;  // 3.14 (per numeri decimali)

// Arrotondamento
cout << floor(3.7) << endl;    // 3 (arrotonda in giù)
cout << ceil(3.2) << endl;    // 4 (arrotonda in su)
cout << round(3.5) << endl;    // 4 (arrotonda al più vicino)

// Costante PI greco
cout << M_PI << endl;        // 3.14159265...
```

<cstdlib> e <ctime> — Numeri Casuali e Utilità

```
#include <cstdlib>
#include <ctime>
using namespace std;

// Inizializza il generatore di numeri casuali (fallo una sola volta)
srand(time(0));

int casuale = rand() % 100; // numero casuale tra 0 e 99
int dado = rand() % 6 + 1; // numero casuale tra 1 e 6

// Conversione da stringa a numero (alternativa a stoi/stod)
int n = atoi("42"); // stringa → intero
double d = atof("3.14"); // stringa → decimale
```

<cctype> — Funzioni sui Caratteri

Per controllare e trasformare singoli caratteri:

```
#include <cctype>
using namespace std;

cout << isupper('A') << endl; // 1 (vero): è maiuscolo?
cout << islower('a') << endl; // 1 (vero): è minuscolo?
cout << isdigit('5') << endl; // 1 (vero): è una cifra?
cout << isalpha('x') << endl; // 1 (vero): è una lettera?
cout << isspace(' ') << endl; // 1 (vero): è uno spazio?

cout << (char)tolower('A') << endl; // 'a'
cout << (char)toupper('a') << endl; // 'A'
```

<algorithm> — Algoritmi Pronti

Contiene algoritmi già implementati per lavorare con collezioni di dati:

```

#include <algorithm>
#include <vector>
using namespace std;

vector<int> v = {5, 2, 8, 1, 9, 3};

// Ordina dal più piccolo al più grande
sort(v.begin(), v.end());
// v è ora: {1, 2, 3, 5, 8, 9}

// Ordina dal più grande al più piccolo
sort(v.begin(), v.end(), greater<int>());
// v è ora: {9, 8, 5, 3, 2, 1}

// Trova il minimo e il massimo
cout << *min_element(v.begin(), v.end()) << endl; // 1
cout << *max_element(v.begin(), v.end()) << endl; // 9

// Cerca un elemento
auto it = find(v.begin(), v.end(), 5);
if (it != v.end()) {
    cout << "Trovato!" << endl;
}

// Inverte l'ordine degli elementi
reverse(v.begin(), v.end());

```

<vector> — Array Dinamici

Per usare array che cambiano dimensione (vedi la sezione dedicata):

```

#include <vector>
using namespace std;

vector<int> v = {1, 2, 3};
v.push_back(4); // aggiunge 4 alla fine
v.push_back(5);
cout << v.size() << endl; // 5

```

<limits> — Valori Massimi e Minimi dei Tipi

Per sapere i limiti di ogni tipo numerico:

```
#include <limits>
using namespace std;

cout << numeric_limits<int>::max() << endl;    // 2147483647 (il più grande
int)
cout << numeric_limits<int>::min() << endl;    // -2147483648 (il più
piccolo int)
cout << numeric_limits<double>::max() << endl; // ~1.8×10^308
```

Tabella riassuntiva

Libreria	Cosa contiene
<iostream>	cout, cin, cerr, endl
<string>	string, getline, stoi, stod, to_string
<cmath>	sqrt, pow, abs, floor, ceil, M_PI
<algorithm>	sort, find, min_element, max_element, reverse
<vector>	vector
<fstream>	ofstream, ifstream, fstream
<cstdlib>	rand, srand, atoi, atof
<cctype>	isupper, islower, isdigit, tolower, toupper
<limits>	numeric_limits
<ctime>	time